

앱 배포 가이드

24.0.0.1000

NEXACRO

이 문서에 잘못된 정보가 있을 수 있습니다. 투비소프트는 이 문서가 제공하는 정보의 정확성을 유지하기 위해 노력하고 특별한 언급 없이 이 문서를 지속적으로 변경하고 보완할 것입니다. 그러나 이 문서에 잘못된 정보가 포함되어 있지 않다는 것을 보증하지 않습니다. 이 문서에 기술된 정보로 인해 발생할 수 있는 직접적인 또는 간접적인 손해, 데이터, 프로그램, 기타 무형의 재산에 관한 손실, 사용 이익의 손실 등에 대해 비록 이와 같은 손해 가능성에 대해 사전에 알고 있었다고 해도 손해 배상 등 기타 책임을 지지 않습니다.

사용자는 본 문서를 구입하거나, 전자 문서로 내려 받거나, 사용을 시작함으로써, 여기에 명시된 내용을 이해하며, 이에 동의하는 것으로 간주합니다.

각 회사의 제품명을 포함한 각 상표는 각 개발사의 등록 상표이며 특허법과 저작권법 등에 의해 보호를 받고 있습니다. 따라서 본 문서에 포함된 기타 모든 제품들과 회사 이름은 각각 해당 소유주의 상표로서 참조용으로만 사용됩니다.

발행처 | (주)투비소프트

발행일 | 2026/03/05

주소 | (06083) 서울시 강남구 봉은사로 617 인탑스빌딩 2-5층

전화 | 02-2140-7700

홈페이지 | www.tobesoft.com

고객지원센터 | support.tobesoft.co.kr

제품기술문의 | 1588-7895 (오전 10시부터 오후 5시까지)

유지보수정책 | 유지보수기간과 범위는 제품 라이선스 계약에 따라 다릅니다.

변경 이력

버전	변경일	내용
24.0.0.1000	2026-03-04	UIScene 설정 (신규), UIScene 설정 (마이그레이션) 항목을 추가했습니다.
24.0.0.100.2	2024-04-01	App Builder (Android, iOS, macOS) 항목에 App Builder 관련 문서 링크를 추가했습니다.
24.0.0.100.1	2024-01-22	앱 개발 및 실행 (Android - Android Studio) 항목에 의존성 관리 설명을 추가했습니다.
24.0.0.100	2023-11-02	

차례

저작권 및 면책조항	2
변경 이력	3
차례	4

파트 I. 개요 9

1. 넥사크로 배포	10
1.1 개요	10
1.2 배포 파일	11
1.2.1 데스크탑 NRE	11
1.2.2 모바일 NRE	12
1.2.3 WRE	13
1.3 라이선스	13

파트 II. Windows App 15

2. 앱 배포 및 실행	16
2.1 주요 용어 설명	16
2.2 런처 서비스를 활용한 배포 및 앱 실행 데모	16
2.3 런처 서비스로 배포하기	20
2.3.1 서비스 페이지에서 런처 서비스 설치 파일 배포하기	20
2.3.2 런처 서비스 설치 파일 실행해서 설치하기	21
2.3.3 서비스 페이지에서 넥사크로 엔진 설치 파일 배포하기	22
2.3.4 넥사크로 엔진 설치 파일 실행해서 설치하기	23
2.4 앱 실행하기	24
2.4.1 넥사크로 엔진 운영 방식의 차이	24
2.4.2 기본 넥사크로 엔진만 사용	26
2.4.3 기본 넥사크로 엔진 + 단축 아이콘 만들기	28
2.4.4 Update(Local+Server) 옵션으로 네트워크 트래픽 줄이기	29

2.4.5	기본 넥사크로 엔진 + 설치 리소스(폴더, 단축 아이콘)	31
2.4.6	기본 넥사크로 엔진 + 설치 리소스 + Update(Local+Server) 옵션 적용하기	34
2.5	앱 업데이트하기	34
2.5.1	넥사크로 엔진 업데이트	35
2.5.2	Update Type이 Server인 경우	35
2.5.3	Update Type이 Update(Local+Server)인 경우	35
2.5.4	Update Type이 "Local"인 경우	36
2.6	NRE 로그 확인하기	36
2.7	Packing (Archive&Update) 메뉴	36
2.7.1	Packing 위저드 실행하고 아카이브 파일 만들기	36
2.7.2	실행 환경에 따른 Update Type 선택	38
2.7.3	실행 환경, Update Type에 따른 설정 옵션	39
2.8	Build App 메뉴	46
2.8.1	Build App 위저드 실행하고 설치 파일 만들기	46
2.9	참고	52
2.9.1	nexacro.exe 실행 옵션	52
2.9.2	운영체제 버전별 배포	53
2.9.3	단축 아이콘 Bootstrap URL 속성값 변경	54

파트 III. 런처 서비스 56

3.	설치 및 기본 사용	57
3.1	런처 서비스 설치	57
3.1.1	PC에 런처 서비스 설치하고 윈도우 서비스 등록하기	57
3.1.2	PC에서 런처 서비스 삭제하기	59
3.1.3	특정 포트에서 서비스가 실행하도록 지정하기	59
3.2	동작 방식	60
3.3	기본 설정	61
3.3.1	주소 설정	61
3.3.2	기본 요소 설정	61
3.3.3	데이터 전송	63
3.3.4	통신 결과 처리	64
3.4	TPLSvcAgent 사용	65
3.4.1	지원 기능	65
3.4.2	사용 방법	66
4.	지원기능 및 사용예제	67
4.1	지원기능	67
4.1.1	속성	67
4.1.2	메서드	69

4.1.3 이벤트	71
4.1.4 참고	71
4.2 사용예제	72
4.2.1 신규 id 요청	73
4.2.2 필수 속성 설정	74
4.2.3 메서드 실행	75
4.2.4 전체 코드	76

파트 IV. App Builder (Android, iOS, macOS) 78

5. App 생성	79
5.1 앱 기본 설정값 입력하기	80
5.1.1 Basic Property	81
5.1.2 Nexacro Library	81
5.1.3 Target OS	83
5.2 추가 옵션 설정하기	85
5.2.1 Basic Property Options	85
5.2.2 Target OS > Android Options	86
5.2.3 Target OS > iOS Options	89
5.2.4 Target OS > macOS Options	91
5.2.5 Target OS > Icon Generate를 사용해 이미지 파일 생성하기	92
5.2.6 Target OS > 프로젝트 파일 내보내고 가져오기	93
5.2.7 Target OS > User Library	94
5.3 앱 빌드하기	95
6. App 목록	97

파트 V. iOS(iPadOS)/Android/macOS App 98

7. 앱 개발 및 실행 (iOS/iPadOS - Xcode)	99
7.1 앱 개발 환경 설정	99
7.2 앱 프로젝트 개발	99
7.2.1 프로젝트 생성	99
7.2.2 넥사크로 라이브러리 설정	101
7.2.3 리소스 설정	102
7.2.4 Config 설정	105
7.2.5 빌드 환경 설정	107
7.2.6 네트워크 연결 없는 앱 실행 설정	110
7.3 앱 테스트	112
7.4 UIScene 설정 (신규)	112
7.4.1 SceneDelegate 설정	113

7.5	UIScene 설정 (마이그레이션)	114
7.5.1	프레임워크 업데이트	115
7.5.2	SceneDelegate 설정	115
7.5.3	Scene 설정 추가	116
7.5.4	Info.plist 설정 확인	117
8.	앱 개발 및 실행 (Android - Android Studio)	118
8.1	앱 개발 환경 설정	118
8.2	앱 프로젝트 개발	118
8.2.1	프로젝트 생성	119
8.2.2	넥사크로 라이브러리 설정	120
8.2.3	리소스 설정	123
8.2.4	Config 설정	125
8.2.5	빌드 환경 설정	127
8.2.6	네트워크 연결 없는 앱 실행 설정	135
8.3	빌드	137
8.3.1	앱 테스트	137
8.3.2	설치 파일 생성	138
9.	앱 개발 및 실행 (macOS - Xcode)	140
9.1	준비 사항	140
9.1.1	개발 환경	140
9.2	앱 프로젝트 개발	140
9.2.1	프로젝트 생성	141
9.2.2	넥사크로 라이브러리 설정	142
9.2.3	리소스 설정	143
9.2.4	빌드 환경 설정	143
부록 A.	경로설정	146
A.1	Alias 경로	146
A.2	nexacro.xml	147
A.3	상대경로	147
A.3.1	Project 내 경로의 상대경로 지원여부	147
부록 B.	예외상황	149
B.1	웹브라우저 옵션	149
B.1.1	자바스크립트 활성화	149
B.1.2	파일 다운로드 활성화	149
부록 C.	NRE 로그 확인하기	151
C.1	로그 파일 생성 설정하기	151
C.1.1	Environment 항목에서 로그 파일 생성 설정하기	151
C.1.2	config 파일 설정하기	152

C.2 nexacro_config.xml 파일 속성	153
C.3 속성별 파일 생성 예시	155
C.3.1 backupfilecount	155
C.3.2 traceduration	156
C.4 로그 파일 읽기	157
C.4.1 로그 파일 읽기 스크립트 예제	157
C.4.2 윈도우 운영체제 실행 시 제약	158
부록 D. NRE 원격 디버깅	159
D.1 NRE 원격 디버깅	159
D.1.1 프로젝트 설정	159
D.1.2 실행 설정	160
D.1.3 Devices inspect 기능 실행	160
D.1.4 로딩 중 멈춤 기능 옵션 설정	163
부록 E. Cordova 플러그인 적용	165
E.1 넥사크로 스튜디오	165
E.2 Android - Android Studio	165
E.3 iOS/iPadOS - Xcode	166

파트 I.

개요

1.

넥사크로 배포

사용자가 넥사크로로 개발된 앱을 사용하려면 사용 환경에 따라 적절한 실행 환경을 만들어주고 필요한 파일을 내려 받아야 합니다. 이러한 과정을 배포라고 합니다. 배포된 파일은 캐시 정책에 따라 내부에서 관리되며 필요할 때 새로운 파일로 교체됩니다.

데스크탑, 모바일 환경에 이미 설치된 웹브라우저에서 넥사크로 앱을 실행하는 경우에는 웹서버에서 필요한 HTML, 자바스크립트, CSS 파일만 내려받아 바로 실행합니다. 하지만 NRE 방식에서는 별도의 배포, 설치 과정이 필요합니다. 세부적인 과정은 각 실행 환경에 따라 달라질 수 있습니다.

1.1 개요

넥사크로 앱이 실행하기 위해 필요한 파일을 먼저 살펴보겠습니다. 실행 환경에 따라 배포되는 파일은 아래와 같습니다.

분류	설명	NRE	WRE
런타임 엔진	스크립트, 메모리, 렌더링 처리 런타임 엔진	O	X
넥사크로 프레임워크	프레임워크, 컴포넌트 라이브러리 (자바스크립트)	O	O
코드	제너레이터에서 변환된 코드 (자바스크립트, CSS)	O	O
라이선스	라이선스 파일	O	O

NRE의 가장 큰 특징은 런타임 엔진입니다. 웹브라우저 환경에서는 필요한 프레임워크, 컴포넌트, 코드를 바로 사용하는 데 반해 NRE를 사용하려면 각 실행 환경에 맞는 런타임 엔진을 별도로 배포해주어야 합니다.

런타임 엔진을 추가로 배포하는 작업이 사용자로서도 불편할 수 있지만, 시스템 환경에 따라 기존에 사용하던 시스템과 연계가 필요하거나 웹브라우저에서 제공하지 못하는 확장된 기능을 사용하기 위해서는 넥사크로 브라우저를 선택할 수 있습니다.



넥사크로에서 제공하는 기본적인 기능은 버전에 상관없이 같지만 디바이스 API나 외부 기기 연동 등 확장 기능은 실행 환경에 따라 다를 수 있습니다.

1.2 배포 파일

버전별 배포되는 파일을 상세하게 살펴보면 아래와 같습니다.



파일 위치 또는 서버 위치를 설명하는 Alias는 [Alias 경로](#)를 참고해주세요.



아래 설명된 파일명과 제공되는 라이브러리는 업데이트에 따라 변경될 수 있습니다.
배포 옵션에 따라 생성되는 파일 목록이 달라질 수 있습니다.

1.2.1 데스크탑 NRE

윈도우 NRE

넥사크로 런타임 엔진 배포 후 실행 방식에 따라 앱을 호출합니다. 콘텐츠는 같지만 프레임워크 자바스크립트 파일을 서버에서 직접 받지 않고 dll 파일로 제공된 라이브러리에 포함해 배포합니다.

분류	파일	설명	파일 위치
Runtime Engine	v8.dll	V8 Script Engine Library	
	XMemLib.dll	Memory Management Library	
	XBasicLib.dll	Base Libaray	
	nexacrolib.dll	Platform & Render Engine Library	
	nexacro.exe	넥사크로 Executor	
Framework DLL	XFrameworkLib.dll	Framework API Library	
Extend DLL Files	protocol adaptor dll	Network Protocol Adaptor	%UPDATE%
	External DLL	External DLLs	%UPDATE%
Runtime Config Files	nexacro.xml	Runtime 실행 환경 파일 & UserProfile	%USERAPP%
Log Files	nexacro_XXXXX.log	로그 파일	%USERAPP%\Log
Launcher	splash.png	Splash Image File	%USERAPP%
Download Files	loadingimage.png	WaitCursor Image File	%USERAPP%

분류	파일	설명	파일 위치
	nexacro.ico	Application Icon File	%USERAPP%
	globalvars.dat	Global Variable List 파일	%USERAPP%



배포되는 파일 목록은 엔진 버전이나 운영체제에 따라 달라질 수 있습니다.
위의 표에서는 주요 파일 목록만 나열했습니다.

macOS NRE

macOS 런타임은 각 운영체제에 따라 별도 패키징 과정이 필요합니다. 제너레이트에서 변환된 코드를 Archive 파일로 생성하고 각 운영체제에 맞게 배포 파일을 생성하게 됩니다.

분류	파일	파일 설명	Packing
macOS App Files	nexacroN.macOS.framework	macOS Library Framework	macOS App
	start_macos.json	macOS Archive Information	
	localizable.strings	macOS Locale String Information	
Execute HTML	index.html		Run Archive
Framework JS			Engine Archive
Component JS			Engine Archive
Resource Files	loadingimage.png		Engine Archive
Theme File	theme_*.css		Theme Archive
Application Files			Application Archive

1.2.2 모바일 NRE

안드로이드 NRE, iOS NRE

iOS/안드로이드 앱은 각 운영체제에 따라 별도 패키징 과정이 필요합니다. 제너레이트에서 변환된 코드를 Archive 파일로 생성하고 각 운영체제에 맞게 배포 파일을 생성하게 됩니다.

분류	파일	파일 설명	Packing
Android App Files	Acceleration-release.aar	Android Archive	Android App
	...		
	star_android.json	Android Archive Information	
	strings.xml	Android Locale String Information	
iOS App Files	nexacro.framework	iOS Library Framework	iOS App
	start_ios.json	iOS Archive Information	
	localizable.strings	iOS Locale String Information	

분류	파일	파일 설명	Packing
Execute HTML	index.html		Run Archive
Framework JS			Engine Archive
Component JS			Engine Archive
Resource Files	loadingimage.png		Engine Archive
Theme File	theme_*.css		Theme Archive
Application Files			Application Archive

1.2.3 WRE

필요한 모든 파일을 웹브라우저에서 직접 로딩하는 방식으로 실행됩니다. 사용하는 웹브라우저에 따라 성능 차이가 있을 수 있습니다.

분류	파일	설명	서버 위치
Execute HTML	index.html		%WEBDEPLOY PROJECT%
Framework JS	BasicObjs.js		%WEBDEPLOY FRAMEWORK%
	SystemBase.js	System Utility	%WEBDEPLOY FRAMEWORK%
	Platform.js	Platform Objects	%WEBDEPLOY FRAMEWORK%
	CssObjs.js	Style Object	%WEBDEPLOY FRAMEWORK%
	ErrorDefine.js	Error Information	%WEBDEPLOY FRAMEWORK%
Component JS	CompBase.js	Component Base Module	%WEBDEPLOY COMPONENT%
	ComComp.js	Common Component Module	%WEBDEPLOY COMPONENT%
	Grid.js	Grid Component Module	%WEBDEPLOY COMPONENT%
	DeviceAPI.js	DeviceAPI Module	%WEBDEPLOY COMPONENT%
Resource Files	waitimage.gif	WaitCursor Image File	%WEBDEPLOY RESOURCE%
Theme File	theme_*.css	Deploy 된 테마 소스	%WEBDEPLOY THEME%
Application Files		Deploy 된 Application 구성 파일	%WEBDEPLOY PROJECT%

1.3 라이선스

넥사크로 앱을 사용자에게 배포하고 사용하기 위해서는 라이선스 정보가 필요합니다. 적절한 라이선스 정보가 없는 경우에는 앱 상단에 경고 메시지를 표시합니다.

상태	설명	동작
라이선스 없음	Application 오브젝트의 licenseurl 속성값을 지정하	오류 메시지 출력

상태	설명	동작
	지 않았거나 해당 경로에 라이선스 파일이 없음	
제품 정보 불일치	라이선스에 명시된 제품 정보가 다름	오류 메시지 출력
사용자 정보 불일치	라이선스에 명시된 IP 주소가 호스트 IP 주소와 다름	오류 메시지 출력
기능 정보 불일치	라이선스에 허가되지 않은 기능 사용	오류 메시지 출력
실행 환경 불일치	라이선스에 허가되지 않은 실행 환경 사용	오류 메시지 출력
기간 불일치	라이선스에 명시된 개시일 전에 사용하거나 만료일 이후에 사용	오류 메시지 출력
파일 변조	라이선스 정보를 임의 변경	오류 메시지 출력
통신 오류	licenseurl 접근 시 통신 오류	오류 메시지 출력

아래와 같은 경우는 라이선스를 체크하지 않습니다.

- 넥사크로 스튜디오에서 QuickView 실행 시
- 시작 파일 경로가 로컬 파일(file://)로 실행 시
- localhost(127.0.0.1)로 실행 시

파트 II.

Windows App

2.

앱 배포 및 실행

이번 장에서는 윈도우 환경에서 넥사크로 앱을 배포하는 방법을 설명합니다.

사용자가 앱 설치 파일을 내려받아 설치한 후 바탕화면에 만들어진 단축 아이콘을 클릭해 앱을 실행할 수도 있고, 런처 서비스를 설치한 후 서비스 페이지에서 [실행] 버튼을 클릭해 앱을 실행할 수도 있습니다.

2.1 주요 용어 설명

넥사크로 엔진

윈도우 운영체제에서 넥사크로 앱을 실행하기 위한 실행환경을 의미합니다. 운영체제 버전에 따라 32비트, 64비트 넥사크로 엔진을 설치할 수 있습니다. 기본 넥사크로 엔진은 변경 없이 그대로 사용하며 설치 리소스를 사용하거나 앱 설치 파일을 사용하는 경우에는 Engine Setup Key와 설치 경로를 따로 지정할 수 있습니다.

런처 서비스

윈도우 서비스로 동작하면서 지정한 넥사크로 엔진을 통해 앱을 실행하는 기능을 수행합니다. 기본 넥사크로 엔진을 사용하는 경우 단축 아이콘을 만들거나 설치 리소스를 같이 배포하는 기능을 처리할 수 있습니다.

2.2 런처 서비스를 활용한 배포 및 앱 실행 데모



이번 장에서는 런처 서비스를 활용한 배포 및 앱 실행 방식을 설명하기 위해 기본적인 기능만 다룹니다. 사용자 환경에 따른 예외 조건을 고려한 스크립트는 아래 링크에 있는 HTML, Javascript 템플릿 파일을 참고하세요.

https://github.com/TOBESOFT-DOCS/sample_Nexacro_N_windows_deploy/tree/master/Sample



데모에서 사용한 프로젝트와 파일은 아래 링크에서 내려받을 수 있습니다.

https://github.com/TOBESOFT-DOCS/sample_Nexacro_N_windows_deploy

런처 서비스 설치 및 실행		넥사크로 기본 엔진		넥사크로 기본 엔진 개별 설치	
(1) 런처 서비스 설치 파일 배포 런처 서비스 실행 후 ID 생성		(3) 기본 넥사크로 엔진 + setproperty		(7) 기본 넥사크로 엔진 개별 사용 + setproperty	
(2) 넥사크로 엔진 설치 파일 배포 넥사크로 엔진 버전 확인		(4) 애플리케이션 실행하기		(8) SetupResource.zip 파일 내려받기	
☒ XHR log globalvalue 설정값		(5) 단축 아이콘 만들기	(6) 단축 아이콘 삭제하기	(9) 넥사크로 엔진 설치 파일 내려받기	

실행 결과

```

(1) Button Click
launcher service id :1629870000
launcher service version: 1,1,0,28
(2) Button Click
nexacro engine version: 21.0.0.100
(3) Button Click
result: success
{"action":"setproperty","id":"1629870000","platform":"nexacro","result":"success","value":{"bjson":{"http://127.0.0.1:4098/_w
indows_/start.json"},"enginesetupkey":{"3E221E7D-2ECF-4EF4-AD16-C69972112C86"},"globalvalue":{"gvTest.av=","key":"TE
ST"},"screenid":"windows_sub_screen"}}
{"action":"event","id":"1629870000","platform":"nexacro","result":"success","value":{"error":{"code":0,"message":""}}}
(5) Button Click
result: success
{"action":"method","id":"1629870000","platform":"nexacro","result":"success","value":{"makeshortcut":{"param":{"test","htt
p://127.0.0.1:4098/_windows/_resource/_images/_sample_test.ico"},"%USERAPP%","desktop",true}}}]
{"action":"event","id":"1629870000","platform":"nexacro","result":"success","value":{"error":{"code":0,"message":""}}}
    
```

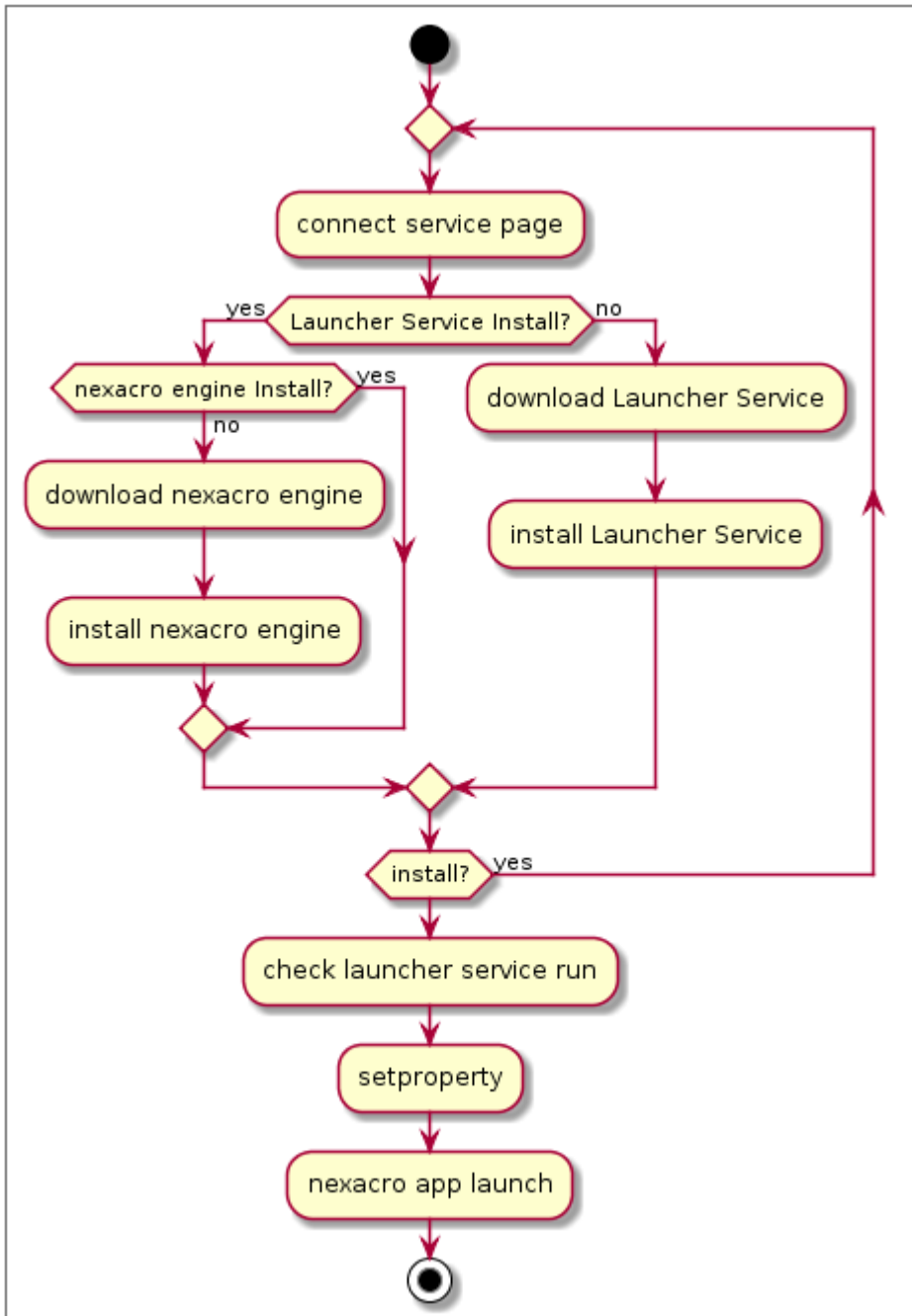
각 버튼은 설치 여부에 따라 동작합니다. 아래 표 설명을 참고해주세요.

번호	동작	설명
런처 서비스 설치 및 실행		
1	런처 서비스 설치 파일 배포 런처 서비스 실행 후 ID 생성	런처 서비스 설치/동작 여부를 확인하고 설치되어 있지 않은 경우에는 설치 파일을 내려받습니다. 서비스 페이지에서 런처 서비스 설치 파일 배포하기 런처 서비스가 설치되어 동작하고 있는 경우에는 ID를 생성하고 반환합니다. 런처 서비스 버전과 ID를 확인할 수 있습니다. 런처 서비스 설치 파일 실행해서 설치하기
넥사크로 엔진 설치 및 앱 실행		
2	넥사크로 엔진 설치 파일 배포 넥사크로 엔진 버전 확인	넥사크로 엔진 설치 여부를 확인하고 설치되어 있지 않은 경우에는 설치 파일을 내려받습니다. 서비스 페이지에서 넥사크로 엔진 설치 파일 배포하기 넥사크로 엔진이 설치된 경우에는 버전 정보를 확인할 수 있습니다. 넥사크로 엔진 설치 파일 실행해서 설치하기



이번 장 설명에서는 사용자에게 따른 분기 처리를 하지 않았습니다.
사용자가 64비트 윈도우를 사용하고 있다는 설정으로 작성했습니다.

런처 서비스 사용 시 아래와 같은 순서에 따라 동작합니다.



2.3 런처 서비스로 배포하기

런처 서비스는 리소스 배포, 앱 실행 등의 관리를 위해 사용자 PC에 설치 후 백그라운드로 동작하는 서비스입니다. 런처 서비스가 동작하기 위해서는 런처 서비스 설치 파일을 배포하고 사용자가 설치하는 과정이 필요합니다.

2.3.1 서비스 페이지에서 런처 서비스 설치 파일 배포하기



서비스 페이지는 필요한 파일을 배포하고 실행하기 위한 웹페이지를 의미합니다. 페이지 로딩 시 스크립트를 처리할 수도 있으며 사용자가 특정 버튼 클릭 시 동작하도록 구성할 수 있습니다. 아래 예제에서는 사용자가 버튼을 클릭했을 때 스크립트가 동작하도록 구성했습니다.

사용자	서비스 페이지
① 서비스 페이지에 접속합니다.	② 런처 서비스 URL을 호출합니다. <code>xhr.open("POST", send_url, true);</code>
	③ 런처 서비스 URL 호출 시 응답이 없으면 런처 서비스가 설치되어 있지 않다고 판단합니다. <code>if(xhr.status==0) {</code>
	④ 런처 서비스 설치 파일을 내려받습니다.



런처 서비스 URL은 아래 형식으로 사용합니다.

`http://127.0.0.1:[PORT]/launcher/nexacro/[TIMESTAMP]`

[PORT]는 런처 서비스 설치 시 설정된 포트입니다.

[TIMESTAMP]는 웹브라우저 캐시 동작으로 오류 발생을 방지하기 위해 사용합니다.

```
var objNexacro = {};
objNexacro.id = '';
var xhr = null;
var that = this;

this.Button00_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    objNexacro.platform = 'nexacro';
    objNexacro.action = 'create';

    xhr = new XMLHttpRequest();
```

```

xhr.reqType = 1;
xhr.onreadystatechange = function()
{
    if(xhr.readyState === 4) {
        if(xhr.status==0) {
            that.FileDownloadTransfer00.download(window.location.protocol + "://" + window.
location.host + "/Service/TPLSvc_Setup.exe");
        }
        else
        {
            var objResult = JSON.parse(xhr.responseText);
            if(objResult.result == "success" && objResult.id.length > 0) {
                objNexacro.id = objResult.id;
                that.fn_addvalue("launcher service id :"+objNexacro.id);
            }
        }
        checkLaunchVersion();
    }
}

var send_url = "http://127.0.0.1:7895/launcher/nexacro/"+new Date().getTime();
xhr.open("POST", send_url, true);
xhr.send(JSON.stringify(objNexacro));
};

```

2.3.2 런처 서비스 설치 파일 실행해서 설치하기

사용자는 런처 서비스 설치 파일을 내려받고 직접 런처 서비스 설치 파일을 실행해 런처 서비스를 설치해야 합니다. 설치 후 서비스 페이지에 다시 접속하는 것은 별도로 안내해주어야 합니다.

사용자	서비스 페이지
❶ 내려받은 런처 서비스 설치 파일을 실행합니다.	
❷ 런처 서비스를 설치합니다.	
❸ 서비스 페이지에 다시 접속합니다.	
	❹ 런처 서비스 URL을 호출합니다.
	❺ 런처 서비스 URL이 정상적으로 요청되면 id 값을 포함한 응답을 반환합니다.

런처 서비스 실행 여부를 확인하고 id 속성값을 저장합니다. id 속성값은 이후 진행되는 런처 서비스와의 통신 시 키

값으로 사용합니다.

```
...
    var objResult = JSON.parse(xhr.responseText);
    if(objResult.result == "success" && objResult.id.length > 0) {
        objNexacro.id = objResult.id;
    }
...

```



사용 환경에 따라 기본 포트(7895)를 사용할 수 없는 경우에는 7895~7935 사이의 포트를 사용합니다. 예제에서는 기본 포트(7895)를 사용할 수 있다는 전제로 설명하고 있습니다.

2.3.3 서비스 페이지에서 넥사크로 엔진 설치 파일 배포하기

앱 실행에 필요한 넥사크로 엔진이 설치되어 있는지 확인합니다. ProductKey와 엔진버전이 일치하는 경우에는 다음 단계로 진행하고 일치하지 않은 경우에는 설치 파일을 내려받습니다.

사용자	서비스 페이지
❶ 서비스 페이지에 접속합니다.	
	❷ 런처 서비스 URL을 호출합니다.
	❸ 런처 서비스 URL이 정상적으로 요청되면 id 값을 포함한 응답을 반환합니다.
	❹ 넥사크로 엔진 설치 여부를 확인합니다.
	❺ 넥사크로 엔진이 설치되어 있지 않거나 버전이 낮은 경우에는 넥사크로 엔진 설치 파일을 내려받습니다. if(!strEngine)

ProductKey에 해당하는 엔진버전 정보가 있는 경우에는 값을 반환하고 그렇지 않은 경우에는 빈값을 반환합니다.

```
this.Button01_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    this.fn_addvalue("(2) Button Click");
    if(objNexacro.id=='')
    {
        that.fn_addvalue('launcher service id was not created');
        return false;
    }
    objNexacro.action = "method";

```

```
objNexacro.value = { "getengineversion": { "param": [{"3E221E7D-2ECF-4EF4-AD16-C69972112C86}"], "result": null } };
```

```
xhr = new XMLHttpRequest();
xhr.reqType = 1;
xhr.onreadystatechange = function()
{
    if(xhr.readyState === 4) {
        var resultJson = JSON.parse(xhr.responseText);
        var reg = new RegExp("21.0.0.100", "i");
        var rtnEngineVer = resultJson.value["getengineversion"].result;
        that.fn_addvalue("nexacro engine version: "+rtnEngineVer);
        var strEngine = reg.test(rtnEngineVer);
        if(!strEngine)
        {
            that.FileDownloadTransfer00.download(window.location.protocol + "://" + window.location.host + "/Service/NexacroN_SetupRuntime_x64.exe");
        }
    }
}

var send_url = "http://127.0.0.1:7895/launcher/nexacro/"+new Date().getTime();
xhr.open("POST", send_url, "true");
xhr.send(JSON.stringify(objNexacro));
};
```



넥사크로 엔진 설치 파일은 운영체제에 따라 2가지(32비트, 64비트)를 지원합니다. 사용자 운영체제가 불특정한 경우에는 운영체제에 따라 적절한 파일을 내려받을 수 있도록 합니다. 아래 문서를 참고하세요.
[운영체제 버전별 배포](#)

2.3.4 넥사크로 엔진 설치 파일 실행해서 설치하기

사용자가 넥사크로 엔진 설치 파일을 내려받은 이후에는 사용자가 직접 넥사크로 엔진 설치 파일을 실행해 넥사크로 엔진을 설치해야 합니다. 서비스 페이지에 다시 접속하는 것은 별도로 안내해주어야 합니다.

사용자	서비스 페이지
① 내려받은 넥사크로 엔진 설치 파일을 실행합니다.	
② 넥사크로 엔진을 설치합니다.	

사용자	서비스 페이지
③ 서비스 페이지에 다시 접속합니다.	
	④ 런처 서비스 URL을 호출합니다.
	⑤ 런처 서비스 URL이 정상적으로 요청되면 id 값을 포함한 응답을 반환합니다.
	⑥ 넥사크로 엔진 설치와 버전을 확인합니다.

2.4 앱 실행하기

프로젝트의 특성에 따라 배포할 파일을 만드는 방법이 달라집니다. 단축 아이콘을 생성해야 하는지, 설치 폴더를 구분해야 하는지, 앱마다 다른 버전의 엔진을 사용해야 하는지 등에 따라 운영 방식을 선택할 수 있습니다. 아래 표에서 기능 차이를 확인해주세요.

2.4.1 넥사크로 엔진 운영 방식의 차이

기능 차이

	기본 넥사크로 엔진	기본 넥사크로 엔진 + SetupResource.zip	앱 설치 파일 (*.exe)
설치 파일 생성	X 기본 넥사크로 엔진 사용	X 기본 넥사크로 엔진 사용	O
설치 파일 디지털 서명	투비소프트	투비소프트	X 디지털 서명 별도 처리
Engine Setup Key	변경할 수 없음	변경 권장	변경 권장
Build App 메뉴	사용하지 않음	O	O
단축 아이콘 생성	△ 런처 서비스를 통해 생성	O	O
스플래시 이미지 지정	△ 런처 서비스를 통해 생성	O	O
설치 폴더 지정	X	O	O



생성된 설치 파일을 디지털 서명 없이 실행 시 Publisher 정보는 "Unknown Publisher"로 표기됩니다. 시스템 환경에 따라 Publisher 정보가 확인되지 않은 경우 앱을 설치할 수 없는 경우도 있을 수 있습니다.



마이크로소프트에서 배포하는 SignTool.exe과 같은 도구를 사용해 디지털 서명을 처리할 수 있습니다.
[https://msdn.microsoft.com/en-us/library/windows/desktop/aa387764\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/aa387764(v=vs.85).aspx)

Deploy 메뉴 설정 차이

	기본 넥사크로 엔진	기본 넥사크로 엔진 설치 리소스	넥사크로 엔진 포함 앱 설치 파일
Packing (Archive&Update)			
NRE (Windows)	선택	선택	선택하지 않음
Setup Filename	변경할 수 없음	변경할 수 없음	
Engine Setup Key	변경할 수 없음	변경할 수 없음	
Version	변경할 수 없음	변경할 수 없음	
Build App			
Install Location	Build App 메뉴 사용하지 않음	Program Files 폴더 기준 상대 경로 입력	
Bootstrap URL		Update Type이 Server, Update인 경우에는 웹 서버에 배치한 start.json URL 입력	
Setup Filename		변경할 수 있음	
Engine Setup Key		Install Location 변경 시 새로운 키 생성을 권장	

Deploy 메뉴에서 생성해 배포해야 하는 파일

Packing (Archive&Update)	
Update, Local	Archive.xzip nexacrolib.zip Resource.zip start.json
Server	_resource_ 폴더 [Service 그룹] 폴더 [nexacrolib] 폴더 [프로젝트명].xadl.js environment.xml.js nexacrolib.zip start.json
Build App	
기본 넥사크로 엔진, 설치 리소스 (Packing 단계에서 NRE 선택한 경우)	NexacroN_SetupRuntime[_운영체제버전].exe SetupResoure.zip

	ProjectResource.zip
넥사크로 엔진 포함, 앱 설치 파일 (Packing 단계에서 NRE 선택하지 않은 경우)	NexacroN_SetupRuntime[_운영체제버전].exe

2.4.2 기본 넥사크로 엔진만 사용

넥사크로 플랫폼에서 제공하는 기본 넥사크로 엔진을 사용합니다. 사용자 PC에 이미 설치된 경우에는 별도 설치할 필요가 없습니다. 넥사크로 스튜디오 메뉴[Deploy > Packing]에서 Update Type을 "Server"로 선택하고 파일을 생성한 후 서버에 올려주면 런처 서비스에서 바로 실행할 수 있습니다.

사용자	서비스 페이지	서버
① 서비스 페이지에 접속합니다.		
	② 런처 서비스를 호출하고 실행 여부를 확인합니다.	
	③ 넥사크로 엔진 설치와 버전을 확인합니다.	
	④ key, bjson, enginesetupkey 속성값을 설정합니다.	
	⑤ 앱을 실행합니다.	
		⑥ start.json 파일을 체크합니다.
		⑦ start.json 파일에 지정된 콘텐츠를 처리합니다.

아래 코드에서 bjson 속성값은 서버에 있는 start.json 파일의 경로를 지정해줍니다.



enginesetupkey 속성값은 제품 버전에 따라 달라질 수 있으며, 배포 방식에 따라 앱에 따라 달라질 수 있습니다.

- 제품 버전에 따른 enginesetupkey 속성값은 제품과 같이 배포되는 버전 정보 문서를 참조해주세요.
- enginesetupkey를 개별 생성한 앱의 경우에는 해당 앱 배포 담당자에게 문의해주세요.

```
this.Button02_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    objNexacro.action = 'setproperty';
    objNexacro.value = {
        "key": "TEST",
        "bjson": window.location.protocol + "://" + window.location.host+"/_windows_/start.json"
    },
    "screenid": "windows_sub_screen",
```

```

        "globalvalue": "gvTest:av="+this.Edit00.text,
        "enginesetupkey": "{3E221E7D-2ECF-4EF4-AD16-C69972112C86}"
    };
    this.sendData();
};

this.Button03_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    objNexacro.action = 'method';
    objNexacro.value = {
        "launch": null
    };
    this.sendData();
};

this.sendData = function()
{
    xhr = new XMLHttpRequest();
    xhr.reqType = 1;
    xhr.onreadystatechange = function()
    {
        if(xhr.readyState === 4) {
            var resultJson = JSON.parse(xhr.responseText);
            if(resultJson.action == "event")
            {
                if(resultJson.value["error"].code!=0)
                {
                    that.fn_addvalue("error code: "+resultJson.value["error"].code);
                    that.fn_addvalue("error message: "+resultJson.value["error"].message);
                }
            }
            else
            {
                that.fn_addvalue("result: "+resultJson.result);
            }
            if(that.CheckBox00.value)
                that.fn_addvalue(xhr.responseText);
            if(resultJson.action != "event")
                that.get_event();
        }
    }
}

```

```

}

var send_url = "http://127.0.0.1:7895/launcher/nexacro/"+new Date().getTime();
xhr.open("POST", send_url, "true");
xhr.send(JSON.stringify(objNexacro));
}

```

2.4.3 기본 넥사크로 엔진 + 단축 아이콘 만들기

기본 넥사크로 엔진을 사용하는 경우에는 앱을 설치하지 않고 start.json 경로만 지정하고 실행하는 방식이기 때문에 단축 아이콘을 만들 수 없습니다. 처음에만 서비스 페이지에서 실행하고 다음부터는 단축 아이콘을 통해 실행하도록 한다면 아래와 같이 런처 서비스의 메소드를 이용해 단축 아이콘을 생성할 수 있습니다.

method: makeshortcut

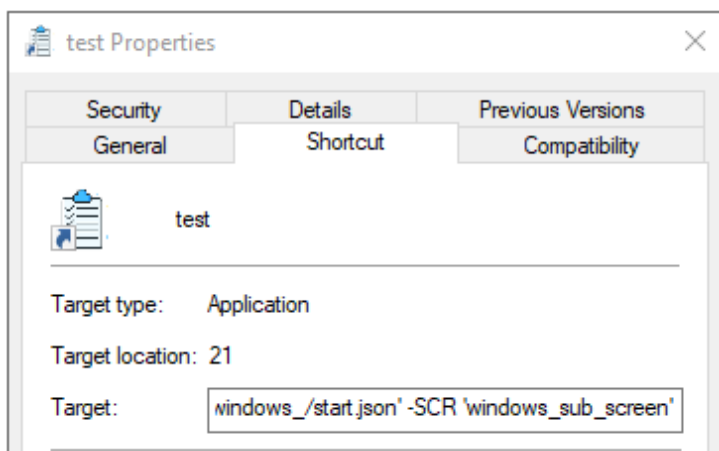
parameter: strShortCutName(단축 아이콘명), strIconPath(아이콘 파일을 내려받을 수 있는 경로), strIconTargetPath(사용자 PC에 아이콘 파일 내려받을 위치), strPosition(단축 아이콘 생성 위치), bAllUser(전체 사용자 사용 허용 여부)

```

this.Button04_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    objNexacro.action = 'method';
    objNexacro.value = { "makeshortcut": { "param": ["test", window.location.protocol + "://" +
window.location.host+"/_windows/_resource/_images/sample_test.ico", "%USERAPP%", "desktop",
true] } };
    this.sendData();
};

```

생성된 단축 아이콘은 지정된 ico 이미지로 생성되며 아래와 같은 타겟 정보를 가집니다.



```
"C:\Program Files\tobesoft\nexacro N\21\nexacro.exe" -K 'TEST' -S 'http://127.0.0.1:4098/_
windows_/start.json' -SCR 'windows_sub_screen'
```

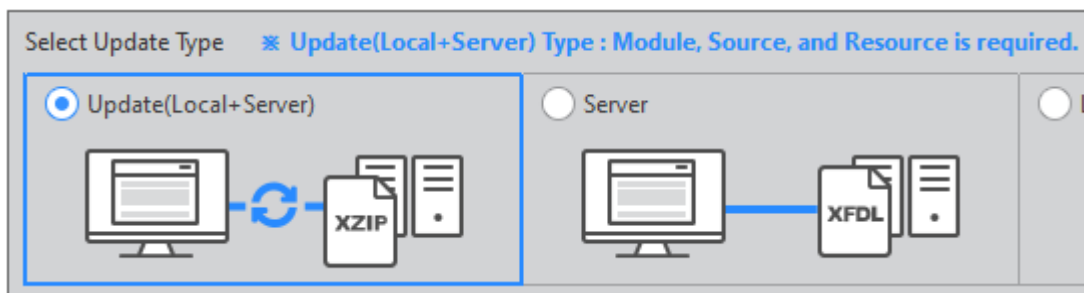
deleteshortcut 메소드를 사용하면 생성한 단축 아이콘을 삭제할 수 있습니다.

```
method: deleteshortcut
parameter: strShortCutName(단축 아이콘명), strPosition(단축 아이콘 생성 위치), bAllUser(전체
사용자 사용 허용 여부)
```

```
this.Button05_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    objNexacro.action = 'method';
    objNexacro.value = { "deleteshortcut": { "param": ["test", "desktop", true] } };
    this.sendData();
};
```

2.4.4 Update(Local+Server) 옵션으로 네트워크 트래픽 줄이기

넥사크로 스튜디오 메뉴[Deploy > Packing]에서 Update Type을 "Server"로 선택하면 매번 실행할 때마다 서버에 접속해서 동작합니다. 일반 웹페이지에 접근할 때와 같은 동작 방식입니다. 하지만, 네트워크 사정이 좋지 않은 지역에서는 필요한 리소스를 압축해서 PC에 내려받고 콘텐츠가 바뀌었을 때만 내려받도록 할 수 있습니다. 이런 경우에는 Update Type을 "Update(Local+Server)"로 선택합니다.



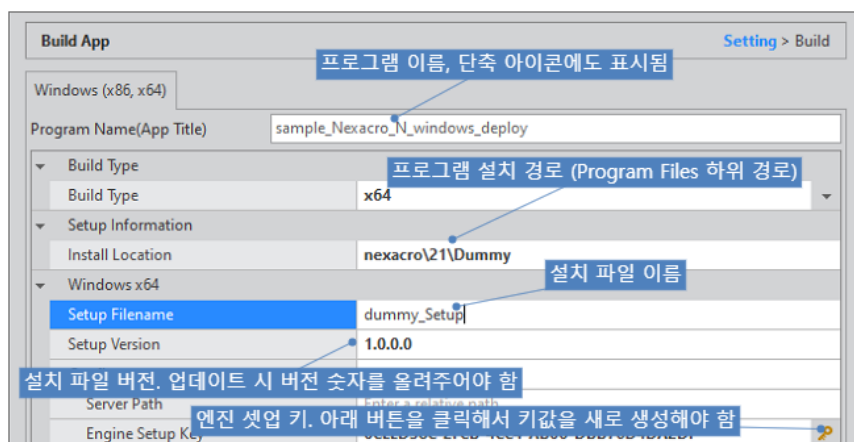
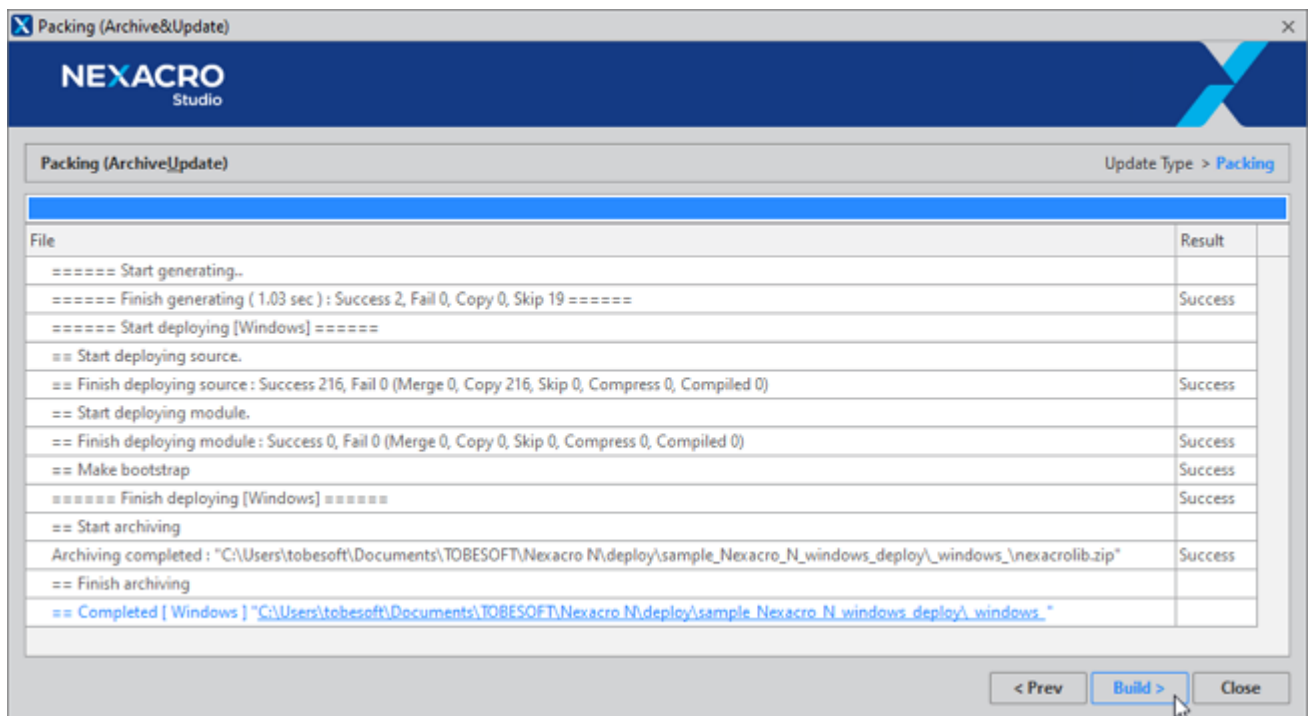
"Server" 방식인 경우에는 일단 해당 리소스를 호출하고 캐시로 저장된 경우에는 지나갑니다. 하지만 "Update(Local+Server)"인 경우에는 리소스를 로컬에 내려받고 버전이 달라지지 않으면 로컬에 있는 파일을 사용합니다.

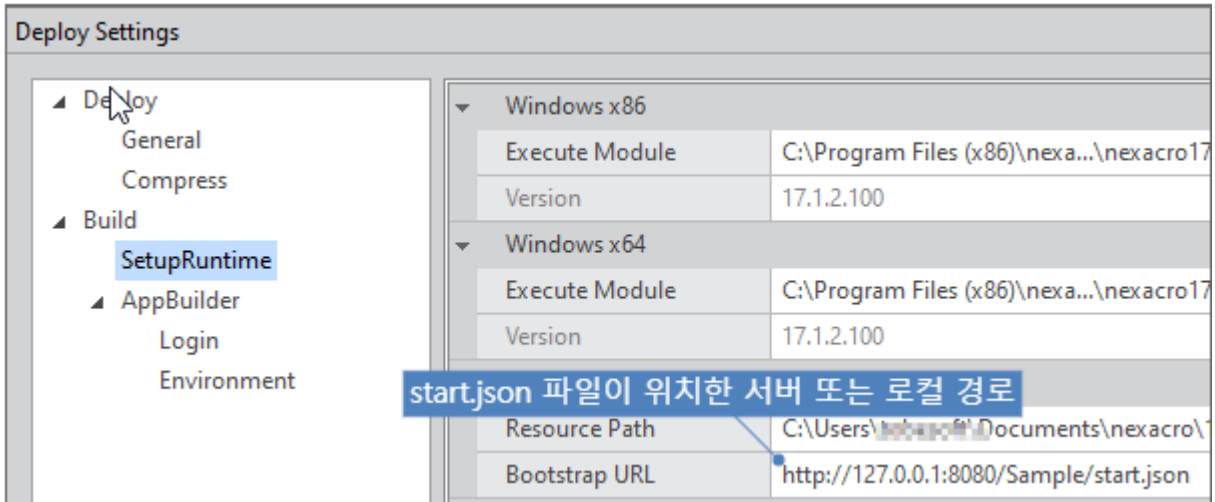
<https://docs.telerik.com/fiddler/Observe-Traffic/Troubleshooting/NoTrafficToLocalhost#problem-traffic-sent-to-httplocalhost-or>

2.4.5 기본 넥사크로 엔진 + 설치 리소스(폴더, 단축 아이콘)

기본 넥사크로 엔진 + 단축 아이콘 만들기에서는 런처 서비스에서 makeshortcut 메소드를 사용해 만들게 되는데, 첫 번째 실행 시에는 서비스 페이지에서 접속해야 하는 제약이 있습니다. 이런 불편함을 줄이려면 넥사크로 엔진 배포 시 단축 아이콘이나 필요한 리소스를 같이 배포하는 방식을 사용할 수 있습니다.

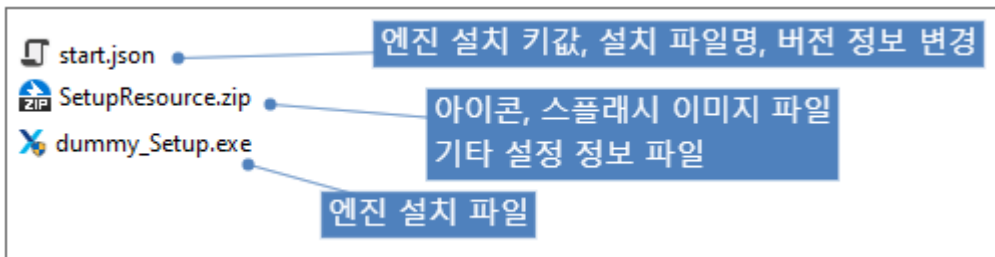
이 방식을 사용하기 위해서는 생성할 폴더, 단축 아이콘 등의 정보를 같이 배포해주어야 합니다. 넥사크로 스튜디오 메뉴[Deploy > Packing] 단계에서 파일 생성 후 [Build] 버튼을 클릭하거나 [Deploy > Build App] 메뉴를 실행합니다.





Engine Setup Key 값은 기본값을 그대로 사용해도 되지만, 기존에 설치된 엔진과 충돌이 생길 수도 있어서 프로그램 설치 경로를 변경한 경우에는 새로운 키값을 생성하는 것을 권장합니다.
그리고 생성한 키값은 런처 서비스를 사용하거나 업데이트 파일 배포 시 같은 키값을 유지해야 합니다.

Build App 단계를 실행하면 3개의 파일이 생성됩니다.



서비스 페이지에서 넥사크로 엔진 설치 파일을 배포하는 것은 이전 단계와 같습니다. 다만 넥사크로 엔진 설치 파일을 실행하기 전에 SetupResource.zip 파일을 내려받아야 합니다. 런처 서비스 downloadresource 메소드를 사용하는 경우에는 임시 경로에 내려받고 설치 후 자동으로 삭제합니다.

아래 설명은 런처 서비스를 사용하는 경우 SetupResource.zip 파일을 내려받는 단계를 설명합니다.

사용자	서비스 페이지
① 서비스 페이지에 접속합니다.	
	② 런처 서비스를 호출하고 실행 여부를 확인합니다.
	③ 런처서비스 setproperty 동작에서 setupresource 속성값을 true로 설정합니다.
	④ 런처서비스 downloadresource 메소드를 실행해 SetupResource 파일을 내려받습니다.
	⑤ 넥사크로 엔진 설치 파일을 내려받습니다. dummy_Setup.exe
⑥ 설치 파일을 실행합니다.	

사용자	서비스 페이지
7 생성된 단축 아이콘을 클릭해 앱을 실행합니다.	

```

this.Button06_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    objNexacro.action = 'setproperty';
    objNexacro.value = {
        "key": "TEST",
        "bjson": window.location.protocol + "://" + window.location.host + "/_dummy_/start.json",
        "screenid": "windows_sub_screen",
        "enginesetupkey": "{DF766894-E778-4A2A-A585-13F9BD8DE053}",
        "setupresource": 'true'
    };
    this.sendData();
};

this.Button07_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    objNexacro.action = 'method';
    objNexacro.value = { "downloadresource": { "param": [window.location.protocol + "://" +
window.location.host + "/_dummy_/"] } } };
    this.sendData();
};

this.Button08_onclick = function(obj:nexacro.Button,e:nexacro.ClickEventInfo)
{
    that.FileDownTransfer00.download(window.location.protocol + "://" + window.location.host +
"/_dummy_/dummy_Setup.exe");
};

```

런처서비스 downloadresource 메소드를 실행해 파일을 내려받은 경우에는 아래 경로의 SetupResource 폴더 아래 파일을 내려받습니다. 내려받은 파일은 넥사크로 엔진 설치 시 사용하고 설치 후에는 자동으로 삭제합니다.

C:\Users\[사용자 계정]\AppData\LocalLow\TOBESOFT\nexacro N\21\SetupResource

바탕화면에 Build App 단계에서 App Title로 지정한 이름으로 단축 아이콘이 생성된 것을 확인할 수 있습니다.



Build App 단계에서 Install Location 항목에 설정한 값에 따라 넥사크로 엔진 설치 경로가 생성됩니다.

2.4.6 기본 넥사크로 엔진 + 설치 리소스 + Update(Local+Server) 옵션 적용하기

Update Type을 "Update(Local+Server)"로 선택한 경우에는 SetupResource.zip과 함께 ProjectResource.zip 파일이 같이 생성됩니다. 서버에는 2개 파일만 올리고 나머지 파일은 올리지 않습니다.

사용자	서비스 페이지
❶ 서비스 페이지에 접속합니다.	
	❷ 런처 서비스를 호출하고 실행 여부를 확인합니다.
	❸ 런처서비스 setproperty 동작에서 setupresource 속성값을 true로 설정합니다.
	❹ 런처서비스 downloadresource 메소드를 실행해 SetupResource, ProjectResource 파일을 내려받습니다.
	❺ 넥사크로 엔진 설치 파일을 내려받습니다.
❻ 설치 파일을 실행합니다.	
❼ 생성된 단축 아이콘을 클릭해 앱을 실행합니다.	

ProjectResource.zip 파일 내에는 start.json 파일과 아카이브 파일을 포함하고 있습니다. 최초 실행 시 로컬에 있는 자원을 확인하므로 서버에 다른 파일을 올리지 않아도 됩니다.



최초 실행 시 서버에 다른 파일이 있는 경우 해당 파일을 다시 내려받아 기존 콘텐츠를 덮어쓰게 됩니다. 업데이트가 아닌 경우 유의해주세요.

2.5 앱 업데이트하기

Update Type이 Server 또는 Update인 경우에는 Packing (Archive&Update) 단계에서 생성한 파일을 서버에 업로드합니다. 업데이트를 위해 Build App 단계는 필요하지 않습니다.

Update Type이 Local인 경우에는 사용자가 설치 파일을 내려받아 실행해서 업데이트를 진행하므로 Build App 단계에서 버전을 수정하고 배포합니다.

2.5.1 넥사크로 엔진 업데이트

Update Type이 "Server" 또는 "Update"인 경우 앱 실행 시 start.json 파일에 설정된 넥사크로 엔진 버전이 달라지면 자동으로 엔진 업데이트를 진행합니다. 사용자가 넥사크로 엔진을 따로 내려받지 않고 임시 경로로 설치 파일을 내려받아 설치한 후 앱이 바로 실행됩니다.

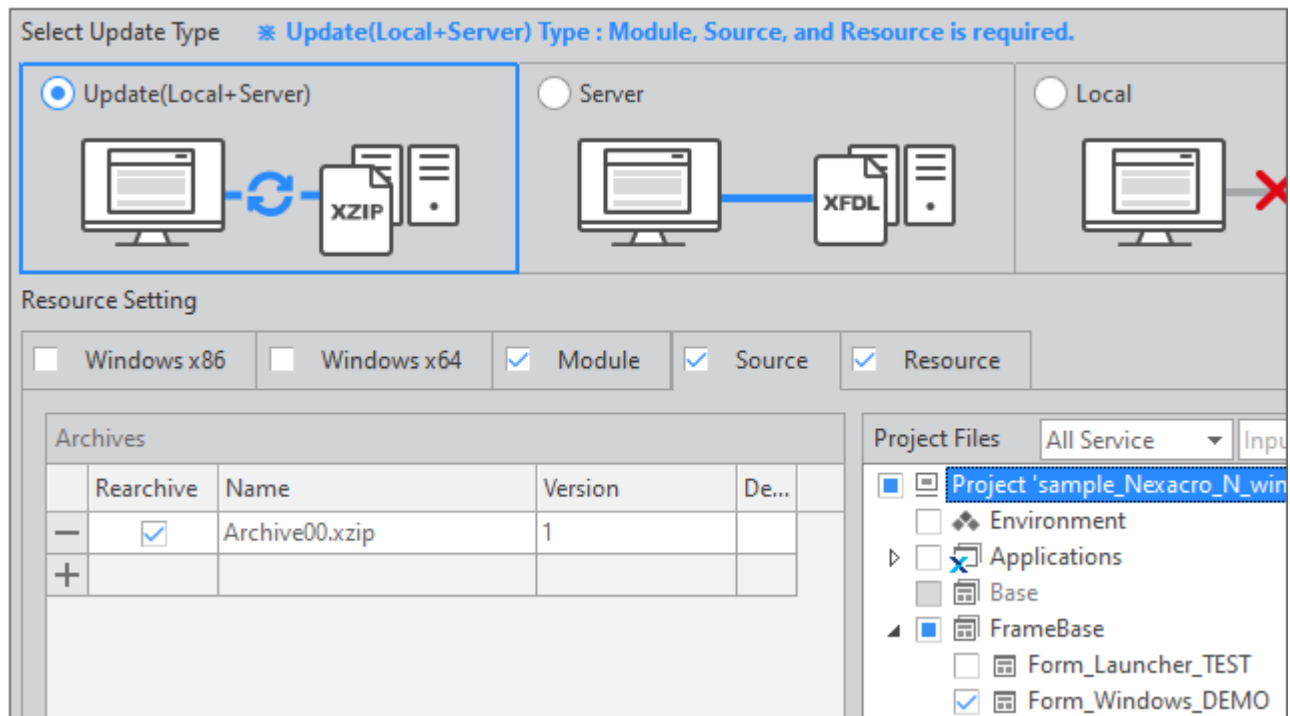
2.5.2 Update Type이 Server인 경우

수정한 파일을 서버에 업데이트합니다.

2.5.3 Update Type이 Update(Local+Server)인 경우

Module, Source, Resource 항목에서 Version 정보를 수정하고 수정된 파일을 선택합니다. 그리고 만들어진 파일을 서버에 업데이트합니다.

Version 정보가 이전보다 높은 값이 아니면 업데이트를 처리하지 않습니다.



2.5.4 Update Type이 "Local"인 경우

Update Type이 "Local"인 경우에는 사용자 PC에 저장된 start.json 파일을 기준으로 동작하기 때문에 앱 실행 시 업데이트를 할 수 없습니다. 이런 경우에는 설치 파일을 다시 배포해서 사용자가 직접 설치하면서 업데이트해야 합니다.

2.6 NRE 로그 확인하기

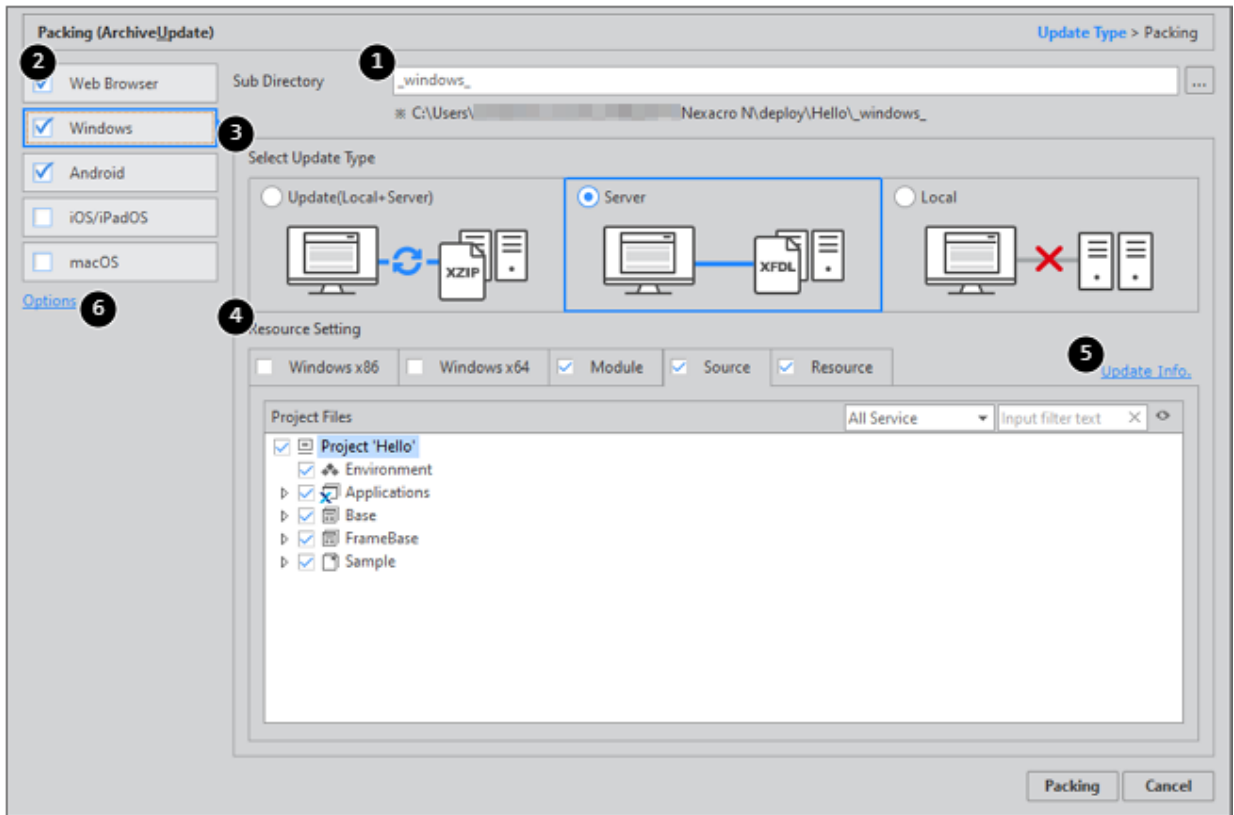
아래 내용을 참고하세요.

 NRE 로그 확인하기

2.7 Packing (Archive&Update) 메뉴

2.7.1 Packing 위저드 실행하고 아카이브 파일 만들기

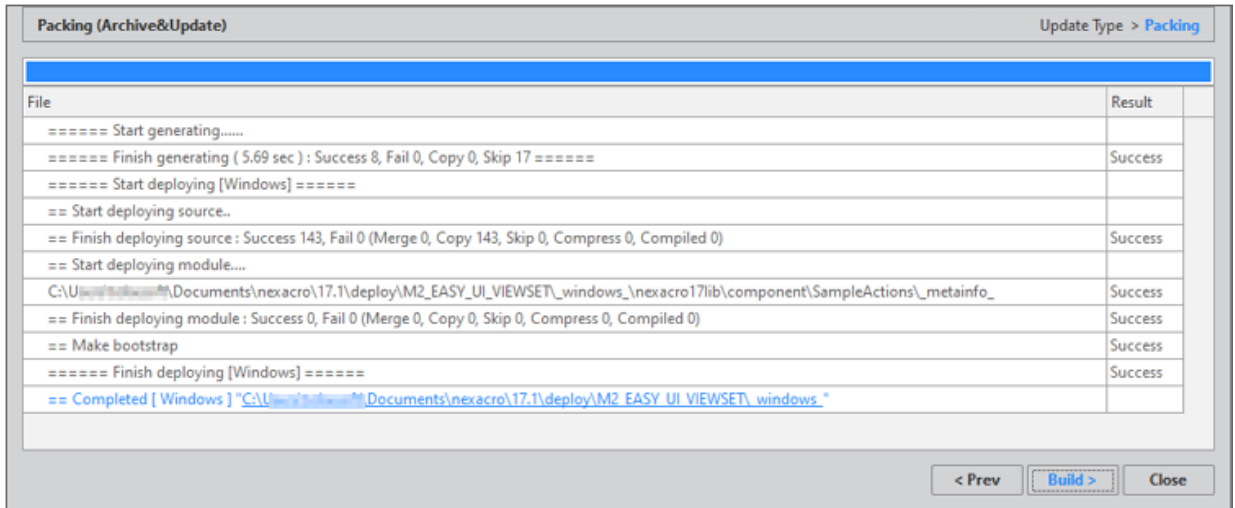
- 1 메뉴 [Deploy > Packing(Archive&Update)]를 선택하면 Packing 위저드가 실행됩니다.
- 2 Output Path 항목에서 아카이브 파일을 생성할 기본 위치를 지정합니다.
- 3 [Next] 버튼을 클릭하면 실행 환경에 따른 Packing 옵션을 선택하는 화면으로 전환됩니다.
- 4 실행 환경을 선택하고 실행 환경에 따른 Update Type과 설정 옵션을 지정합니다. 아카이브 파일의 압축, 난독화 옵션을 지정하는 경우에는 Options 항목을 선택해 원하는 옵션을 지정합니다.



	항목	설명
1	Sub Directory	아카이브 파일을 생성할 폴더를 지정합니다. 선택한 실행 환경에 따라 자동으로 하위 폴더가 생성됩니다. 실행 환경별 하위 폴더는 아래와 같습니다. Web Browser (_web_) Windows (_windows_) Androids (_android_) iOS (_ios_) macOS (_macos_)
2	실행 환경	아카이브 파일을 생성할 실행 환경을 선택합니다. 선택한 실행 환경의 설정 옵션을 지정할 수 있습니다. 선택하지 않은 실행 환경의 설정 옵션은 수정할 수 없습니다.
3	Update Type	앱 배포 방식에 따라 3가지 Update Type을 지정합니다. 선택한 Update Type에 따라 설정 옵션이 변경됩니다. Web Browser 실행 환경은 Update Type을 선택하지 않습니다.
4	Resource Setting	실행 환경별 설정 옵션을 지정합니다.
5	Update Info	Update Info 설정창을 띄웁니다.
6	Options	Deploy 환경 설정창을 띄웁니다. 메뉴 [Deploy > Settings]와 같은 기능입니다.

⑤ [Packing] 버튼을 클릭하면 선택한 실행 환경과 설정에 따라 아카이브 파일을 만들고 결과를 표시합니다.

성공적으로 아카이브 파일이 만들어졌다면 해당 폴더를 파란색 링크로 표시합니다. 링크를 클릭하면 만들어진 폴더를 띄워줍니다. 왼쪽 하단의 [Build] 버튼을 클릭하면 설치 파일을 만드는 Build App 화면으로 이동합니다. 실행 환경을 Web Browser만 선택한 경우에는 [Build] 버튼이 활성화되지 않습니다.



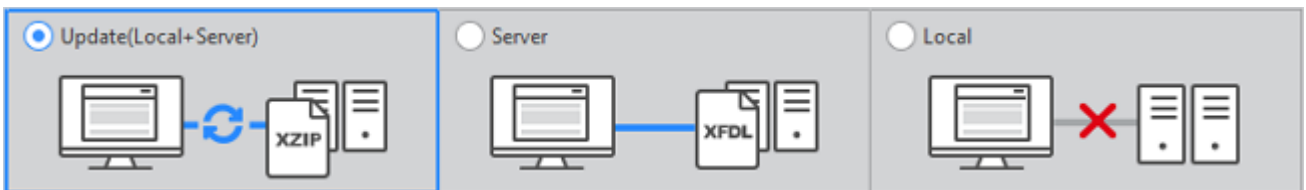
index.html

Web Browser 실행 환경은 별도의 설치 파일을 만들지 않고 웹브라우저에서 접근할 수 있는 HTML 파일을 만듭니다. _web_ 폴더에 index.html 파일을 포함해 서버에 업로드해야 하는 파일이 생성된 것을 확인할 수 있습니다. _web_ 폴더에 있는 파일을 모두 서버에 업로드하면 웹브라우저에서는 아래와 같이 앱에 접근할 수 있습니다.

http://[도메인 주소]/index.html

2.7.2 실행 환경에 따른 Update Type 선택

실행 환경이 Windows, Android, iOS, macOS인 경우에는 Update Type 항목을 선택할 수 있습니다. 선택한 Update Type 항목에 따라 선택할 수 있는 설정 옵션이 달라지며 생성되는 아카이브 파일이 달라집니다.



Update Type

- Update (Local + Server)
최초 설치 시 필요한 아카이브 파일을 내려받아 로컬 스토리지에 저장합니다. 콘텐츠 업데이트 시 버전 정보를 변경하면 네트워크에 연결해 필요한 파일을 내려받아 로컬 스토리지에 저장합니다.
- Server
네트워크에 연결된 상태에서만 실행할 수 있습니다. 필요한 파일을 서버에서 내려받아 사용합니다.
- Local
아카이브 파일을 포함한 상태로 배포 파일이 생성되며 설치 시 로컬 스토리지에 파일을 설치합니다. 네트워크 연결 없이 앱을 실행할 수 있습니다.



앱에 따라 Update Type 항목이 Local인 경우에도 데이터를 조회하거나 서버에 연결할 필요가 있는 경우 네트워크 연결이 필요할 수 있습니다.

2.7.3 실행 환경, Update Type에 따른 설정 옵션

실행 환경, Update Type에 따라 배포할 대상을 설정할 수 있습니다. 설정 옵션 항목에서 배포할 파일을 선택하거나 추가할 수 있습니다.



Module, Source, Resource 업데이트 시 Version 정보가 이전에 배포한 파일의 Version 정보와 같다면 실제 구동 시 같은 파일로 판단하고 업데이트를 진행하지 않습니다.

Update Info

최신 콘텐츠를 서버에서 받아서 처리하는 업데이트 정책을 지정할 수 있습니다. 콘텐츠를 받아올 경로를 지정하고 일정 횟수 정해진 시간만큼 업데이트를 시도하다가 콘텐츠를 받지 못하면 실패로 처리합니다. 업데이트를 자동으로 처리할지 사용자 요청 시 처리할지도 지정할 수 있습니다.

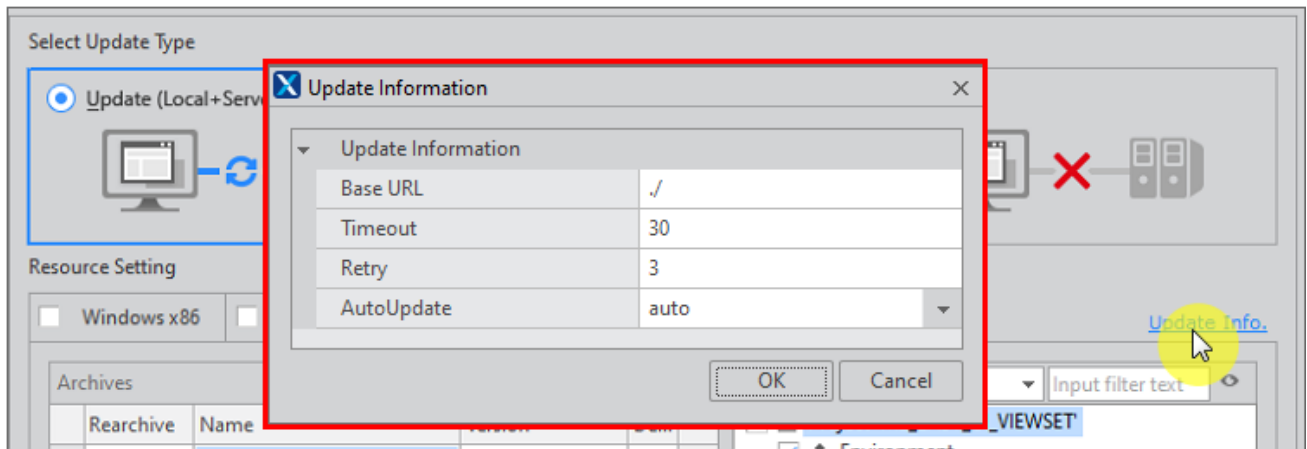
실행 환경, Update Type 별 설정 옵션 여부

	Update (Local + Server)	Server	Local
Web Browser	X		
Windows	O (필수)	O (필수)	X
Android	O (필수)	O (필수)	X
iOS	O (필수)	O (필수)	X

	Update (Local + Server)	Server	Local
macOS	O (필수)	O (필수)	X

설정 옵션

Resource Setting 항목 오른쪽에 있는 [Update Info] 항목을 클릭하고 내용을 편집합니다.



항목	설명
Base URL	SetupResource.zip, SetupRuntime.exe 등 업데이트가 필요한 파일의 기본 접근 경로
Timeout	대기시간
Retry	재시도 횟수
AutoUpdate	auto, manual 중 선택



Update Info 항목은 필수 항목이며 체크를 해제할 수 없습니다.

Setup

Build App 위저드를 사용하지 않고 넥사크로 기본 Setup을 배포할 수 있습니다. 부트스트랩 파일과 리소스 파일만 생성하며 부트스트랩 정보에 Setup 업데이트 정보가 추가됩니다.

실행 환경, Update Type 별 설정 옵션 여부

	Update (Local + Server)	Server	Local
Web Browser	X		
Windows	O	O	O

	Update (Local + Server)	Server	Local
Android	X	X	X
iOS	X	X	X
macOS	X	X	X

설정 옵션

배포할 운영체제 타입을 선택합니다. 선택한 항목에 대한 정보는 부트스트랩 파일(start.json)에 추가됩니다.

Resource Setting

☒ Windows x86
☐ Windows x64
☒ Module
☒ Source
☒ Resource

NRE (Windows)

Server Path

Enter a relative path

Setup Filename

NexacroN_SetupRuntime_x86.exe

Engine Setup Key

{3D10EE09-348B-4B0B-AD04-69A3CE921D1C}

Version

21.0.0.100

항목	설명
Use	배포할 운영체제 선택
Type	운영체제 타입
Server Path	Update Info 항목에서 설정한 Base URL 상대 경로 Setup 배포 경로가 Base URL과 다른 경우 지정합니다. 예를 들어 Base URL 항목은 "UPDATE"로 지정하고 Server Path 항목을 "SET UP_FILE"로 지정한 경우에는 아래와 같이 접근합니다. http://localhost/UPDATE/SETUP_FILE/[setup file name]
Setup Filename	Setup 파일명
Engine Setup Key	Setup Key
Version	Setup 버전 정보



Setup 설정 옵션을 선택하고 바로가기 아이콘이나 스플래시 이미지를 수정하거나 엔진 셋업키를 변경하기 원한다면 Build App 단계에서 변경할 수 있습니다.

Module

넥사크로 앱을 실행하는 데 필요한 프레임워크(nexacrolib) 배포 여부를 설정할 수 있습니다. 실행 환경이 Windows 라면 외부 모듈을 추가할 수 있습니다.

실행 환경, Update Type 별 설정 옵션 여부

	Update (Local + Server)	Server	Local
Web Browser	X		
Windows	O	O	O (필수)
Android	O	X	O (필수)
iOS	O	X	O (필수)
macOS	O	X	O (필수)

설정 옵션

배포할 모듈 파일을 추가하거나 변경, 삭제할 수 있습니다. 실행 환경에 따라 설정 옵션 중 일부만 제공될 수 있습니다.

항목	설명
Type	모듈 타입 확장 모듈의 경우 운영체제 버전을 표시합니다 (Ext [운영체제버전]) 프레임워크(nexacrolib), 파일(File) 모듈 타입을 표시합니다.
Server Path	Update Info 항목에서 설정한 Base URL 상대 경로 모듈 배포 경로가 Base URL과 다른 경우 지정합니다. 예를 들어 Base URL 항목은 "UPDATE"로 지정하고 Server Path 항목을 "MODULE"로 지정한 경우에는 아래와 같이 접근합니다. http://localhost/UPDATE/MODULE/[module file name]
Name	모듈 파일명 (변경할 수 없음)
TargetPath	로컬 환경에 파일을 내려받는 경로를 지정합니다. 기본 설정(./)으로 지정한 경우에는 %USERAPP% 경로에 파일을 저장합니다. %USERAPP%Component/TEST/ 와 같은 식으로 특정 경로를 지정할 수도 있습니다.
Version	모듈 버전
Description	기타 메모 (시스템 실행에는 영향을 미치지 않습니다).



%USERAPP% Alias 경로 참고

설정 옵션 (Update Type이 Local인 경우)

프레임워크 파일(nexacrolib.zip)이 기본 설정되어 있으며 삭제하거나 변경할 수 없습니다. 설정 옵션 항목은 Type, Name 두 가지 값만 표시합니다.

설정 옵션 (실행 환경이 Windows인 경우)

확장 모듈, 파일을 추가하는 기능을 사용할 수 있습니다. 실행 환경이 Android, iOS, macOS인 경우에는 프레임워크 파일 외 다른 모듈 파일을 배포할 수 없습니다.

설정 옵션 (실행 환경이 Web Browser 또는 Update Type이 Server인 경우)

프레임워크 파일을 압축된 파일 형태로 내려받지 않고 Web Browser에서 실행되는 파일 형태로 배포합니다.

Source

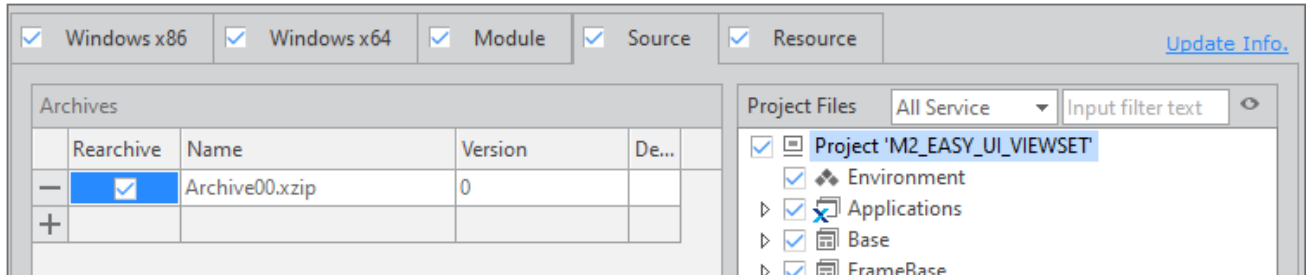
프로젝트 파일 중에서 배포할 대상을 선택합니다. 최초 배포 시에는 전체 프로젝트를 선택해야 하며 업데이트 시에는 필요한 대상만 선택하고 배포할 수 있습니다. 실행 환경과 Update Type에 따라 파일을 직접 배포하거나 압축된 아카이브 파일 형태로 배포합니다.

실행 환경, Update Type 별 설정 옵션 여부

	Update (Local + Server)	Server	Local
Web Browser	O		
Windows	O	O	O (필수)
Android	O	O	O (필수)
iOS	O	O	O (필수)
macOS	O	O	O (필수)

설정 옵션

Project Files 항목에서 배포 대상 파일을 선택합니다. Archives 항목이 있는 경우에는 소스 파일을 담은 아카이브 파일을 선택한 후에 Project Files 항목에서 배포 대상을 선택할 수 있습니다.



항목	설명
Rearchive	체크박스를 선택해 원하는 소스 아카이브 파일만 생성할 수 있습니다.
Name	아카이브 파일명
Version	아카이브 파일 버전
Description	기타 메모 (시스템 실행에는 영향을 미치지 않습니다).

설정 옵션 (실행 환경이 Web Browser 또는 Update Type이 Server인 경우)

아카이브 파일을 만들지 않고 Project Files에서 선택한 항목만 배포합니다. 설정 옵션에서 Archives 항목이 표시되지 않습니다.

설정 옵션 (Update Type이 Local인 경우)

설정 옵션 항목은 Rearchive, Name 두 가지 값만 표시합니다.

Resource

리소스 파일 중에서 배포할 대상을 선택합니다. 최초 배포 시에는 전체 리소스를 선택해야 하며 업데이트 시에는 필요한 대상만 선택하고 배포할 수 있습니다. 실행 환경과 Update Type에 따라 파일을 직접 배포하거나 압축된 아카이브 파일 형태로 배포합니다.

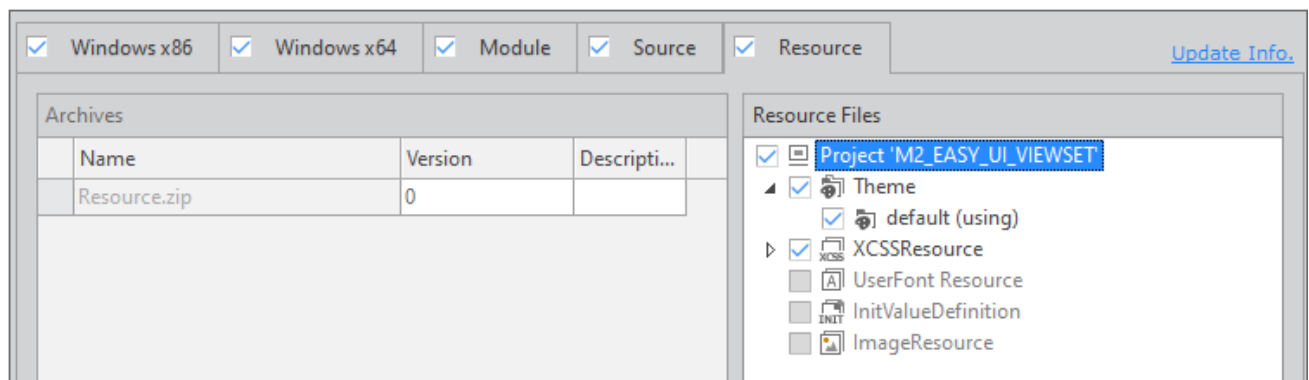
실행 환경, Update Type 별 설정 옵션 여부

	Update (Local + Server)	Server	Local
Web Browser	○		

	Update (Local + Server)	Server	Local
Windows	O	O	O (필수)
Android	O	O	O (필수)
iOS	O	O	O (필수)
macOS	O	O	O (필수)

설정 옵션

Resource Files 항목에서 배포 대상을 선택합니다. Archives 항목이 있는 경우에는 Resource.zip 하나의 파일만 만들 수 있습니다.



항목	설명
Name	아카이브 파일명 (변경할 수 없음)
Version	아카이브 파일 버전
Description	기타 메모 (시스템 실행에는 영향을 미치지 않습니다).

설정 옵션 (실행 환경이 Web Browser 또는 Update Type이 Server인 경우)

아카이브 파일을 만들지 않고 Resource Files에서 선택한 항목만 배포합니다. 설정 옵션에서 Archives 항목이 표시되지 않습니다.

설정 옵션 (Update Type이 Local인 경우)

설정 옵션 항목은 Name만 표시합니다.

2.8 Build App 메뉴

2.8.1 Build App 위저드 실행하고 설치 파일 만들기

- 1 Packing 위저드에서 아카이브 파일을 만들고 [Build] 버튼을 클릭하거나 메뉴 [Deploy > Build App]를 선택하면 Build App 위저드가 실행됩니다.

Packing 단계에서 [Build] 버튼을 클릭한 경우에는 속성 설정화면으로 넘어가고 Build App 메뉴로 접근한 경우에는 Build Target 선택 후 설정 화면으로 이동합니다.

- 2 Build Target 항목 중 Windows 운영체제를 선택한 경우에 Setup 관련 설정을 확인하고 [Build] 버튼을 클릭합니다.

항목	설명
Program Name	프로그램 추가/삭제에서 표시되는 프로그램 명입니다.기본값은 프로젝트명으로 설정합니다.
Build Type	
Build Type	운영체제 버전을 선택합니다.
Setup information	
Install Location	앱 설치 파일 실행 시 앱이 설치될 경로를 지정합니다. 운영체제 "Program Files" 폴더 아래에 설치될 경로를 지정할 수 있습니다. 기본값은 "TOBESOFT\Nexacro N\21\{program Name}"입니다.
Windows [운영체제 버전]	

항목	설명
32비트, 64비트 중에서 선택한 운영체제 정보만 설정합니다. 실행 환경에 따라 설치 파일명을 변경할 수 있습니다.	
Setup Filename	생성할 설치 파일명을 지정합니다.
Setup Version	설치 파일 버전을 지정합니다.
Windows [운영체제 버전] > Configure - Server Path나 Engine Setup Key 변경 시 설정합니다.	
Server Path	Update Info 항목에서 설정한 Base URL 상대 경로 Setup 배포 경로가 Base URL과 다른 경우 지정합니다. 예를 들어 Base URL 항목은 "UPDATE"로 지정하고 Server Path 항목을 "SETUP_FILE"로 지정한 경우에는 아래와 같이 접근합니다. http://localhost/UPDATE/SETUP_FILE/[setup file name]
Engine Setup Key	엔진 설치 키 (clsid)를 지정합니다.

▼ Advanced	
Install Mode	Default
Auto Update	Auto
– Shortcut	
Type	Select at Install time
Icon File	
– Splash	
Image File	
Show Message	Default
Update Browser	latest version
Rename Executable	
Only One	false

항목	설명
Advanced	
Install Mode	Default: 기본 설치 모드 Silent: 설치 중 대화상자가 나타나지 않고 기본 설정값으로 설치
Auto Update	Packing 단계에서 Update Info에 설정한 AutoUpdate 설정보다 우선해서 넥사크로 엔진 자동 업데이트 여부를 지정합니다. 기본값은 "Auto"입니다. "Manual" 항목 지정 시 넥사크로 엔진 자동 업데이트를 처리하지 않습니다.
Shortcut Type	Desktop : 바탕화면에 바로가기 아이콘을 생성합니다. Start Menu : 시작메뉴에 바로가기 아이콘을 생성합니다. Desktop + Start Menu : 바탕화면과 시작메뉴에 바로가기 아이콘을 생성합니다. Select at Install time: 엔진 설치 시 바로가기 아이콘 생성 위치를 지정

항목	설명
	합니다. No use : 바로가기 아이콘을 생성하지 않습니다. 기본값은 "Select at Install time"입니다. Install Mode 옵션값이 "Silent"인 경우에는 "Select at Install time"를 사용할 수 없으며 기본값은 "Desktop"입니다.
Shortcut Icon File	앱 설치 파일, 실행 및 삭제 프로그램의 아이콘을 변경합니다. 기본값은 넥사크로 아이콘을 사용합니다.
Splash Image File	스플래시 화면 이미지를 지정한 파일로 변경합니다.
Splash Show Message	Default : 기본 메시지 표시 Do not show: 메시지를 표시하지 않음
Update Browser	브라우저 에뮬레이션을 최신 버전으로 업데이트할지를 설정합니다.
Rename Executable	실행 프로그램 파일명을 변경합니다. 기본값은 "nexacro.exe"입니다.
Only One	같은 키 값을 가진 앱을 중복 실행하지 않도록 설정합니다.



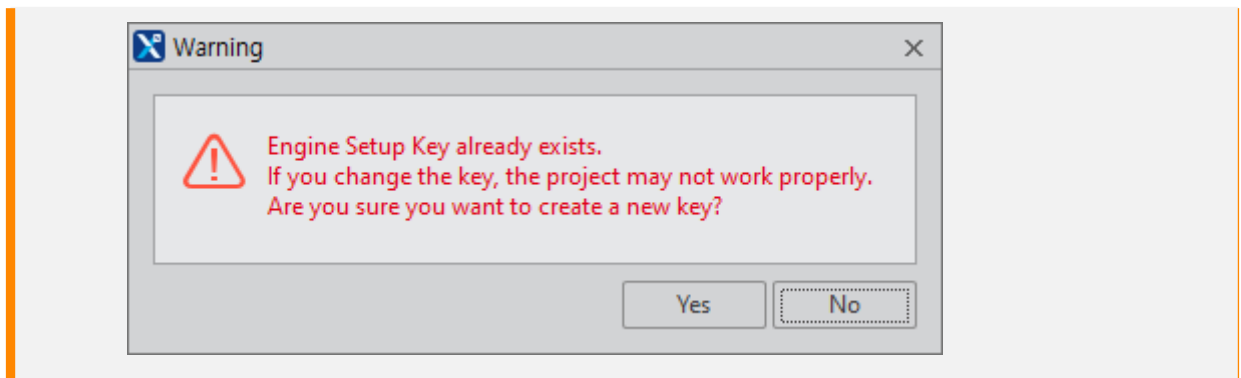
앱에서 WebBrowser 컴포넌트를 사용하는 경우 Update Browser 옵션으로 설정한 값에 따라 실행되는 브라우저 에뮬레이션 버전이 달라집니다. 사용자 PC에 2개 이상의 앱이 설치된 경우 Update Browser 옵션이 다르면 앱 간의 설정 충돌이 발생할 수 있습니다.
Rename Executable 옵션에서 기본값인 "nexacro.exe" 대신 다른 이름으로 파일명을 지정하면 설치된 앱 간의 설정 충돌을 방지할 수 있습니다.

기본 Engine Setup Key를 그대로 사용하거나 입력칸 오른쪽에 있는 열쇠 모양의 버튼을 클릭하면 Engine Setup Key를 새로 생성할 수 있습니다. 사용자 PC에 여러 개의 앱이 설치되어야 하는 경우에는 Engine Setup Key를 새로 생성하고 따로 관리하는 것을 권장합니다.

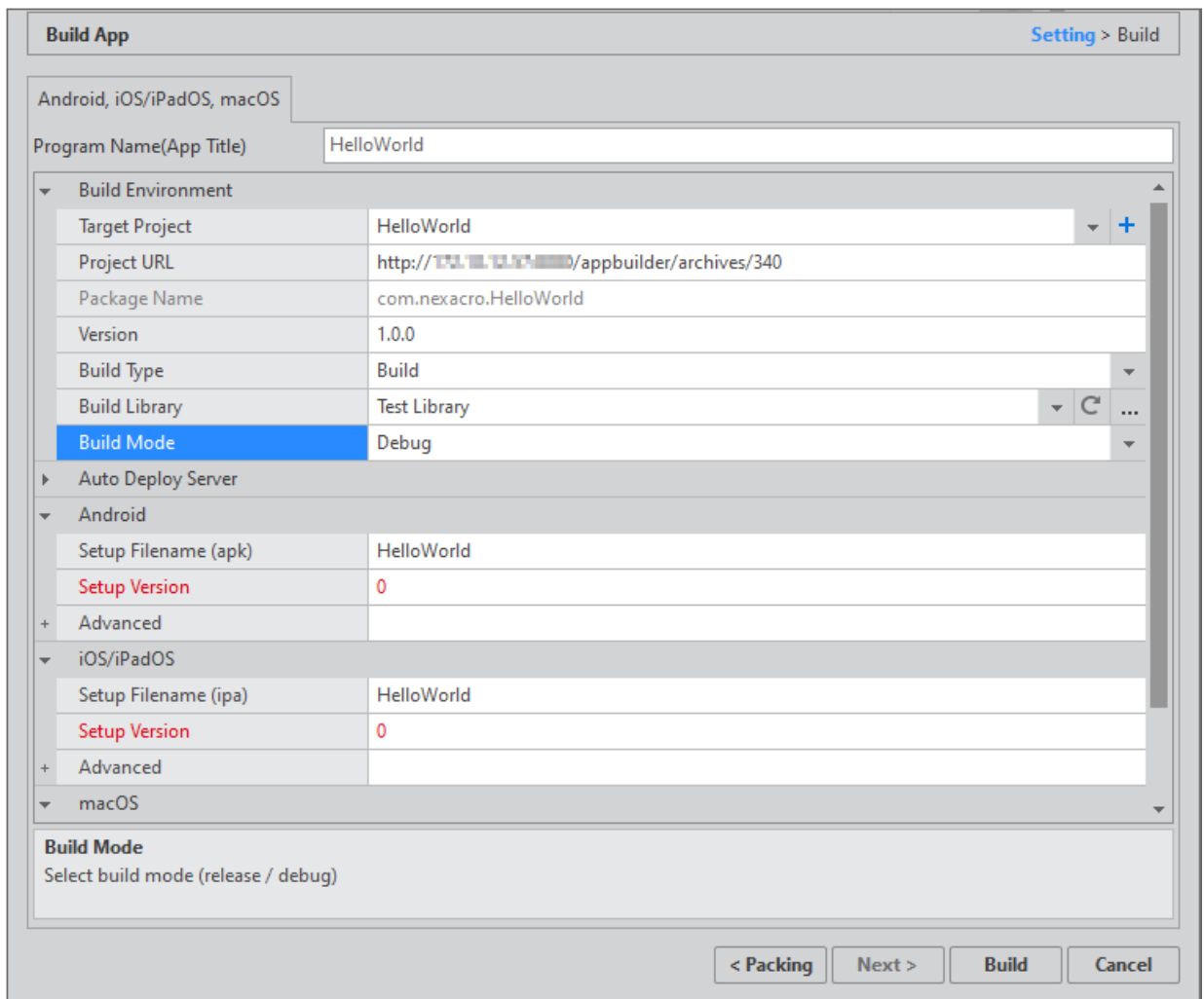
Windows x86	
Setup Filename	NexacroN_SetupRuntime_x86.exe
Setup Version	21.0.0.100
Configure	
Server Path	Enter a relative path
Engine Setup Key	50 



업데이트 과정에서 Engine Setup Key를 새로 생성하면 설치된 앱을 업데이트할 수 없습니다.



- 3 Build Target 항목 중 Android, iOS, macOS 운영체제를 선택한 경우에 AppBuilder 관련 옵션을 설정합니다.



항목	설명
Project information	
Program Name	운영체제에 앱 설치 시 표시되는 프로그램명입니다.
Build Environment	
TargetProject	앱빌더 프로젝트를 신규로 생성하거나 기존에 있는 앱빌더 프로젝트를

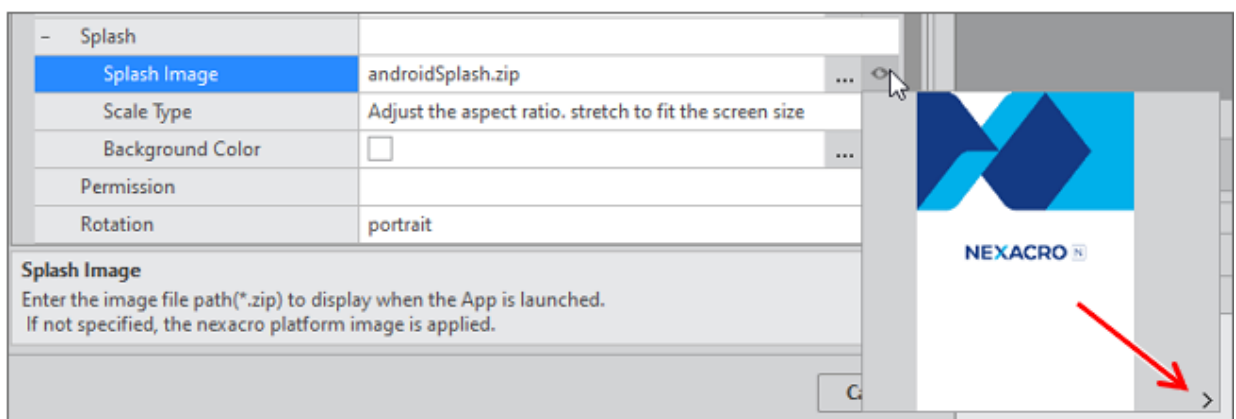
항목	설명
Project information	
	선택합니다.
Project URL	앱을 배포할 URL을 지정할 수 있습니다. 기본값은 AppBuilder에서 자동생성되는 URL입니다.
Package Name	Package 이름을 표시합니다.
Version	파일 버전을 지정합니다.
Build Type	- Build 선택한 프로젝트를 새로 빌드합니다. 이미 빌드한 경우에는 다시 빌드합니다. - Upload Contents 선택한 프로젝트의 콘텐츠만 업로드합니다.
Build Library	프로젝트 빌드 시 적용할 넥사크로 라이브러리를 선택합니다.
Build Mode	빌드 모드를 선택합니다.
Auto Deploy Server	
Deploy Mode	Deploy 서버 사용 여부를 설정합니다. true 선택 시 Deploy Server를 선택할 수 있습니다.
Deploy Server	Deploy Mode 항목값을 true 선택 시 앱빌더에 설정한 Deploy Server 목록에서 Deploy 서버를 선택합니다.
Deploy Directory	Deploy Server에 설정된 경로를 표시합니다. 설정값은 수정할 수 있습니다.
[운영체제]	
Setup Filename	생성할 설치 파일명을 지정합니다.
Setup Version	설치 파일 버전을 지정합니다.
[운영체제 > Advanced]	
Signing	프로젝트 빌드 시 적용할 Signing 정보를 지정합니다.
Plist URL	In-House 업데이트를 처리하기 위한 plist URL 정보를 지정합니다. AppBuilder 서버를 사용하는 경우에는 기본값을 사용하며 별도 배포 서버를 운영하는 경우에 URL 값을 변경할 수 있습니다. iOS에서만 사용합니다.
Built-in Library	빌드 시 적용할 32비트, 64비트 공유 라이브러리를 선택합니다.
User Library	프로젝트 빌드 시 적용할 User Library를 선택합니다. Android, iOS만 지원하며 macOS는 지원하지 않습니다.
Icon	앱 아이콘 이미지 파일을 지정합니다.
Splash	- Splash Image 앱 실행 시 표시되는 Splash 이미지 파일을 지정합니다. - Scale Type Splash 이미지 크기를 설정합니다. 아래 4가지 옵션 중 하나를 선택할 수 있습니다. > Adjust the aspect ratio. stretch to fit the screen size : 화면 비율 조정. 화면 크기에 맞게 stretch 처리

항목	설명
Project information	
	> Maintain the aspect ratio. expand without blank area : 화면 비율 유지. 빈 영역 없이 확장 처리 > Maintain the aspect ratio. expand to fit : 화면 비율 유지. 크기에 맞게 확장 > Original size : 원본 이미지 크기 - Background Color Splash 영역 배경 색상을 지정합니다.
Dmg Image	Apple disk image (DMG) 배경 이미지 파일을 지정합니다. macOS에서만 사용합니다.
Permission	앱 실행 권한을 지정합니다. Android만 지원하며 iOS, macOS는 지원하지 않습니다.
Rotation	앱에서 화면을 세로(Portrait) 또는 가로(Landscape) 로 고정시키거나 고정을 풀고 센서를 활성화(Sensor) 하는 옵션을 선택합니다.

아이콘, 이미지 파일

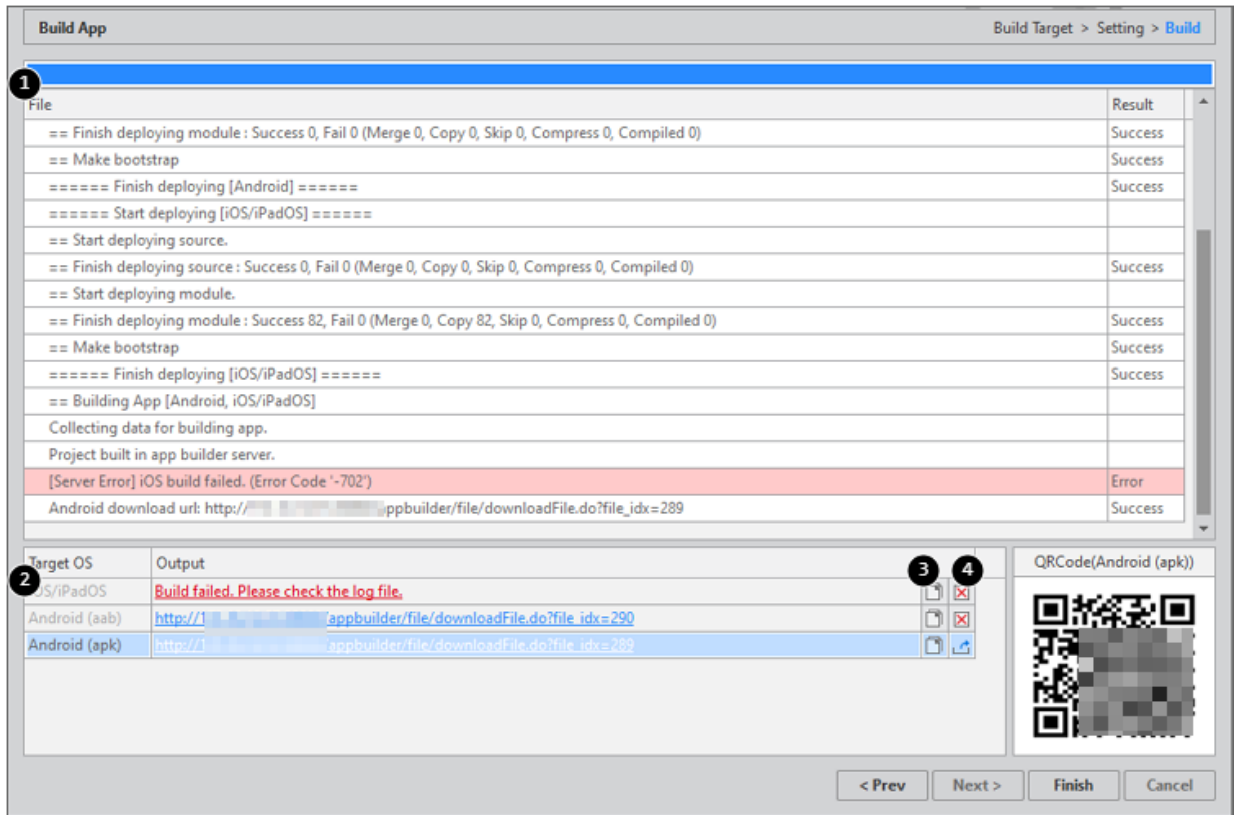
운영체제에 따라 지정된 형식에 맞는 파일을 지정해주어야 합니다.

Icon이나 Splash Image 등 이미지 파일을 포함하고 있는 경우에 눈 모양 아이콘 클릭 시 이미지 미리보기 기능을 제공합니다. zip 파일로 여러 이미지 파일을 묶은 경우 오른쪽 아래 [>] 버튼을 클릭해 다음 이미지를 확인할 수 있습니다.



- 4 [Build] 버튼을 클릭하면 설정에 따라 설치 파일을 생성합니다.

설치 파일 생성이 완료되면 생성된 파일을 확인할 수 있습니다. 운영체제 또는 파일에 따라 QR 코드가 지원되는 경우에는 산출물 선택 시 QR 코드 이미지가 표시되며 그렇지 않으면 산출물 URL만 제공합니다. 빌드 실패 시에는 빨간색으로 표시되며 표시합니다.



	항목	설명
1	Log	앱 빌더에서 처리한 작업 로그를 표시합니다. 오류가 발생한 경우에는 오류 메시지를 표시합니다.
2	Download url	배포 대상 환경별 설치 파일을 내려받을 수 있는 URL을 표시합니다. 마우스 포인터를 가져가면 오른쪽에 QR 코드 이미지가 표시됩니다.
3	Copy url	Download url 값을 클립보드에 복사합니다.
4	Save QR Code	QR Code 이미지를 PNG 파일로 저장합니다.

2.9 참고

2.9.1 nexacro.exe 실행 옵션

nexacro.exe 실행 옵션을 아래 표와 같이 지정할 수 있습니다.

옵션	설명
-K	앱을 구별하기 위한 키값을 문자열로 설정합니다.
-S	부트스트랩 경로를 설정합니다.

옵션	설명
-C	컴포넌트가 설치된 경로를 설정합니다.
-THR	엔진 내부 통신 및 기타 Manager 의 쓰레드 수를 숫자로 설정합니다. 쓰레드의 개수는 최소 3개, 최대 10개이며 기본값은 3입니다.
-SP	스플래시 이미지 경로를 설정합니다.
-SM	스플래시 이미지에 시스템 메시지가 표시되지 않도록 "none" 값을 설정합니다.
-LF	앱의 Left 위치 좌푯값을 설정합니다.
-TP	앱의 Top 위치 좌푯값을 설정합니다.
-WD	앱의 너비를 픽셀 단위 숫자로 설정합니다.
-HT	앱의 높이를 픽셀 단위 숫자로 설정합니다.
-LG	Loading Image 의 경로를 설정합니다.
-TW	엔진 내부 Manager 의 쓰레드가 대기할 최대시간을 설정합니다.
-OO	동일한 키의 앱의 실행을 한 개만 허용할 지 Boolean 값으로 설정합니다.
-V	앱이 실행될 때 추가될 Global 변수를 "변수명:타입=값 변수명:타입=값" 형식으로 설정합니다. 각 변수는 콤마(",")문자로 구분하여 설정합니다. "타입"에 "ev"를 설정 시 Environment 의 Variables 영역에 변수가 추가됩니다. "타입"에 "ec"를 설정 시 Environment 의 Cookies 영역에 변수가 추가됩니다. "타입"을 설정하지 않거나 "av"를 설정 시 Application 의 Variables 영역에 변수가 추가됩니다.
-SIE	앱 실행 시 에러가 발생하면 에러 메시지를 표시하고 종료할 지 Boolean 으로 설정합니다.
-CT	초기 네트워크가 연결되어 있는지 체크하는 최대 대기시간을 설정합니다. 네트워크가 연결되어 있는지 여부를 앱이 구동되기 전에 체크합니다. 대기시간동안 네트워크가 연결여부가 확인되지 않으면 앱이 종료됩니다.
-SCR	LayoutManager 에서 사용하기 위한 ScreenID 를 설정합니다.
-CC	V8 엔진의 CodeCache 기능을 비활성화 하기위해 설정합니다. "NO" 설정 시 CodeCache 기능을 사용하지 않습니다. "AL" 설정 또는 옵션값 생략 시 CodeCache 기능이 활성화 됩니다.
-AU	앱이 실행될 때 엔진의 업데이트 여부를 설정합니다. "auto" 설정 시 엔진을 최신버전으로 업데이트합니다. "manual" 설정 시 엔진을 업데이트 하지 않습니다.
-NC	로컬 경로의 config 설정 파일을 설정합니다.
-RPP	NRE 원격 디버깅 시 실행 옵션입니다.
-brk	NRE 원격 디버깅

2.9.2 운영체제 버전별 배포

32비트, 64비트 버전별 설치파일을 생성한 경우 사용자에게 모든 파일의 내려받기 정보를 제공한 후 사용자가 선택해서 내려받을 수도 있고 스크립트를 사용해 운영체제 버전을 확인 후 적절한 링크 정보를 제공할 수 있습니다.

```

<script type="text/javascript">
var SetupRuntimeName="";
function oninit()
{
    var userAgent = navigator.userAgent;
    if(userAgent.indexOf("WOW64") != -1
        || userAgent.indexOf("Win64") != -1)
        SetupRuntimeName="NexacroN_SetupRuntime_x64.exe";
    else
        SetupRuntimeName="NexacroN_SetupRuntime_x32.exe";

    document.getElementById("test").innerHTML = "<a href='"+SetupRuntimeName+"'>"
        +SetupRuntimeName+"</a>"
}
</script>
<body onload="oninit()">
Download link -> <span id="test"></span>
</body>

```

32비트 운영체제에서 해당 페이지 접근 시 운영체제 버전을 확인하고 적절한 링크를 제공할 수 있습니다.

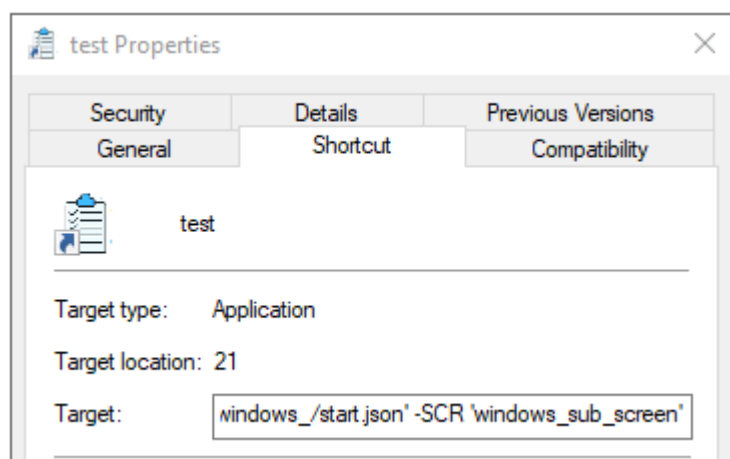


스크립트는 설명을 위해 작성된 코드입니다. 운영하는 시스템에 맞게 변경해 적용하실 수 있습니다. 운영체제 변경에 따라 해당 스크립트가 동작하지 않을 수도 있습니다.

2.9.3 단축 아이콘 Bootstrap URL 속성값 변경



사용자에게 설치된 단축 아이콘의 Bootstrap URL 속성값만 변경해서 사용하는 경우 설치된 제품 버전과 배포된 앱 버전이 달라 정상적으로 동작하지 않을 수 있습니다. 임의로 Bootstrap URL을 변경하는 것은 권장하지 않습니다. 다른 앱을 배포할 경우에는 Engine Setup Key, Install Location 값이 다른 설치파일을 생성하고 배포하는 것을 권장합니다.



파트 III.

런처 서비스

3.

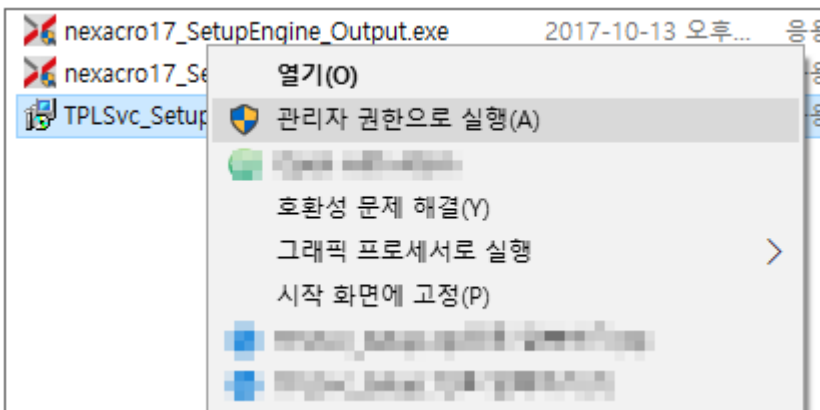
설치 및 기본 사용

3.1 런처 서비스 설치

3.1.1 PC에 런처 서비스 설치하고 윈도우 서비스 등록하기

런처 서비스는 윈도우 서비스로 등록되어 동작합니다. 런처를 사용하기 위해서는 먼저 윈도우 서비스를 등록해야 합니다.

- ① 설치 파일을 내려받아 관리자 권한으로 실행합니다.



설치 시 관리자 권한으로 실행하지 않은 경우에는 서비스 동작 시 오류가 발생할 수 있습니다.

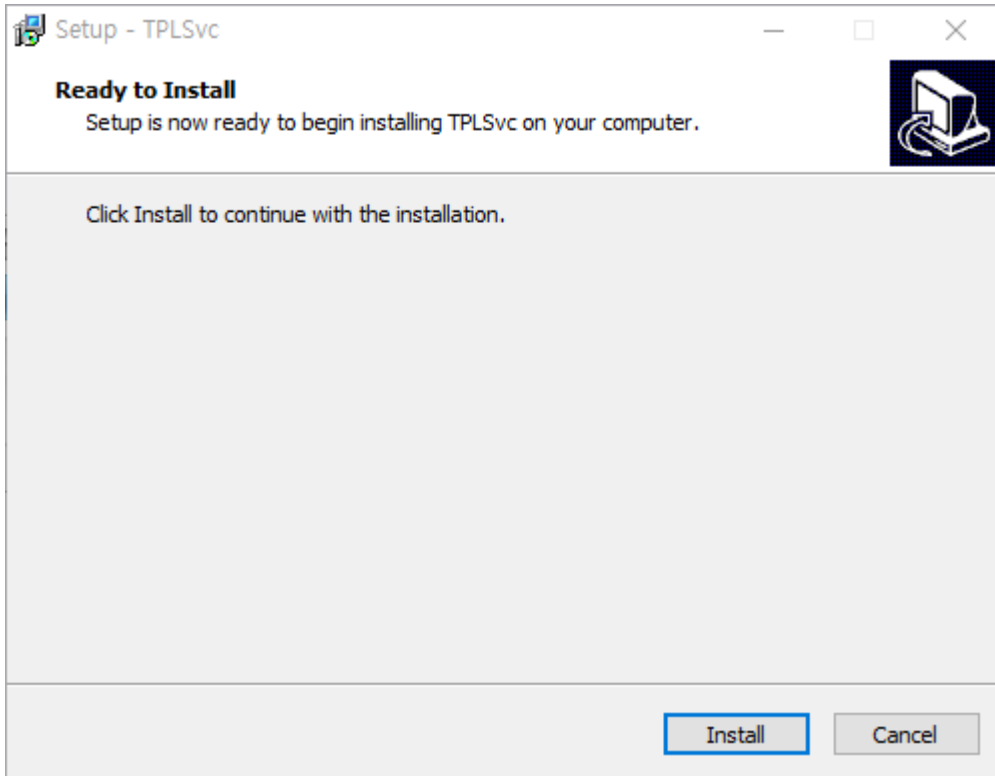


런처 서비스 설치 시 설치 마법사 화면이 표시되지 않도록 하려면 `/VERYSILENT`, `/SILENT` 옵션을 사용할 수 있습니다. `/VERYSILENT` 옵션은 설치 파일 실행 시 아무런 메시지 없이 서비스가 설치됩니다. `/SILENT` 옵션은 사용자의 조작 없이 설치 진행 상태만 표시하고 서비스가 설치됩니다.

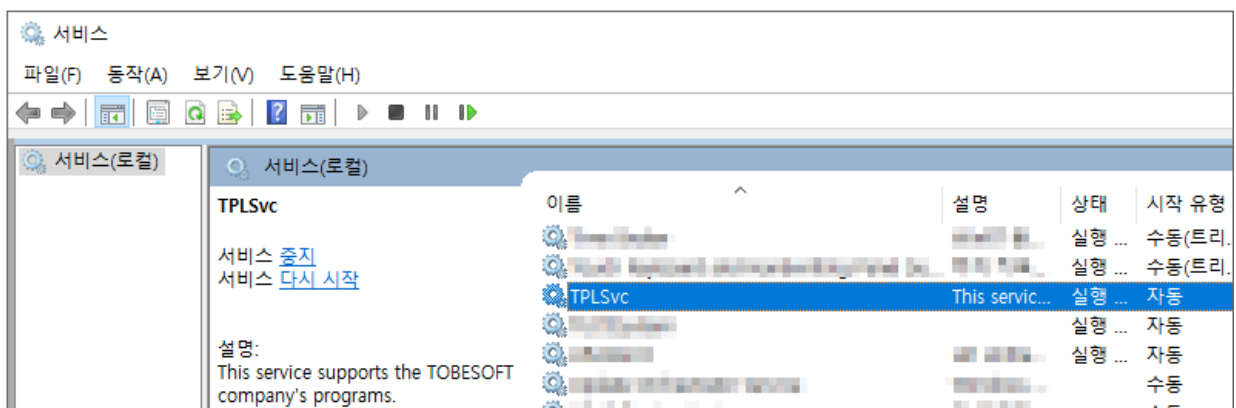
TPLSvc_Setup.exe /VERYSILENT

TPLSvc_Setup.exe /SILENT

- ② 설치 마법사 화면이 실행됩니다. [InstalI] 버튼을 클릭하고 설치를 진행합니다. 설치가 완료되면 윈도우 서비스로 등록되고 자동으로 실행됩니다.



[제어판 > 시스템 및 보안 > 관리자 도구 > 서비스] 항목에서 등록된 서비스가 동작하는 것을 확인할 수 있습니다. 해당 창에서 서비스를 중지하거나 다시 시작할 수 있습니다.

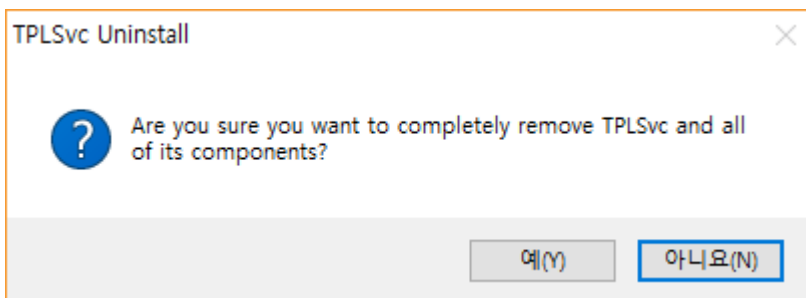


3.1.2 PC에서 런처 서비스 삭제하기

- 1 설치된 앱(프로그램) 목록에서 TPLSvc 항목을 찾아 [제거] 버튼을 클릭합니다.



- 2 삭제 여부를 확인합니다. [예] 버튼을 클릭하면 삭제가 진행됩니다. 등록된 서비스가 해제되고 런처 서비스가 삭제됩니다.



3.1.3 특정 포트에서 서비스가 실행하도록 지정하기

런처 서비스는 사용하는 포트가 지정되어 있는데, 이를 변경하고자 하는 경우에는 명령 프롬프트에서 직접 파라미터를 지정해 서비스를 실행할 수 있습니다.

- 1 런처 서비스가 설치된 경로를 확인합니다. 아래 경로에서 설치된 실행 파일을 확인할 수 있습니다.

C:\Program Files (x86)\TPLSvc\TPLSvc.exe

- 2 관리자 권한으로 명령 프롬프트를 실행하고 아래와 같이 파라미터를 지정해 윈도우 서비스를 해제할 수 있습니다. 런처 서비스를 삭제하는 것이 아니라 등록된 서비스만 해제합니다.

```
TPLSvc_Setup.exe -r false
```

- ③ 서비스를 다시 등록하기 위해서는 아래와 같이 실행합니다.

```
TPLSvc_Setup.exe -r true
```

런처는 통신 방식에 따라 지정된 포트 중 하나를 사용합니다. 런처가 실행되는 시스템에 다른 서비스가 해당 포트를 이미 사용하고 있다면 포트를 변경해 사용할 수 있습니다. 이런 경우에는 서비스 등록 시 포트를 지정하는 파라미터(-p 포트번호1, -p 포트번호2 ... -p 포트번호n)를 추가로 지정합니다.

통신 방식	기본 포트	포트 지정 파라미터
HTTP	7895 ~ 7935	-p
HTTPS	7936 ~ 7966	-https
WS	7970 ~ 7981	-wsp
WSS	7982 ~ 7992	-wssp

예를 들어 HTTP 통신 시 8080포트와 8888포트를 사용하고자 한다면 아래와 같이 서비스를 등록합니다.

```
TPLSvc_Setup.exe -r true -p 8080 -p 8888
```

```

C:\Program Files (x86)\TPLSvc>TPLSvc.exe -r false
Service stopped successfully

C:\Program Files (x86)\TPLSvc>TPLSvc.exe -r true -p 8080 -p 8888
Service start pending...
Service started successfully.
  
```

3.2 동작 방식

런처 서비스와 웹브라우저 간 동작 방식을 간단하게 설명하면 아래 표와 같습니다.

런처 서비스		웹브라우저
(2) id 할당 및 저장 후 id 반환	↔	(1) 신규 id 요청 (create)
(4) 필수 속성 저장 후 결과값 반환	↔	(3) 필수 속성 설정 (setproperty)
메서드 실행 후 결과값 반환	↔	메서드 실행 요청 (method)
속성 정보 반환	↔	속성 정보 요청 (getproperty)
이벤트 정보 반환	↔	이벤트 정보 요청 (event)

런처 서비스		웹브라우저
id에 해당하는 정보삭제	↔	id에 해당하는 정보삭제요청 (destroy)

- 웹브라우저에서 런처 서비스 실행을 위해 형식에 맞는 데이터를 전송해야 합니다.
- 데이터는 JSON 형식으로 작성된 텍스트로 전송하며 결과값도 같은 형식입니다.
- id 값은 런처 실행을 위한 필수 정보입니다. 반드시 신규 id 요청을 전송해야 합니다. (1, 2번 동작)
- 필수 속성 정보를 설정해야 합니다. (3, 4번 동작)
- 대소문자를 구분하므로 주의해야 합니다.

3.3 기본 설정

3.3.1 주소 설정

주소 설정은 필수 부분과 추가 부분으로 구분할 수 있습니다. 필수 부분에 추가 부분을 조합해 사용합니다.

필수 부분	로컬주소, 포트, 예약어(launcher)
추가 부분	플랫폼명, 타임스탬프

```
http://127.0.0.1:7895/launcher/nexacro/123456789
http://127.0.0.1:7895/launcher/xplatform/123456789
http://127.0.0.1:7895/launcher/miplatform/123456789
```

- 로컬주소(127.0.0.1)과 예약어(launcher)은 수정할 수 없습니다.
- 설치 시 특정 포트를 열어놓은 경우에는 해당 포트 번호를 사용할 수 있습니다.
- 타임스탬프는 웹브라우저 캐시 동작으로 통신이 되지 않는 것을 방지합니다.

3.3.2 기본 요소 설정

기본적으로 설정하는 요소는 platform, action, id, value입니다. 각 용도에 맞게 설정합니다.

platform

플랫폼을 나타내는 문자열입니다. 제품에 따라 아래 3가지 중 하나를 지정할 수 있습니다.

nexacro, xplatform, miplatform

action

런처 서비스에 요청할 동작 문자열을 지정합니다. 아래와 같은 동작을 지정할 수 있습니다.

동작	설명
create	런처 서비스에서 최초 id를 할당받을 때
destroy	할당받은 id에 해당하는 정보를 런처 서비스에서 삭제
setproperty	속성 설정
getproperty	속성 정보 확인
method	메서드 실행
event	이벤트 관련 정보 확인
check	런처 서비스 버전 정보 확인

id

런처 서비스로부터 할당받은 문자열이며 반드시 최초 동작 시 요청해야 합니다.

value

동작에 대한 구체적인 JSON 형식의 정보이며 동작에 따라 구성이 다릅니다.

최초 id를 할당받기 위한 JSON : value 없음, 결과로 id가 전달됨

```
var objNexacro = new Object();
objNexacro.platform = 'nexacro';
objNexacro.action = 'create';
//objXP.id = ''; // 통신이 성공하면 정보가 채워지므로 생략해도 무관합니다.
```

관련 속성 설정을 위한 JSON

```
var objNexacro = new Object();
objNexacro.platform = 'nexacro';
objNexacro.action = 'setproperty';
objNexacro.value = {"property name" : property value, ... };
```

설정된 속성값을 확인하기 위한 JSON

```
var objNexacro = new Object();
objNexacro.platform = 'nexacro';
objNexacro.action = 'getproperty';
```

```
objNexacro.value = {"property name" : "", ... };
```

메서드 호출을 위한 JSON

```
var objNexacro = new Object();
objNexacro.platform = 'nexacro';
objNexacro.action = 'method';
objNexacro.value = {"method name": {"param": method parameter array, "result": }};
```

이벤트 정보를 확인하기 위한 JSON

```
var objNexacro = new Object();
objNexacro.platform = 'nexacro';
objNexacro.action = 'event';
objNexacro.value = {"event name" : {...}};
```

런처 서비스 버전 정보 확인을 위한 JSON

```
var objNexacro = new Object();
objNexacro.platform = 'nexacro';
objNexacro.action = 'check';
objNexacro.value = {"getlauncherversion": {}};
```

3.3.3 데이터 전송

데이터 전송은 HTTP/HTTPS 통신(GET, POST)과 WebSocket 통신으로 구분합니다.

GET 방식은 주소 뒤에 ?와 JSON 형식의 문자열이 첨부됩니다

```
var objNexacro = new Object();
var xhrObject = new XMLHttpRequest();
xhrObject.onreadystatechange = resultProcess;
var jsonData = JSON.stringify(objNexacro);
var openurl = "http://127.0.0.1/launcher/nexacro/"+new Date().getTime();
xhrObject.open("GET", openurl+"?" +jsonData, "true");
xhrObject.send(null);
```

POST 방식은 send 메서드 파라미터에 JSON 형식의 문자열이 첨부됩니다

```
var objNexacro = new Object();
var xhrObject = new XMLHttpRequest();
xhrObject.onreadystatechange = resultProcess;
```

```

var jsonData = JSON.stringify(objNexacro);
var openurl = "http://127.0.0.1:80/launcher/nexacro/"+new Date().getTime();
xhrObject.open("POST", openurl, "true");
xhrObject.send(jsonData);

```

WebSocket 통신 시 데이터 전송

```

var objNexacro = new Object();
var openurl = "ws://127.0.0.1:7970/launcher/nexacro/"+new Date().getTime();
var objWebSocket = new WebSocket(openurl);
var jsonData = jsonData = JSON.stringify(objNexacro);
if (objWebSocket != null) {
    objWebSocket.send(jsonData);
} else {
    return null;
}

```

데이터 전송은 JSON 형식의 문자열을 구성하면 어떤 식으로든 처리할 수 있습니다. stringify 메서드를 지원할 수 없거나 직접 설정하기 원한다면 다음과 같은 형태로 문자열을 만들어 처리합니다.

```

create
'{"platform":"nexacro","action":"create"}'

```

```

launch method
id값은 create에서 받은 값으로 치환해야 합니다.
'{"platform":"nexacro","id":"12345678","action":"method", "value":{"launch":
null }}'

```

3.3.4 통신 결과 처리

JSON 형식의 문자열 데이터를 적절히 변환해 처리합니다.

HTTP/HTTPS 통신 결과 처리

```

var result = xhrObject.responseText; // xhrObject는 XMLHttpRequest임
var objResult = eval('(' + result + ')');

```

WebSocket 통신 결과 처리

```

function onMessage(evt) {
    var data = evt.data;

```

```
var objResult = eval('(' + result + ')');
...
}
```

처리결과가 성공이라면 result 문자열이 'success' 로 처리됩니다.

```
if(objResult.result == "success" && objResult.id.length > 0)
{
    objNexacro = objResult; // objXP는 송신정보를 담고 있었던 객체임
} else {
    window.console.log("error!!!");
}
```

3.4 TPLSvcAgent 사용

특정 환경에서 런처 서비스 실행 시 윈도우 운영체제 보안 제약 사항으로 서비스 실행에 필요한 일부 사용자 정보를 확인할 수 없습니다. 이를 우회하기 위해 별도의 에이전트 파일(TPLSvcAgent.exe)을 제공하며 multisession 속성 설정값에 따라 사용자 정보 필요시 에이전트 파일을 실행해 정보를 확인합니다.



TPLSvcAgent.exe 파일과 multisession 속성은 1.1.0.29 이상 버전에서 지원합니다.

3.4.1 지원 기능

multisession 속성값을 true로 설정하고 에이전트 실행 시 아래와 같은 기능을 지원합니다.

- 관리자 권한으로 nexacro.exe 실행
- 프록시 서버 사용 시 바로가기 이미지 파일 내려받기 지원
- 가상환경(citrix, vmware, windows remote desktop)에서 사용자 정보 확인



가상환경 설정에 따라 사용자 정보 확인 기능을 사용하지 못할 수도 있습니다.

3.4.2 사용 방법

에이전트 실행을 활성화하기 위해서는 런처 서비스 구동을 위한 setproperty 함수에서 multisession 속성값을 true로 설정합니다.

```
objNexacro.action = 'setproperty';
objNexacro.value = {
...
    "multisession": true,
...
};
```

가상환경에서 사용자 정보 확인 시에는 최초 id 할당 시(create action) 콜백 함수에서 id 값을 document의 title 속성값으로 설정해 주어야 합니다.

```
function createProcess() {
...
    if (bMultiSession == true)
    {
        top.document.title = objNexacro.id;
    }
...
}
```

4.

지원기능 및 사용예제

4.1 지원기능

4.1.1 속성

Property	default	Data Type	설명
key	없음	string	필수속성입니다. 서비스의 Key값을 설정 동일 start.json 상의 유일한 값이어야 함
bjson	없음	string	필수속성입니다. 서비스의 start.json 경로를 지정
commthreadcount	3	int	넥사크로 Engine이 사용하는 통신 Thread 개수를 지정 여러 개의 통신을 동시에 사용하기를 원할 경우 숫자를 높여 주면 빠른 응답 결과를 얻을 수 있음. 하지만 네트워크 속도가 느린 경우는 숫자를 높여도 응답 속도에 의미 있는 영향을 미 치지 않음 최댓값: 10
commthreadwaittime	-1	int	넥사크로 Engine이 사용하는 통신 Thread 종료 전 대기 시 간(초)을 지정 통신 Thread를 사용하지 않을 경우 해당 Thread는 종료 처 리됨 기본값 -1 설정 시 넥사크로 Engine 내부에 설정된 대기 시간 후 종료 처리
globalvalue	없음	string	넥사크로 Engine이 구동될 때 globalvariable에 추가될 변수 를 설정
splashimage	없음	string	넥사크로 Engine이 Loading되는 동안 보여줄 스플래시 이미 지 경로를 지정 지정하지 않을 경우 넥사크로 기본이미지가 출력되며, 이미지는 화면의 중앙에 표시됨
onlyone	false	boolean	key와 bjson 값에 매칭되는 Instance를 하나만 띄울 것인지 에 대한 여부를 설정

Property	default	Data Type	설명
version	없음	string	넥사크로 N 동작 버전 지정 - "21": 21.0.0.X - "24": 24.0.0.X
screenid	없음	string	앱이 실행되는 Screen ID를 설정
enginesetupkey	없음	string	필수속성입니다. 실행경로로 사용할 특정 버전의 넥사크로 ProductKey 값을 지정. 사이트에서 별도 작성한 Setup Module을 사용하는 경우, 이 Property에 Setup을 만들 때 부여되는 Product Key를 설정함. 설정하지 않을 경우 서비스가 제대로 동작하지 않을 수 있음
setupresource		boolean	SetupResource.zip 파일을 내려받는 기능의 사용 여부를 설정 1.1.0.9 이후 버전부터 지원
componentpath		string	컴포넌트가 저장될 경로를 설정
loadingimage		string	화면, 데이터 로딩 시 표시되는 이미지를 설정
splashmessage		string	splashimage 사용 시 메시지 출력 여부 설정 'none' 또는 ''(빈 값) 설정 시 메시지를 출력하지 않음
multisession	false	boolean	TPLSvcAgent 사용 여부를 설정 1.1.0.29 이후 버전부터 지원



속성값에 특수문자가 포함된 경우 예기치 못한 결과가 발생할 수 있습니다.



key, bson 속성값에 세미콜론(;) 또는 ".Run" 문자열이 포함된 경우 보안상 문제가 될 수 있어 해당 문자열을 삭제하여 처리합니다 (1.1.0.12 이후 버전부터 적용됩니다).

key, bson 속성값에 콤마(,), TAB 문자(\t), 개행(\n, \r) 또는 ". Run" 처럼 점(.)과 "run" 문자열 사이의 공백문자가 포함된 경우 보안상 문제가 될 수 있어 해당 문자열을 삭제하여 처리합니다 (1.1.0.16 이후 버전부터 적용됩니다).



key 속성값에 상대경로를 표시하는 "../", "..\w" 문자열이 포함된 경우 보안상 문제가 될 수 있어 해당 문자열을 삭제하여 처리합니다 (1.1.0.27 이후 버전부터 적용됩니다).



splashimage 속성에 다음 확장자를 가진 파일은 지정할 수 없습니다 (1.1.0.16 이후 버전부터 적용됩니다).

bat, bin, cmd, com, cpl, exe, gadget, inf1, ins, inx, isu, job, jse, lnk, msc, msi, msp, mst, paf, pif, ps1, reg, rgs, scr, sct, shb, shs, u3p, vb, vbe, vbs, vbscript, js, ws, wsf, wsh, dll, hta



componentPath 속성 설정 시 아래와 같은 제약이 있습니다 (1.1.0.23 이후 버전부터 적용됩니다).

- 허용된 alias만 사용해서 값을 설정할 수 있습니다.
- 경로에 ".."을 포함해 상위 폴더를 지정할 수 없습니다.



commthreadcount, commthreadwaittime 속성 설정 시에는 런처 서비스 1.1.0.36 이상, 제품(넥사크로 Engine) 버전은 21.0.0.1000 이상이어야 합니다.

4.1.2 메서드

Method	Result	Parameter	Description
launch	없음	없음	넥사크로 Engine을 NRE로 실행시킴.
launchasadmin	없음	없음	관리자 권한으로 넥사크로 Engine을 NRE로 실행시킴. 1.1.0.43 이후 버전부터 지원(multisession 속성값이 true인 경우에는 지원하지 않음)
makeshortcut	없음	strShortCutName strIconPath strIconTargetPath strPosition bAllUser	단축아이콘을 생성함. - strShortCutName:바로가기 아이콘명 - strIconPath: 바로가기 아이콘 파일을 내려받을 수 있는 서버 경로 - strIconTargetPath: 바로가기 아이콘을 내려받은 로컬 경로 (alias 허용) - strPosition: 바로가기 생성위치 "startmenu" / "startup" / "programs" / "desktop" 중 택일함 잘못된 값이 들어오면 "desktop"으로 처리함 - bAllUser :전체 사용자가 사용할 수 있도록 생성할지에 대한 여부(default는 false)
isexistshortcut	boolean	strShortcutName strPosition bAllUser	입력받은 인자에 해당하는 바로가기 아이콘이 존재하는지 확인. - strShortCutName:바로가기 아이콘명 - strPosition: 바로가기 생성위치 "startmenu" / "startup" / "programs" / "desktop" 중 택일함 잘못된 값이 들어오면 "desktop"으로 처리함 - bAllUser :전체 사용자가 사용할 수 있도록 생성할지에 대한 여부(default는 false)
getengineversion	string	strEngineKeyName	시스템에 설치된 엔진 키에 해당하는 엔진 설치 버전을 가져옴.

Method	Result	Parameter	Description
			strEngineKeyName: Engine Setup Key
download	없음		앱 실행 시 사용되는 자원을 다운로드.
deleteshortcut	boolean	strShortcutName strPosition bAllUser	바로가기 아이콘을 삭제. - strShortCutName: 바로가기 아이콘명 - strPosition: 바로가기 생성위치 "startmenu" / "startup" / "programs" / "desktop" 중 택일함 잘못된 값이 들어오면 "desktop"으로 처리함 - bAllUser :전체 사용자가 사용할 수 있도록 생성할지에 대한 여부(default는 false)
addWebInfo	없음	strCookie	현재 브라우저의 쿠키정보를 자동으로 global variable의 cookie변수에 추가. strCookie: 브라우저의 쿠키정보
downloadresource		param	SetupResource.zip 파일이 위치한 서버 경로를 지정 1.1.0.9 이후 버전부터 지원



메서드 파라미터값에 특수문자가 포함된 경우 예기치 못한 결과가 발생할 수 있습니다.



makeshortcut 메서드 실행 시 문자열로 전달되는 파라미터값에 아래 항목이 포함된 경우 보안상 문제가 될 수 있어 해당 문자열을 삭제하여 처리합니다 (1.1.0.15 이후 버전부터 적용됩니다).

- 세미콜론(;)
 - ".Run" 문자열



makeshortcut, deleteshortcut, isexistshortcut 메서드 실행 시 문자열로 전달되는 strShortCutName 파라미터값에 아래 항목이 포함된 경우 보안상 문제가 될 수 있어 해당 문자열을 삭제하여 처리합니다 (1.1.0.16 이후 버전부터 적용됩니다).

- 세미콜론(;)
 - 콤마(,)
 - TAB 문자(\t)
 - 개행(\n, \r)
 - ". Run" 문자열(.과 "Run" 문자열 사이 공백문자가 포함된 경우)



makeshortcut, deleteshortcut 메서드 실행 시 문자열로 전달되는 파라미터값에 아래 항목이 포함된 경우 보안상 문제가 될 수 있어 해당 문자열을 삭제하여 처리합니다 (1.1.0.45 이후 버전부터 적용됩니다)

다).

- WScript.CreateObject
- WScript.Shell
- CreateShortcut

4.1.3 이벤트

Event	parameter	Occurs
error	nError strMsg	런처 서비스 실행 중에 에러가 발생하면 세팅되는 이벤트 - nError: 에러코드 - strMsg: 에러 메시지

Error Code	Error Message	Description
102	Not exist key. Please input key.	key 값을 설정하지 않은 경우
103	Not exist bootstrap. Please input bootstrap.	bjson 값을 설정하지 않은 경우
104	Fail to download.	이미지 파일을 내려받을 수 없는 경우
105	Fail to load config file.	nexacro.xml 파일을 읽을 수 없는 경우
106	Fail to write config file.	nexacro.xml 파일을 기록할 수 없는 경우

4.1.4 참고

디지털 서명을 유지한 상태로 설치 파일을 배포

SetupRuntime.exe 설치 파일을 실행하기 전에 SetupResource.zip 파일을 사용자 PC에 먼저 내려받아야 합니다.



디지털 서명을 유지한 상태로 설치 파일을 배포하는 방식은 런처서비스 버전 1.1.0.9 이후 버전부터 지원됩니다.

setupresource 속성 설정

런처 서비스 구동을 위한 setproperty 함수에서 setupresource 속성값을 아래와 같이 true로 설정합니다.

```
objNexacro.action = 'setproperty';
if (action_sub == 'basic') {
    objNexacro.value = {
        "key": strKey,
        "bjson": strBjson,
        "splashimage": strSplashImage,
        "componentpath": strComponentPath,
        "loadingimage": strLoadingImage,
        "globalvalue": strGlobalValue,
        "onlyone": bOnlyone,
        "splashmessage": 'none',
        "setupresource": 'true',
        "enginesetupkey": strEnginesetupKey
    };
}
```

downloadresource 메서드 설정

start.json, SetupResource.zip 파일이 위치한 서버 경로를 인자값으로 설정합니다. downloadresource 메서드가 실행되면 설정된 경로에서 SetupResource.zip 파일을 "%USERAPP%SetupResource" 경로에 내려받습니다.

```
else if (action == 'downloadresource') {
    objNexacro.value = {
        "downloadresource": {
            "param": ["http://127.0.0.1:8080/test/"]
        }
    };
}
```

downloadresource 메서드 실행 이후에 SetupRuntime.exe 파일을 내려받는 스크립트가 동작하도록 합니다.

4.2 사용예제

런처 실행 스크립트를 어떻게 구현하는지 간략한 예제를 설명합니다. 예제는 자바스크립트로 구현되었으며 사용하는 환경에 맞게 수정해 사용할 수 있습니다.



아래 예제는 구글 크롬 브라우저에서 넥사크로플랫폼 앱이 동작하는 예제입니다. IE 브라우저에서 동작하는 예제는 전체 코드를 참조해주세요.

4.2.1 신규 id 요청

platform, action 요소 항목값을 채우고 신규 id를 요청합니다.

```
function start()
{
    objNexacro.platform = 'nexacro';
    objNexacro.action = 'create';
    sendData(true, true, createProcess);
}

function sendData(openpost, is_create, resultCallback)
{
    delete xhrCreateObject;
    xhrCreateObject = null;

    var sendObj = null;
    sendObj = new XMLHttpRequest();
    sendObj.reqType = 1;

    xhrCreateObject = sendObj;
    sendObj.onreadystatechange = resultCallback;
    var jsonData;
    jsonData = JSON.stringify(objNexacro);
    var timestamp = "/" + new Date().getTime();
    var send_url = openurl + ":" + openport + openurl_add + timestamp;
    console.log(send_url);

    sendObj.open("POST", send_url, "true");
    sendObj.send(jsonData);
    console.log(jsonData);
    return sendObj;
}

function createProcess()
```

```
{
  if ( xhrCreateObject.readyState == 4 || xhrCreateObject.reqType == 2) {
    console.log(xhrCreateObject.responseText);
  }
}
```

요청되는 jsonDate 항목은 아래와 같습니다. 요소 항목만 채워서 전송됩니다.

```
{"platform":"nexacro","action":"create"}
```

런처 서비스에서 요청을 받고 id를 할당해 반환합니다. id 항목과 result 항목을 확인합니다. id값은 이후 요청 시 활용하게 됩니다.

```
{"action":"create","id":"1522821857","platform":"nexacro","result":"success"}
```

4.2.2 필수 속성 설정

예제에서 [property setting(basic)] 버튼 클릭 시 동작하는 기능입니다. 속성 중에서 key, bjson, enginesetupkey 항목값을 채웁니다.

```
function do_property(test_action, action_sub)
{
  objNexacro.action = 'setproperty';
  objNexacro.value = { "key": "LauncherService", "bjson": "http://127.0.0.1:8080/start.json",
    "enginesetupkey":enginesetupkey};
  sendData(true, false, resultProcess);
}
```

넥사크로플랫폼 앱 배포 시 Local 설정으로 배포한 경우에는 start.json 경로를 로컬 주소로 지정해야 합니다. 이런 경우에는 아래와 같이 지정합니다.

```
function do_property(test_action, action_sub)
{
  objNexacro.action = 'setproperty';
  objNexacro.value = { "key": "LauncherService", "bjson": "D:\\nexacro\\17\\start.json", "
    enginesetupkey":enginesetupkey};
  sendData(true, false, resultProcess);
}
```

요청되는 jsonDate 항목은 아래와 같습니다. 이전 단계에서 확인된 id값을 설정하고 key, bjson, enginesetupkey 항목값이 채워져 있습니다.

```
{ "action": "setproperty", "id": "1522822903", "platform": "nexacro", "result": null, "value": { "key": "LauncherService", "bjson": "http://127.0.0.1:8080/start.json", "enginesetupkey": "{94FAAFF5-0E54-4539-AA67-*****}" }
```

런처 서비스에서 필수 속성값을 저장하고 성공 여부("result": "success")를 반환합니다.

```
{ "action": "setproperty", "id": "1522823068", "platform": "nexacro", "result": "success", "value": { "bjson": "http://127.0.0.1:8080/start.json", "enginesetupkey": "{94FAAFF5-0E54-4539-AA67-*****}", "key": "LauncherService" } }
```

4.2.3 메서드 실행

예제에서 [launch] 버튼 클릭 시 동작하는 기능입니다.

```
function do_method(test_action) {
    objNexacro.action = 'method';
    objNexacro.value = { "launch": null };
    sendData(true, false, resultProcess);
}
```

이전 단계에서 key, bjson, enginesetupkey 항목값을 전송했고 런처 서비스에서 해당하는 값을 저장하고 있습니다. id값을 키값으로 값을 저장하고 있기 때문에 이후 메서드를 실행하거나 다른 동작을 할 때는 id값을 기준으로 동작하게 됩니다.

```
{ "action": "method", "id": "1522823068", "platform": "nexacro", "result": null, "value": { "launch": null } }
```

런처 서비스에서 메서드를 실행하고 성공 여부("result": "success")를 반환합니다. 정상적으로 메서드가 실행되었다면 운영체제에 설치된 앱이 실행됩니다.

```
{ "action": "method", "id": "1522823068", "platform": "nexacro", "result": "success", "value": { "launch": null } }
```

4.2.4 전체 코드

아래 링크에서 사용 예제 코드를 내려받을 수 있습니다.

[localtest_nexacro_sample.html](#)

예제 코드 내에는 아래와 같은 문제 처리 방법이 포함되어 있습니다.

- 인터넷 익스플로러(IE) 8, 9 버전에서 Cross Domain 문제 회피 방법
XMLHttpRequest.open 메서드 사용 시 액세스를 허용하지 않는다는 에러 메시지가 나올 경우 XMLHttpRequest 객체 대신 XDomainRequest 객체를 사용하면 에러가 발생하지 않습니다.

```
if (window.XMLHttpRequest) {
    sendObj = new XMLHttpRequest();
    sendObj.reqType = 1;
} else if (window.XDomainRequest) {
    sendObj = new XDomainRequest();
    sendObj.reqType = 2;
}
```

- 런처 서비스에서 열린 포트를 검색한 후 사용하는 방법
모든 경우에 사용해야 하므로 action이 'create'인 경우에 지속적으로 통신시도를 해 결과값으로 id를 받았을 때 port 번호를 이후 통신에 사용하는 방법입니다.

```
function http_onerror()
{
    if (objNexacro.action == 'create') {
        if (findport == false) {
            if ( setport > 0) {
                setport = 0;
                openport = parseInt(getCookie("tplsvcopenport"), 10) | 0;
                if (openport > 0) {
                    sendData(true, true, createProcess);
                    return;
                }
            }
            openport = 7895;
        } else {
            openport++;
        }
    }
}
```

```
findport = true;

if (openport <= 7935) //신규 런처는 7895 ~ 7935 사이 하나의 port를 open 함;
    sendData(true, true, createProcess);
else
    alert("런처 서비스가 설치되지 않았거나 동작이 멈춰있는지 확인 바랍니다.");
}
}
```

파트 IV.

App Builder (Android, iOS, macOS)

5.

App 생성

App Builder 관련 문서

- 서버 설정/개발 가이드 > App Builder

[App Builder 설치](#)

[서버 환경 설정](#)

[디플로이 서버 설정](#)

[운영체제별 서명 설정](#)

[사용자 라이브러리 설정](#)

[넥사크로 라이브러리 설정](#)

[사용자 설정](#)

- 앱 배포 가이드 > App Builder

[App 생성](#)

[App 목록](#)

- 개발도구 가이드 > Build App 단계에서 설치 파일 만들기

[Build App 단계에서 설치 파일 만들기](#)

App Builder에서 앱을 빌드하기 위한 기본 단위는 "App"입니다. 새로운 앱을 빌드하기 위해서는 "App"을 생성해야 하며 생성된 "App"을 수정하거나 삭제할 수 있습니다. "App" 설정에 따라 Android, iOS, macOS 앱을 빌드하고 배포할 수 있습니다.

메뉴[BuildApp > App List] 항목에서 [New] 버튼을 클릭하면 새로운 "App"을 생성할 수 있는 창이 나타납니다.



5.1 앱 기본 설정값 입력하기

정보를 입력하고 [Save] 또는 [Build] 버튼을 클릭하기 전에는 "App"이 생성되지 않습니다.

← New App Save

1 Basic Property

*Project Name HelloWorld *Package Name com.nexacro.HelloWorld

*App Title HelloWorld

*Project URL <http://192.168.1.100:8080/appbuilder/archives/340> Options +

2 Nexacro Library ▼ +

3 Target OS Upload History

☐ Android ☒ iOS ☒ macOS

start_ios.json file upload ↑ detail

Nexacro Resource

	Nexacro App Resource File	Version	Upload	Download	Delete

App

	Name	Version

Signing Info

*Sign Name ▼ + Options +

Build

	항목	설명
1	Basic Property	빌드할 앱 기본 정보와 빌드 시 필요한 설정을 지정합니다.
2	Nexacro Library	넥사크로 라이브러리를 지정합니다.
3	Target OS	앱을 배포할 대상 운영체제를 선택하고 빌드 시 필요한 설정을 지정합니다.

5.1.1 Basic Property

1 Basic Property

Project Name **1** HelloWorld

Package Name **2** com.nexacro.HelloWorld

App Title **3** HelloWorld

Project URL **4** <http://nexacro.com:8080/appbuilder/archives/340>

	항목	설명
1	Project Name	앱 프로젝트 이름을 입력합니다.
2	Package Name	Package 이름을 입력합니다.
3	App Title	빌드할 앱 이름을 입력합니다.
4	Project URL	넥사크로 앱이 배포된 URL을 입력합니다. URL 입력 시 마지막 슬래시 표기는 생략합니다.



Package Name은 빌드한 앱을 구별하는 식별자입니다. 일반적으로 도메인 주소를 반대로 기재한 형태로 Package Name을 지정합니다.

예) com.nexacro

Package Name을 하나의 단어로만 입력하는 경우에는 빌드 시 오류가 발생할 수 있습니다.



Target OS에 iOS가 포함된 경우에는 Provisioning Profile 발급 요청시 입력한 Package 명과 같은 값으로 Package Name을 입력하여야 합니다.

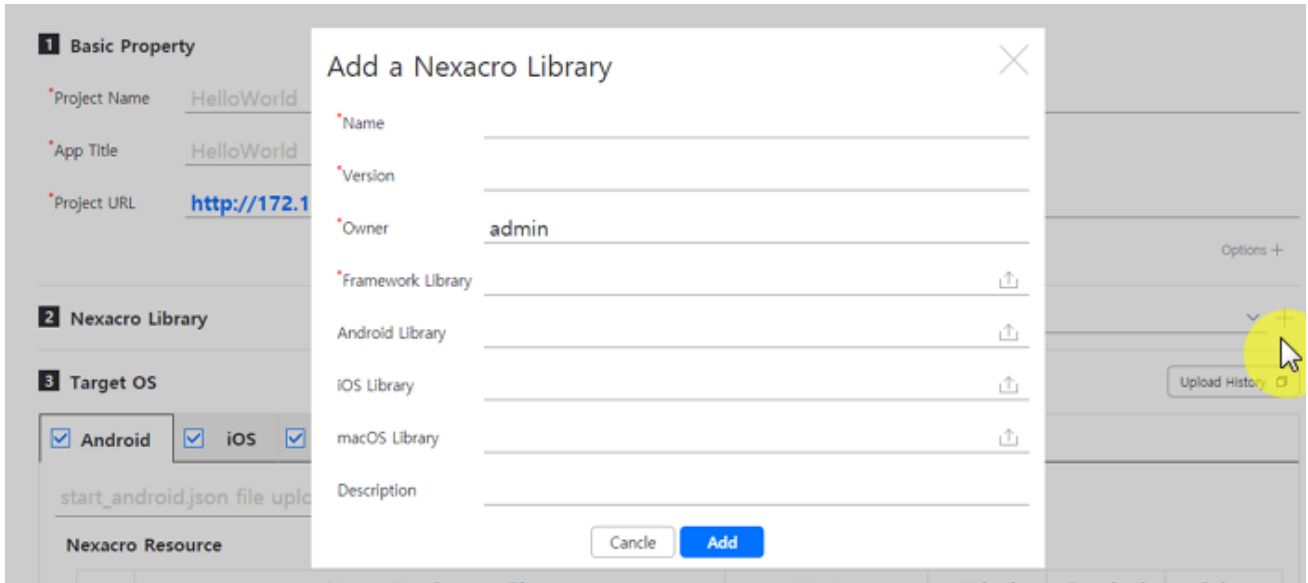
5.1.2 Nexacro Library

2 Nexacro Library

Test Library

	항목	설명
	Nexacro Library	넥사크로 라이브러리를 선택합니다. 앱 개발 시 사용한 넥사크로와 같은 버전의 라이브러리를 선택해야 합니다.

[+] 버튼을 클릭하고 새로운 라이브러리를 추가할 수 있습니다.



	항목	설명
1	Name	라이브러리 이름을 입력합니다.
2	Version	라이브러리 버전을 입력합니다.
3	Owner	소유자 정보를 표시합니다.
4	Nexacro Framework File	배포되는 nexacrolib.zip 파일을 업로드합니다. 업로드한 항목은 마우스로 클릭 시 파일을 내려받을 수 있습니다.
5	Version	업로드한 Nexacro Framework File 버전입니다. Nexacro Framework File에 포함된 JSON 파일에서 버전 정보를 확인하고 표시합니다.
6	Library File	라이브러리 파일을 업로드합니다. 예) Android : nexacroN_Android_2021xxxx_1.zip iOS : nexacroN_iOS_2021xxxx_1.zip macOS : nexacroN_macOS_2021xxxx_1.zip 업로드한 항목은 마우스로 클릭 시 파일을 내려받을 수 있습니다. - Version: Library File에 포함된 JSON 파일에서 버전 정보를 확인하고 표시합니다. - BuildProcessor: 라이브러리 Version 정보에 따라 빌드프로세서 선택해서 표시합니다.
7	Description	라이브러리에 대한 설명을 입력합니다.

5.1.3 Target OS

3 Target OS Upload History

☒ Android ☐ iOS ☐ macOS

start_android.json detail

Nexacro Resource

	Nexacro App Resource File	Version	Upload	Download	Delete
	Archive00.xzip	0			
	Resource.zip	0			

App

	Name	Version
--	------	---------

Signing Info

*Sign Name android-signing +

Options +

앱을 빌드할 대상 운영체제를 선택합니다. 선택한 운영체제에 대한 세부 설정을 지정할 수 있습니다. 넥사크로 스튜디오가 아니라 관리 콘솔에서 직접 리소스를 업로드하는 경우에는 선택한 운영체제에 맞는 start_(os_name).json 파일을 등록하면 [Nexacro application resource] 항목에 필요한 리소스 목록을 출력하고 리소스 파일을 업로드할 수 있습니다.

[detail] 버튼을 클릭하면 json 파일 내용을 표 형태로 보여줍니다.

3 Target OS

☒ Android ☒ iOS ☒ macOS

start_android.json detail

Nexacro Application Resource

android

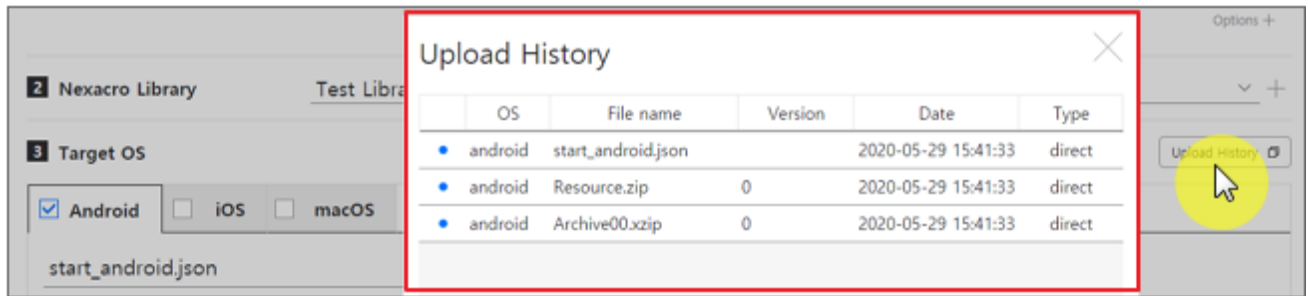
os	update_url	timeout	retry	auto_update
android	✓	30	3	auto

common

Nexacro Application Resource

type	file	target_path	version	description
Module	nexacrolib.zip	✓	0	
File	Archive00.xzip	✓	0	
Resource	Resource.zip	✓	0	

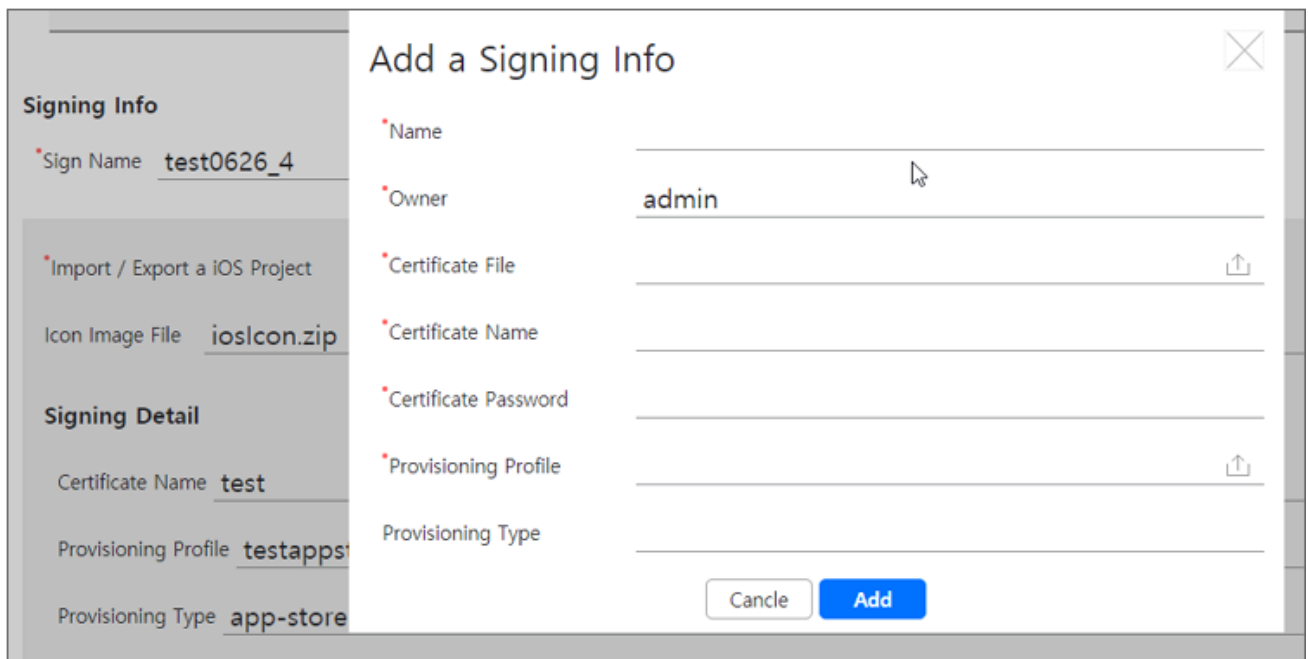
[Upload Histroy] 버튼을 클릭하면 리소스 파일 등을 업로드한 이력을 확인할 수 있습니다.



macOS에서 Singing Info 정보를 "none" 항목으로 선택한 경우 Singing Info를 포함하지 않고 앱 빌드를 처리합니다. 빌드는 정상적으로 동작하지만 앱 설치 후 실행 시 보안 관련 설정을 변경해주어야 합니다.

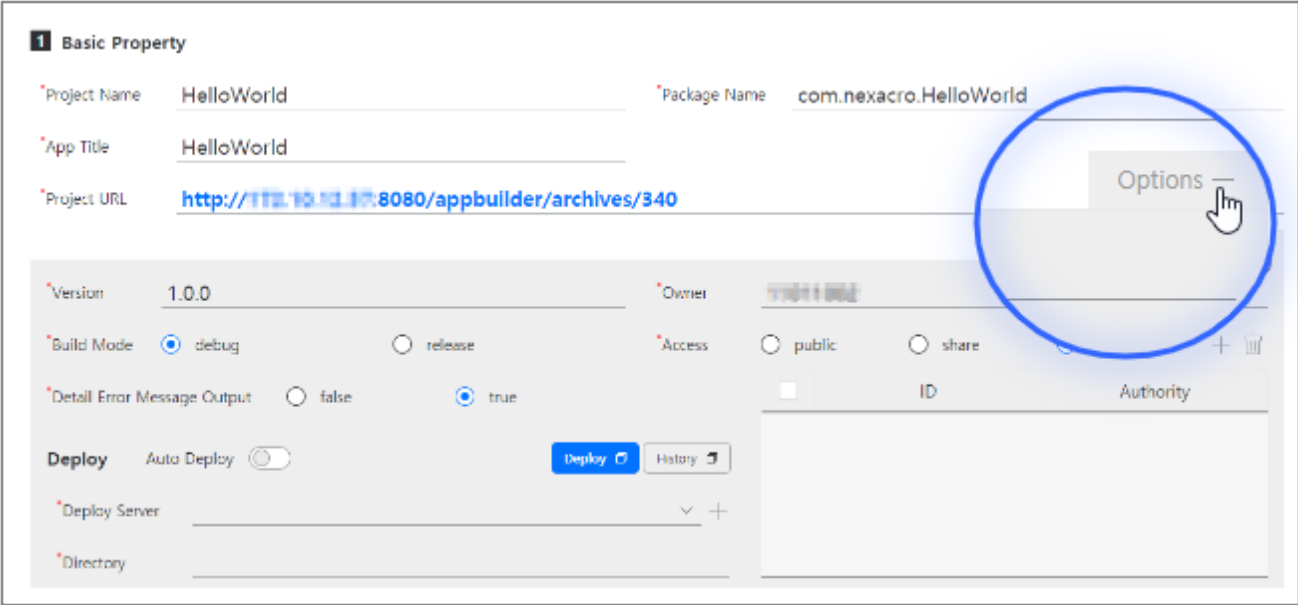
https://support.apple.com/kb/PH14369?locale=en_US

Singing 정보를 선택하거나 새로 등록할 수 있습니다. Sing Name 항목 옆에 [+] 버튼을 클릭하면 Target OS에 따라 정보를 입력할 수 있습니다.

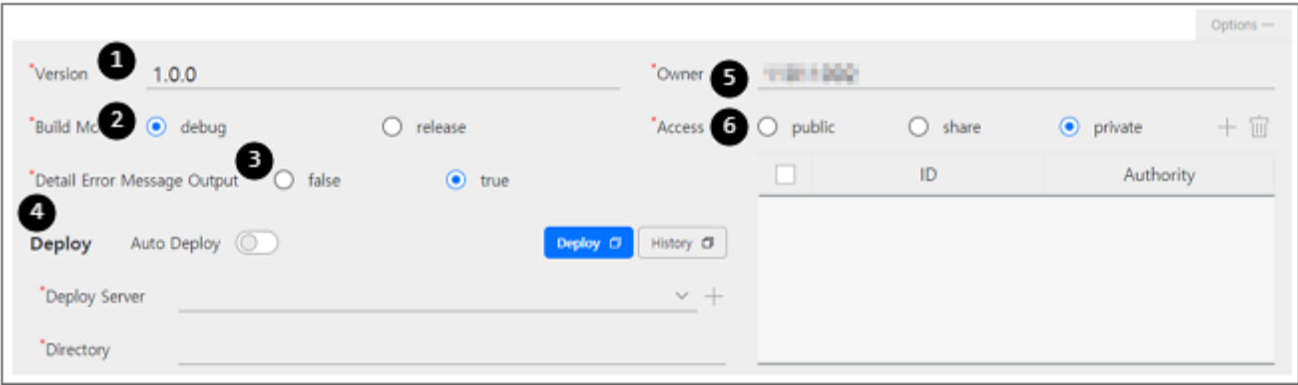


5.2 추가 옵션 설정하기

기본 설정값만 입력하면 앱 빌드 작업까지 진행할 수 있습니다. 각 항목별로 추가적인 옵션을 설정하려면 항목 오른쪽 아래에 있는 [Option +] 버튼을 클릭하고 옵션 항목을 펼친 후에 설정할 수 있습니다.



5.2.1 Basic Property Options



	항목	설명
1	Version	빌드할 앱 버전을 입력합니다.
2	Build Mode	빌드 모드를 선택합니다. "debug" 모드 선택 시 ADB(Android Debug Bridge)를 통한 디버깅 작업을 수행할 수 있습니다.
3	Detail Error Message	앱 로딩 중 발생하는 오류 메시지를 사용자에게 노출할지 설정
4	Deploy	디플로이 서버와 서버 경로를 선택하거나 추가합니다. Auto Deploy: 항목 선택 시 빌드 후 바로 디플로이 서버에 배포할 수

	항목	설명
		있습니다. [Deploy] 버튼 클릭 시 지정한 서버에 리소스를 배포합니다. [History] 버튼 클릭 시 디플로이 이력을 확인할 수 있습니다. [+] 버튼 클릭 시 새로운 디플로이 서버 정보를 추가할 수 있습니다.
5	Owner	"App" 소유자를 지정합니다. (로그인 계정으로 자동 입력됩니다).
6	Access	"App" 사용 권한을 지정합니다. 사용 권한에 따라 [App List]에 표시되는 목록이 달라집니다. - share: 지정된 사용자만 사용할 수 있습니다. - public: 모든 사용자가 사용할 수 있습니다. - private: Owner만 사용할 수 있습니다.

5.2.2 Target OS > Android Options

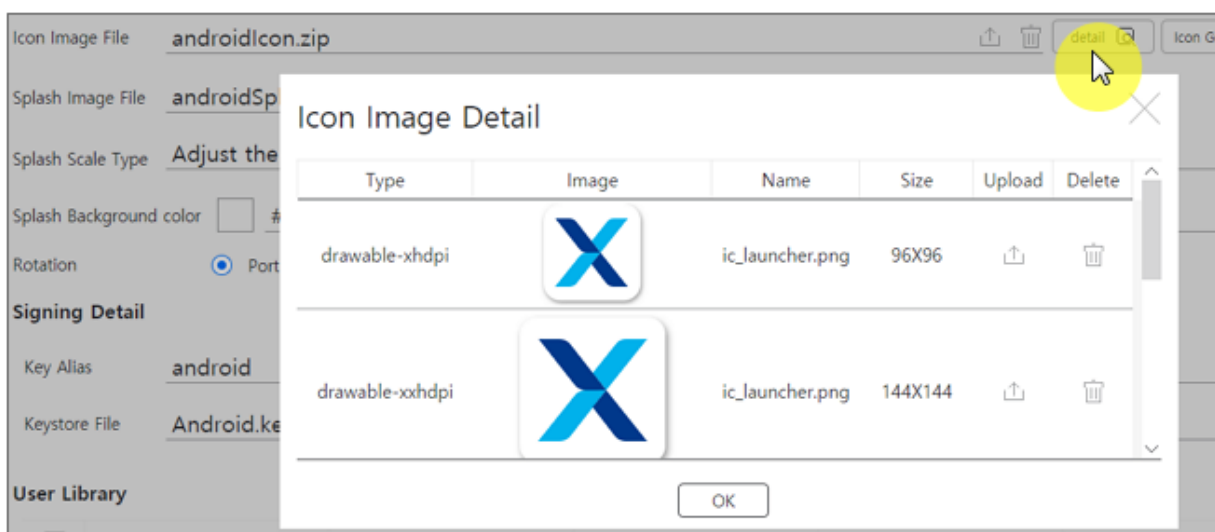
The screenshot shows the 'Options' dialog for 'Import / Export a Android Project'. It contains several sections with input fields and buttons. Numbered callouts (1-9) point to specific elements:

- 1**: 'Import / Export a Android Project' header.
- 2**: 'Icon Image File' input field with value 'androidIcon.zip'.
- 3**: 'Splash Image File' input field with value 'androidSplash.zip'.
- 4**: 'Splash Scale Type' dropdown menu with value 'Adjust the aspect ratio. stretch to fit the screen size'.
- 5**: 'Splash Background color' input field with value '#ffffff'.
- 6**: 'Rotation' section with radio buttons for 'Portrait' (selected), 'Landscape', and 'Sensor'.
- 7**: 'Signing Detail' section header.
- 8**: 'User Library' section header.
- 9**: 'Device API' section header.

Other visible elements include 'Export', 'Import', and 'Revert' buttons at the top; 'detail' and 'Icon Generate' buttons next to the image file inputs; and table headers for 'User Library' (Name, Version, Android Class, Date, Owner) and 'Device API' (Name, Key Type, Value, File Name, File).

	항목	설명
1	Import / Export	생성된 "App" 정보를 안드로이드 스튜디오에서 편집할 수 있는 프로젝트 형태로 내보내거나 안드로이드 스튜디오에서 편집한 내용을 가져올 수 있습니다.
2	Icon Image File	앱 아이콘 이미지 파일을 등록합니다. 별도로 파일을 지정하지 않으면 넥사크로 기본 이미지가 적용됩니다. 여러 이미지 파일을 압축한 후 하나의 압축파일(zip 파일)로 등록하거나 개별 파일을 등록할 수 있습니다.
3	Splash Image File	앱 실행 시 표시되는 Splash 이미지 파일을 등록합니다. 별도로 파일을 지정하지 않으면 넥사크로 기본 이미지가 적용됩니다. 여러 이미지 파일을 압축한 후 하나의 압축파일(zip 파일)로 등록하거나 개별 파일을 등록할 수 있습니다.
4	Splash Scale Type	Splash 이미지 크기를 설정합니다. 아래 4가지 옵션 중 하나를 선택할 수 있습니다. - Adjust the aspect ratio. stretch to fit the screen size : 화면 비율 조정. 화면 크기에 맞게 stretch 처리 - Maintain the aspect ratio. expand without blank area : 화면 비율 유지. 빈 영역 없이 확장 처리 - Maintain the aspect ratio. expand to fit : 화면 비율 유지. 크기에 맞게 확장 - Original size : 원본 이미지 크기
5	Splash Background color	Splash 영역 배경 색상을 지정합니다.
6	Rotation	앱에서 화면을 세로(Portrait) 또는 가로(Landscape) 로 고정시키거나 고정을 풀고 센서를 활성화(Sensor) 하는 옵션을 선택합니다.
7	Signing Detail	선택한 Signing 상세 정보를 보여줍니다.
8	User Library	User Library를 추가할 수 있습니다.
9	Device API	디바이스 API 권한을 추가할 수 있습니다.

Icon 이미지와 Splash 이미지 항목 옆에 [detail] 버튼을 클릭하면 설정한 이미지 목록을 확인할 수 있습니다.





압축파일로 등록하는 경우에는 압축파일 내 폴더와 파일명을 아래와 같이 작성해야 합니다.

ic_launcher.png: 아이콘 이미지 파일

splashimage_phone_landscape.png: 가로 방향 Splash 이미지 파일

splashimage_phone_portrait.png: 세로 방향 Splash 이미지 파일

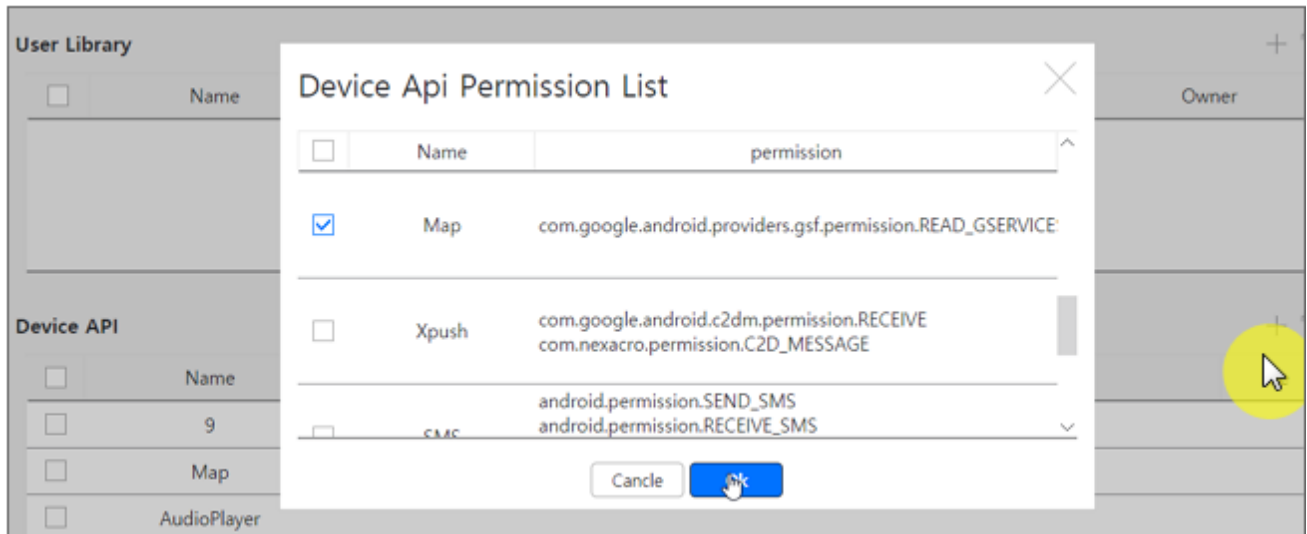


폴더명과 파일명이 틀릴 경우 정상 빌드가 되지 않거나 앱 실행 중 정상적으로 동작하지 않을 수 있습니다.

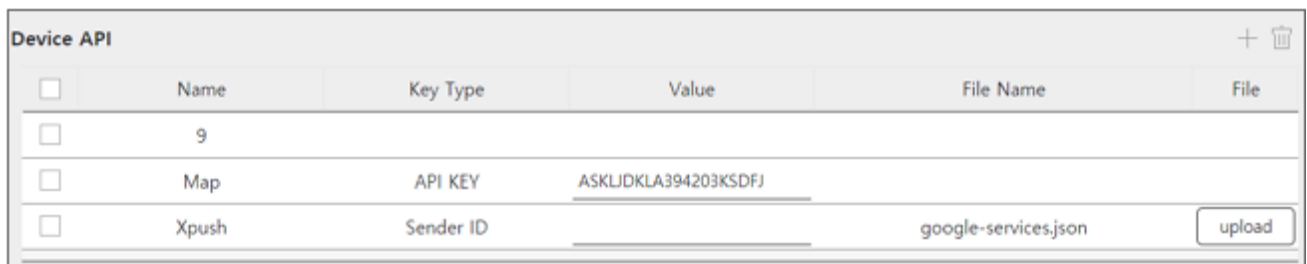
Directory	file name	Image Size(px)
drawable-hdpi	ic_launcher.png	72x72
	splashimage_phone_landscape.png	화면에 맞게
	splashimage_phone_portrait.png	
drawable-ldpi	ic_launcher.png	36x36
	splashimage_phone_landscape.png	
	splashimage_phone_portrait.png	
drawable-mdpi	ic_launcher.png	48x48
	splashimage_phone_landscape.png	
	splashimage_phone_portrait.png	
drawable-xhdpi	ic_launcher.png	96x96
	splashimage_phone_landscape.png	
	splashimage_phone_portrait.png	
drawable-xxhdpi	ic_launcher.png	144x144
	splashimage_phone_landscape.png	
	splashimage_phone_portrait.png	
drawable-xxxhdpi	ic_launcher.png	192x192
	splashimage_phone_landscape.png	
	splashimage_phone_portrait.png	

Device API 권한 추가

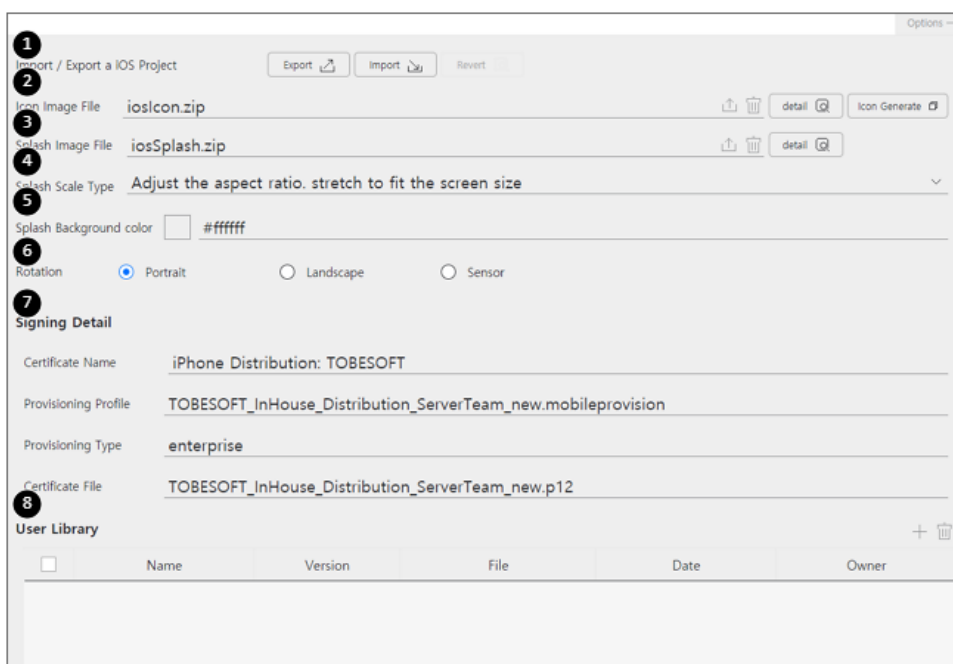
넥사크로 디바이스 API 권한을 선택합니다. [+] 버튼을 클릭하면 전체 디바이스 API 목록이 표시되며 필요한 권한을 선택한 후 [OK] 버튼을 클릭합니다.



Map, X-PUSH를 사용하는 경우에는 추가로 키값을 입력해주어야 합니다. X-PUSH 권한 지정 시에는 Firebase Android 구성 파일(google-services.json)을 첨부해주어야 합니다.



5.2.3 Target OS > iOS Options



	항목	설명
1	Import / Export	생성된 "App" 정보를 XCode에서 편집할 수 있는 프로젝트 형태로 내보내거나 XCode에서 편집한 내용을 가져올 수 있습니다.
2	Icon Image File	앱 아이콘 이미지 파일을 등록합니다. 별도로 파일을 지정하지 않으면 넥사크로 기본 이미지가 적용됩니다. 여러 이미지 파일을 압축한 후 하나의 압축파일(zip 파일)로 등록하거나 개별 파일을 등록할 수 있습니다.
3	Splash Image File	앱 실행 시 표시되는 Splash 이미지 파일을 등록합니다. 별도로 파일을 지정하지 않으면 넥사크로 기본 이미지가 적용됩니다. 여러 이미지 파일을 압축한 후 하나의 압축파일(zip 파일)로 등록하거나 개별 파일을 등록할 수 있습니다.
4	Splash Scale Type	Splash 이미지 크기를 설정합니다. 아래 4가지 옵션 중 하나를 선택할 수 있습니다. - Adjust the aspect ratio. stretch to fit the screen size : 화면 비율 조정. 화면 크기에 맞게 stretch 처리 - Maintain the aspect ratio. expand without blank area : 화면 비율 유지. 빈 영역 없이 확장 처리 - Maintain the aspect ratio. expand to fit : 화면 비율 유지. 크기에 맞게 확장 - Original size : 원본 이미지 크기
5	Splash Background color	Splash 영역 배경 색상을 지정합니다.
6	Rotation	앱에서 화면을 세로(Portrait) 또는 가로(Landscape) 로 고정시키거나 고정을 풀고 센서를 활성화(Sensor) 하는 옵션을 선택합니다.
7	Signing Detail	선택한 Signing 상세 정보를 보여줍니다.
8	User Library	User Library를 추가할 수 있습니다.

Icon 이미지와 Splash 이미지 항목 옆에 [detail] 버튼을 클릭하면 설정한 이미지 목록을 확인할 수 있습니다.

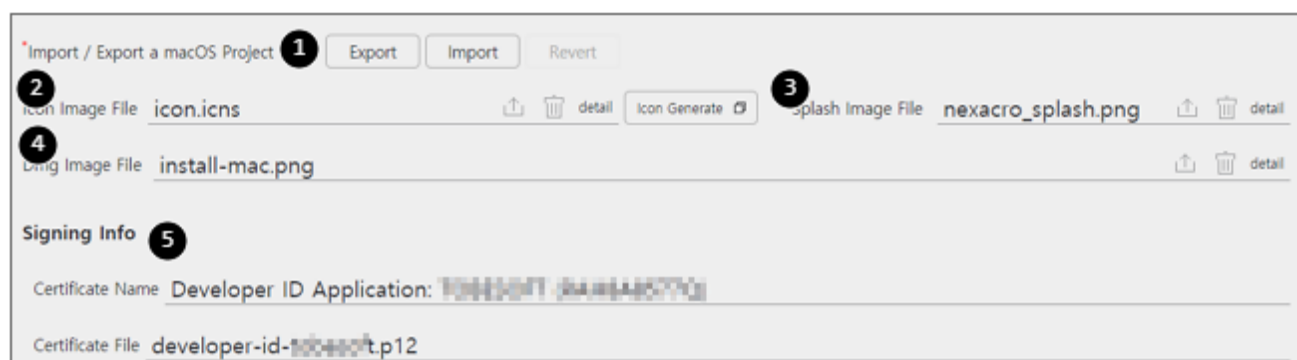


아이콘, Splash 파일명은 아래와 같이 작성해야 합니다.

file name	Image Size(px)	설명
Icon.png	57x57	Home screen on iPhone/iPod touch (iOS 6.1 and earlier)
Icon@2x.png	114x114	Home screen on iPhone/iPod Touch with retina display (iOS 6.1 and earlier)
[크기].png		57X57 (57.png) 76X76 (76.png) 120x120 (120.png) 152X152 (152.png) 167X167 (167.png)
splashimage_pad_landscape.png	1024x768	Pad 가로 방향 Splash 이미지 파일
splashimage_pad_portrait.png	768x1024	Pad 세로 방향 Splash 이미지 파일

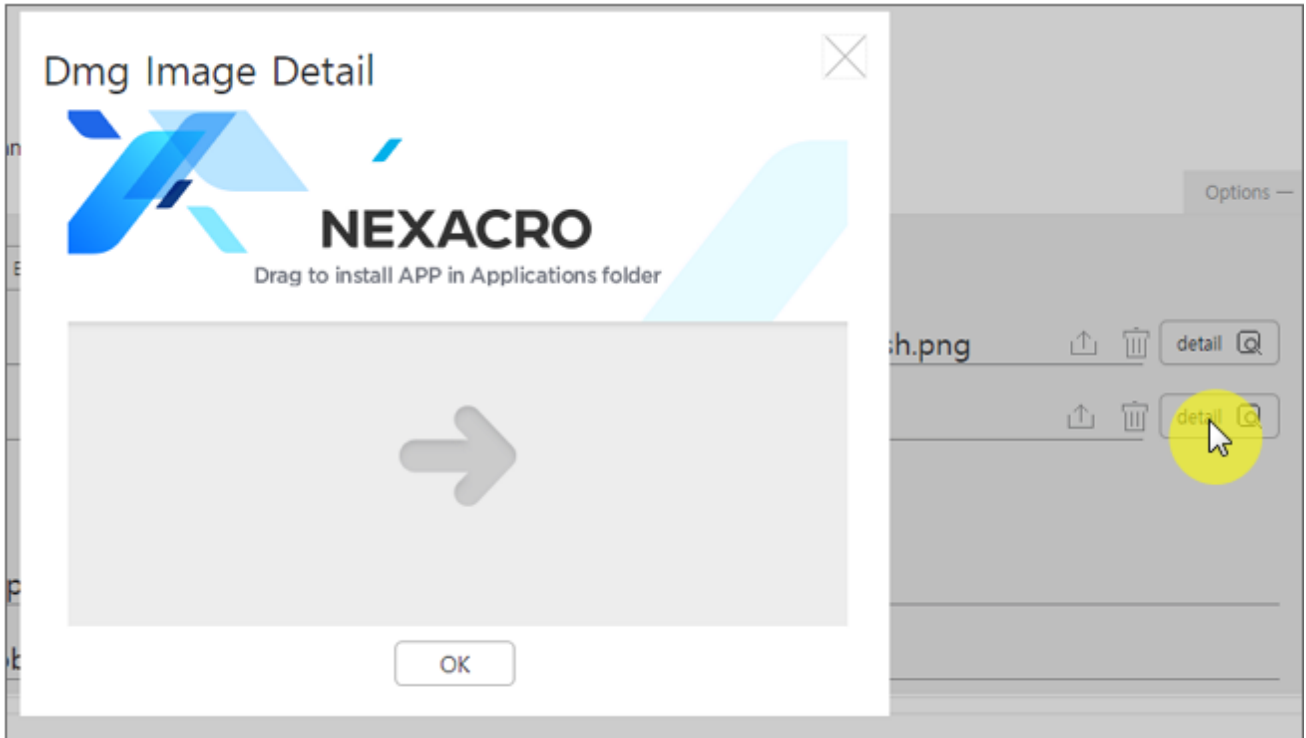
file name	Image Size(px)	설명
g		
splashimage_phone_landscape.png	960x640	Phone 가로 방향 Splash 이미지 파일
splashimage_phone_portrait.png	640x960	Phone 세로 방향 Splash 이미지 파일
splashimgae_phone5_landscape.png	1136x640	Phone 가로 방향 Splash 이미지 파일
splashimgae_phone5_portrait.png	640x1136	Phone 세로 방향 Splash 이미지 파일

5.2.4 Target OS > macOS Options



	항목	설명
1	Import / Export	생성된 "App" 정보를 XCode에서 편집할 수 있는 프로젝트 형태로 내보내거나 XCode에서 편집한 내용을 가져올 수 있습니다.
2	Icon Image File	앱 아이콘 이미지 파일을 등록합니다. 별도로 파일을 지정하지 않으면 넥사크로 기본 이미지가 적용됩니다. 파일 확장자는 .icns 입니다.
3	Splash Image File	앱 실행 시 표시되는 Splash 이미지 파일을 등록합니다. 별도로 파일을 지정하지 않으면 넥사크로 기본 이미지가 적용됩니다.
4	Dmg Image File	앱 설치 시 표시되는 이미지 파일을 등록합니다. 별도로 파일을 지정하지 않으면 넥사크로 기본 이미지가 적용됩니다.
5	Signing Detail	선택한 Signing 상세 정보를 보여줍니다.

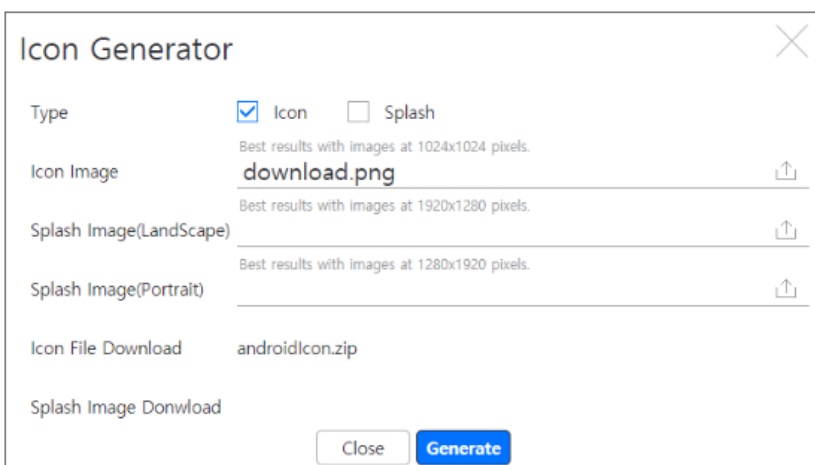
Icon, Splash, Dmg 이미지 항목 옆에 [detail] 버튼을 클릭하면 설정한 이미지 목록을 확인할 수 있습니다.



Dmg Image File에 등록된 이미지 파일의 DPI가 72X72가 아닌 경우 Mac OS 10.7 이상에서 배경이 왜곡되어 보여질 수 있습니다.

5.2.5 Target OS > Icon Generate를 사용해 이미지 파일 생성하기

이미지 파일 설정 화면 옆에 [Icon Generate] 버튼을 클릭하면 Icon Generator 화면이 실행됩니다. 생성할 이미지 Type을 선택하고 이미지 파일을 업로드한 후 [Generate] 버튼을 클릭하면 선택한 Target OS에 따라 압축파일을 생성합니다. 생성한 파일은 내려받아 업로드해서 이미지를 설정할 수 있습니다.





업로드하는 Image 파일은 png 확장자만 지원합니다.

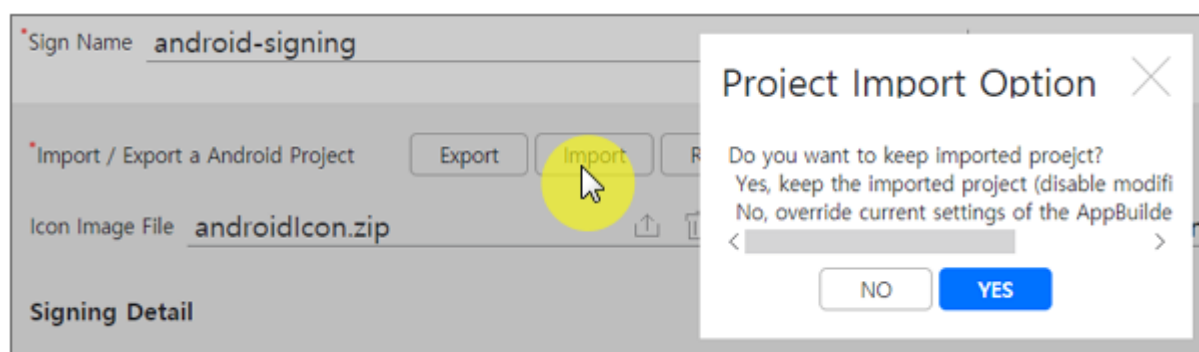
5.2.6 Target OS > 프로젝트 파일 내보내고 가져오기

App Builder에서 지원하지 못하는 기능을 필요로 하는 경우 생성된 "App" 정보를 프로젝트 형태로 내보내 각 운영 체제를 지원하는 개발도구에서 필요한 기능을 추가하거나 편집할 수 있습니다. [Project Modification] 기능을 사용하면 현재 "App" 정보를 압축 파일 형태로 내보내거나 외부 개발도구에서 수정한 프로젝트를 가져올 수 있습니다.



	항목	설명
1	Export	현재 "App" 프로젝트를 압축파일로 내보냅니다.
2	Import	현재 "App" 프로젝트에 사용자가 수정한 프로젝트를 가져옵니다.
3	Revert	App Builder에 저장된 데이터로 원복합니다.

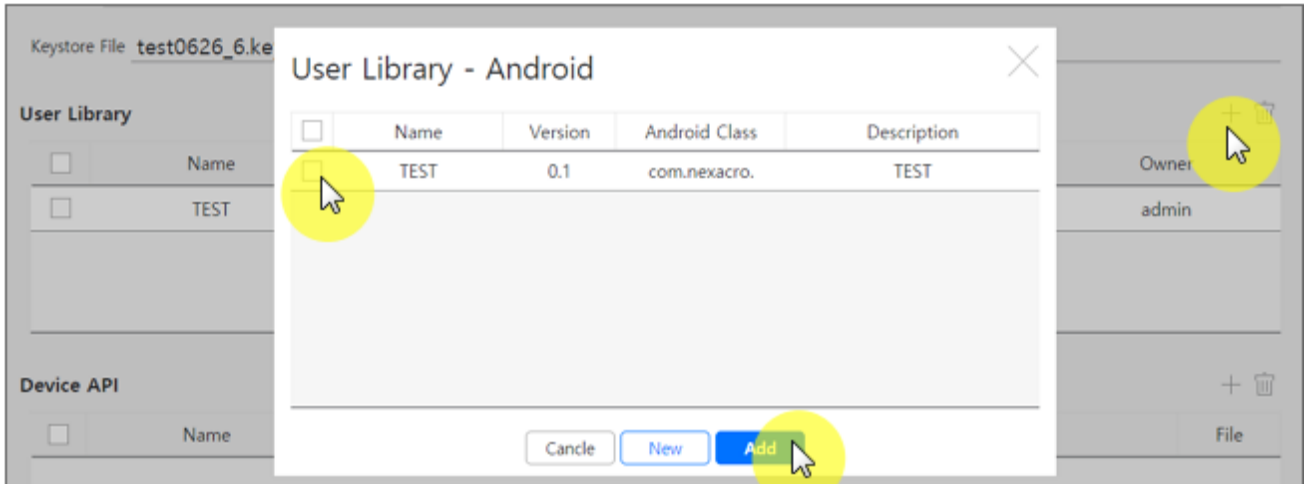
[Import] 버튼 클릭 시에는 두 가지 옵션 중 하나를 선택할 수 있습니다. 버튼 클릭 시 아래 그림처럼 옵션을 선택할 수 있는 창이 표시되며 [YES], [NO] 버튼 선택에 따라 수정한 프로젝트를 가져오는 방식을 결정합니다.



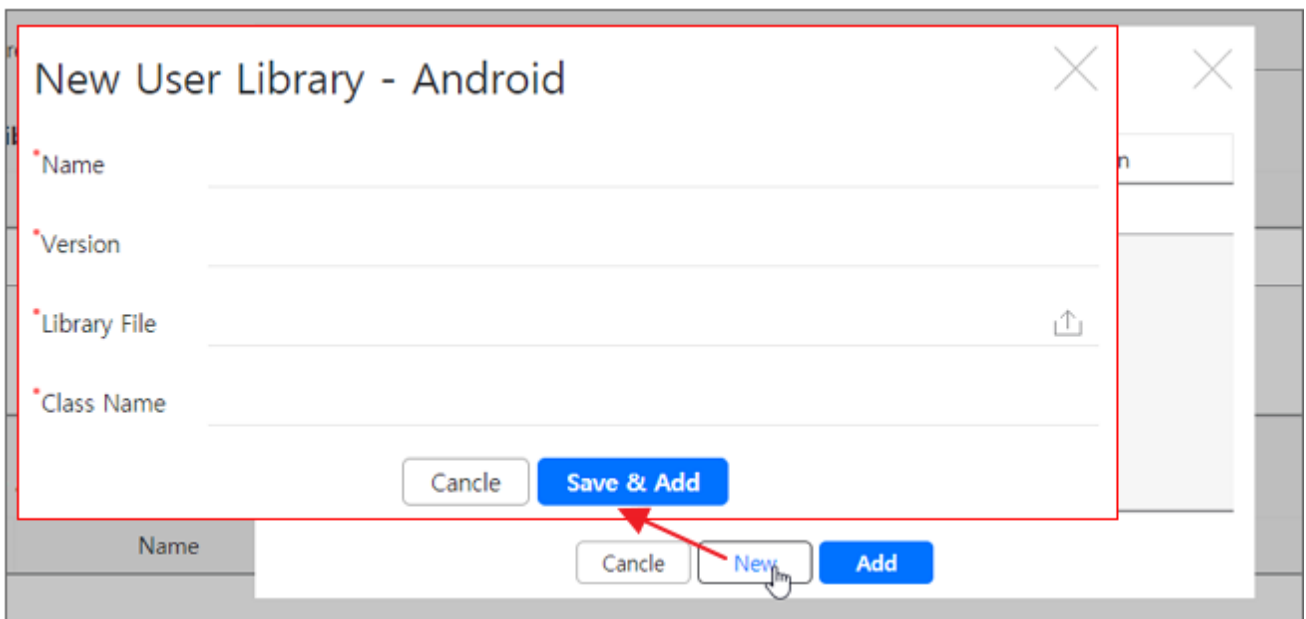
	항목	설명
1	YES	사용자가 수정한 후 Import 한 프로젝트의 설정을 유지합니다. - 프로젝트의 각 플랫폼 별 설정화면의 상태를 비활성화 합니다. - 공통 설정 항목은 수정 가능하나 해당 플랫폼은 적용하지 않습니다.
2	NO	사용자가 수정한 후 Import 한 프로젝트의 설정을 유지 하지 않습니다. - 프로젝트의 각 플랫폼 별 설정 화면의 상태를 활성화 합니다. - Import 된 프로젝트에 AppBuilder 콘솔화면에 명시된 설정을 덮어쓰고 빌드합니다.

5.2.7 Target OS > User Library

Android, iOS 운영체제에서 사용할 수 있는 User Library를 선택하거나 새로 등록할 수 있습니다. Options 화면에서 User Library 항목 옆에 있는 [+] 버튼을 클릭하면 목록을 선택할 수 있는 창이 표시되고 원하는 라이브러리 항목을 체크한 후 [Add] 버튼을 클릭합니다.



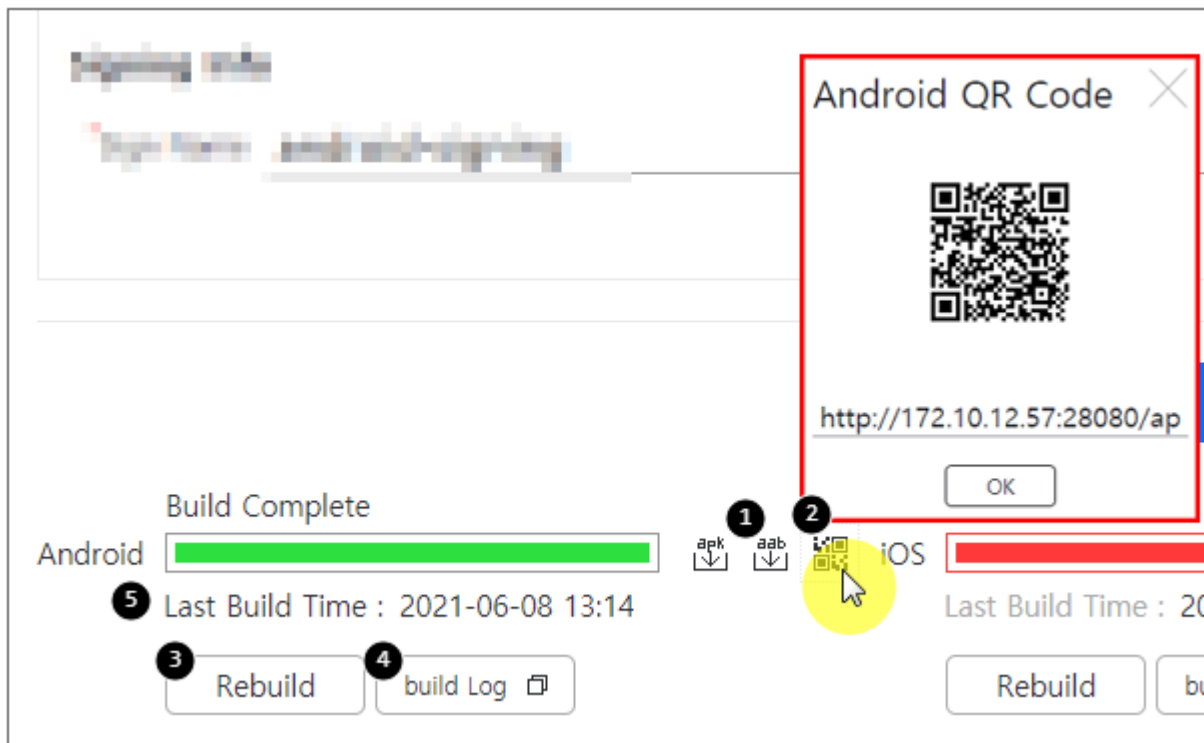
새로 라이브러리를 추가해야 한다면 [New] 버튼을 클릭해서 새로운 항목을 등록할 수 있습니다.



5.3 앱 빌드하기

새로 "App"을 생성한 경우에는 [Build]라는 버튼이 표시됩니다. 버튼을 클릭하면 선택한 운영체제에 해당하는 앱 빌드를 시작합니다.

빌드 결과를 표시하며 빌드를 성공한 경우에는 생성된 앱을 내려받을 수 있는 링크를 생성합니다. [Rebuild All] 버튼을 클릭해 전체 앱을 다시 빌드하거나 운영체제마다 따로 빌드할 수 있는 기능도 지원합니다.



	항목	설명
1	다운로드	생성된 앱 설치 파일을 내려받습니다.
2	QRCode	생성된 앱 설치 파일을 내려받을 수 있는 URL, QRCode를 표시합니다.
3	rebuild	선택된 운영체제 앱을 다시 빌드합니다.
4	build Log	앱 빌드 시 생성된 로그를 확인할 수 있습니다.
5	Last Build Time	마지막으로 앱을 빌드한 시간을 표시합니다.



설치 파일명은 아래와 같이 생성됩니다. "App Title", "Build mode" 정보는 [App Info] 항목에서 설정합니다.

Android: [App Title]-[Build mode].apk, [App Title]-[Build mode].aab

iOS, macOS: [App Title].ipa

예) sample-debug.apk, sample.ipa

모바일 디바이스에는 "App Title" 정보에 지정한 이름으로 설치됩니다.



QRCode 기능을 제공하는 앱을 사용하는 경우 QRCode를 인식한 후 URL을 Chrome 브라우저에서 처리하는 과정에서 "downloadFile.do" 또는 "downloadFile.htm"으로 주소창에 표시되고 파일을 내려받지 못하는 경우가 있습니다. 이런 경우에는 아래와 같이 조치합니다.

- Chrome 브라우저 캐시 삭제
- Chrome 브라우저를 최신 버전으로 업데이트



아래와 같은 경우는 QR코드를 지원하지 않습니다.

- macOS
- iOS Provisioning profile type이 "app-store"인 경우
- Android App Bundle(.aab) 파일

6.

App 목록

App Builder 관련 문서

- 서버 설정/개발 가이드 > App Builder

- App Builder 설치

- 서버 환경 설정

- 디플로이 서버 설정

- 운영체제별 서명 설정

- 사용자 라이브러리 설정

- 넥사크로 라이브러리 설정

- 사용자 설정

- 앱 배포 가이드 > App Builder

- App 생성

- App 목록

- 개발도구 가이드 > Build App 단계에서 설치 파일 만들기

- Build App 단계에서 설치 파일 만들기

생성된 "App" 목록을 확인할 수 있습니다. 관리 콘솔에서 메뉴[BuildApp > App List] 항목을 선택합니다. [App List]에서는 로그인한 사용자가 권한을 가지고 있는 "App" 목록만 표시됩니다.

App List

App Name

NewCopy

☐	App Name	Package Name	Version	Device Type	Build	Download	LastBuildTime	Owner	Deploy
☐	TEST	com.nexacro	1.0.0	Android	☐		2021-06-08 13:14:39		Deploy
				iOS	☒		2021-06-08 13:13:56		
				macOS					

항목별로 빌드 상태와 빌드에 성공한 경우 설치 파일을 내려받을 수 있습니다. 항목을 선택한 후 오른쪽 상단 [Copy] 버튼을 클릭하면 "[App_Name]_copy"라는 이름으로 복사된 "App"을 생성합니다.

파트 V.

iOS(iPadOS)/Android/macOS App

7.

앱 개발 및 실행 (iOS/iPadOS - Xcode)

7.1 앱 개발 환경 설정

iOS용 앱은 Xcode, iOS SDK를 이용해 개발합니다. Xcode, iOS SDK는 Windows 계열의 PC에서는 사용할 수 없으며 Mac에서만 구동합니다. 그리고 Xcode를 내려받고 앱을 만들고 테스트를 하기 위해서는 Apple 개발자 계정이 필요합니다.

iOS 앱 개발 환경은 기본적으로 다음과 같은 절차에 의해 구성되며 세부 내용은 개발자 사이트 업데이트에 따라 변경될 수 있습니다.

1. 개발자 계정 등록
2. Xcode, iOS SDK 설치
3. 배포 시 필요한 작업



이번 장에서는 Xcode 15.0 버전을 기준으로 작성했습니다.

7.2 앱 프로젝트 개발

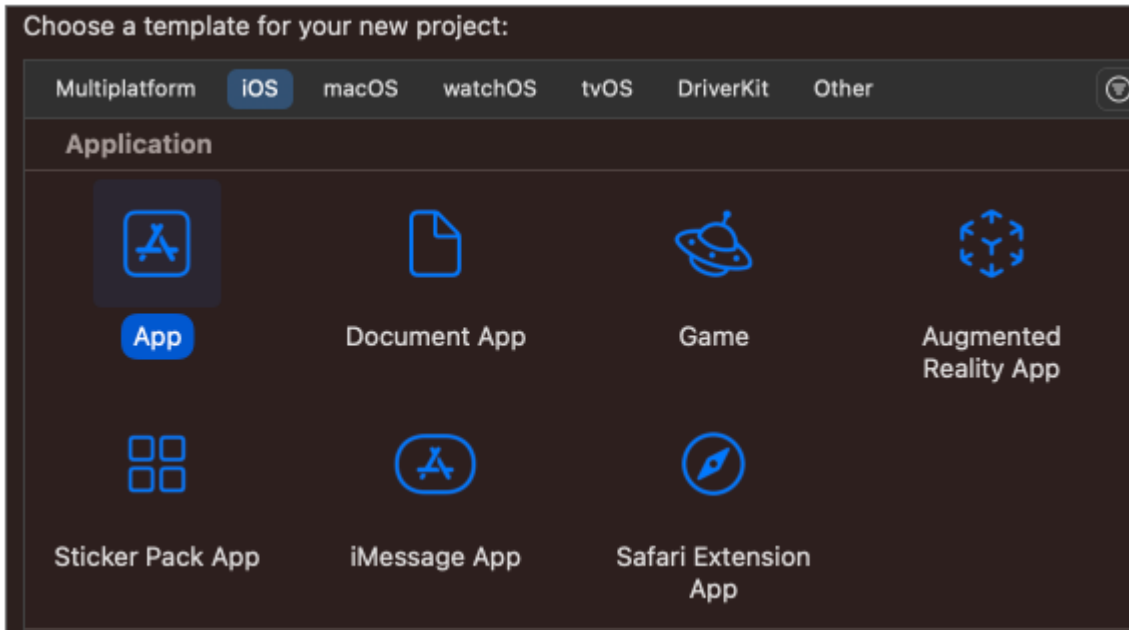
Xcode에서 아래와 같은 절차에 따라 iOS용 앱 프로젝트를 개발합니다. 앱 프로젝트를 진행하기 전에 넥사크로 스튜디오에서 만든 아카이브 파일은 지정된 경로에 위치해야 합니다.

7.2.1 프로젝트 생성

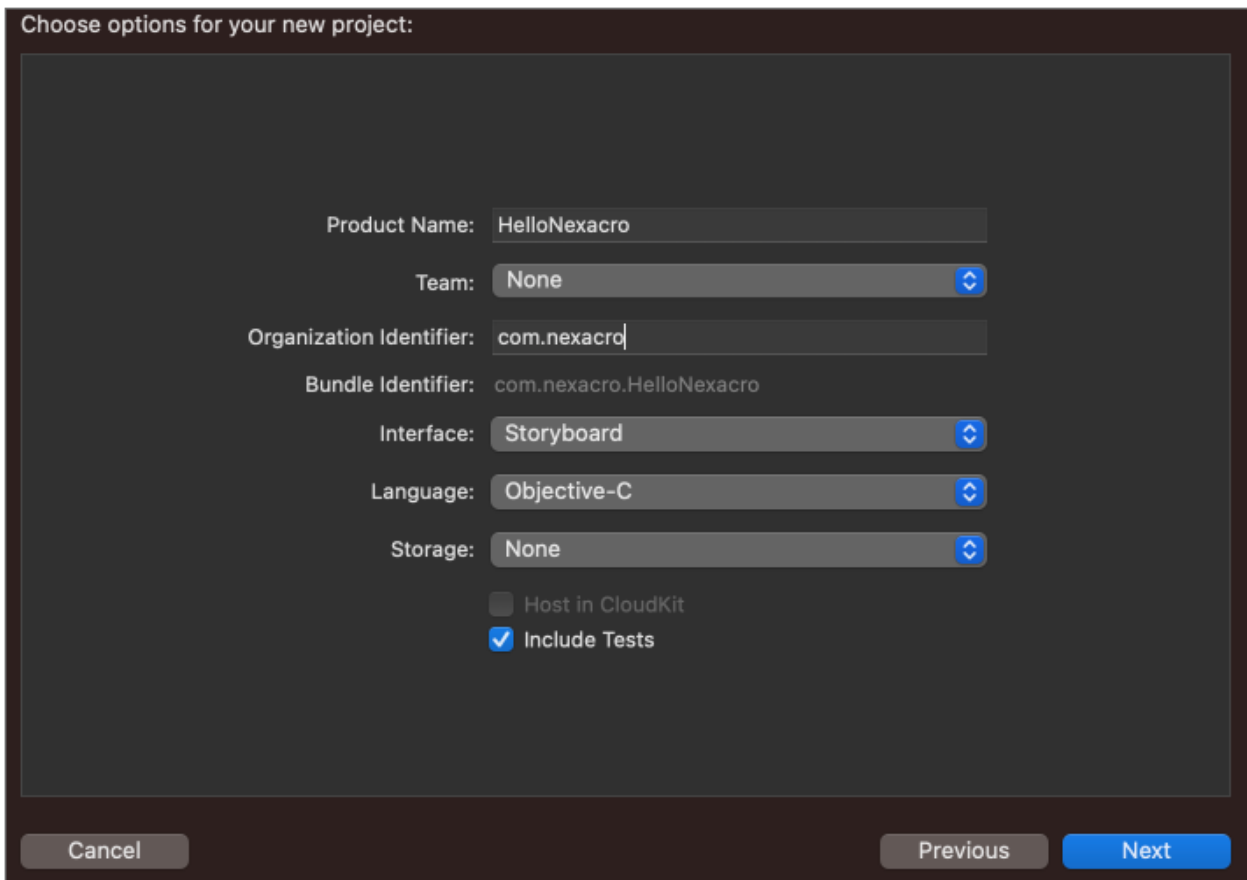
넥사크로로 개발된 앱을 담은 iOS 프로젝트를 생성하고 기본 환경을 설정해야 합니다. 새로운 프로젝트는 아래 메뉴에서 생성할 수 있습니다.

File > New > Project

프로젝트 생성을 위한 템플릿 화면에서 'App' 항목을 선택합니다.



Product Name과 필요한 항목을 지정한 후 Next 버튼을 클릭합니다. 열린 팝업 창에서 프로젝트를 생성할 위치를 지정한 후 'Create' 버튼을 클릭하면 새로운 프로젝트가 생성됩니다.





설치된 Xcode 버전에 따라 프로젝트 생성 방법이 조금씩 달라질 수 있습니다.

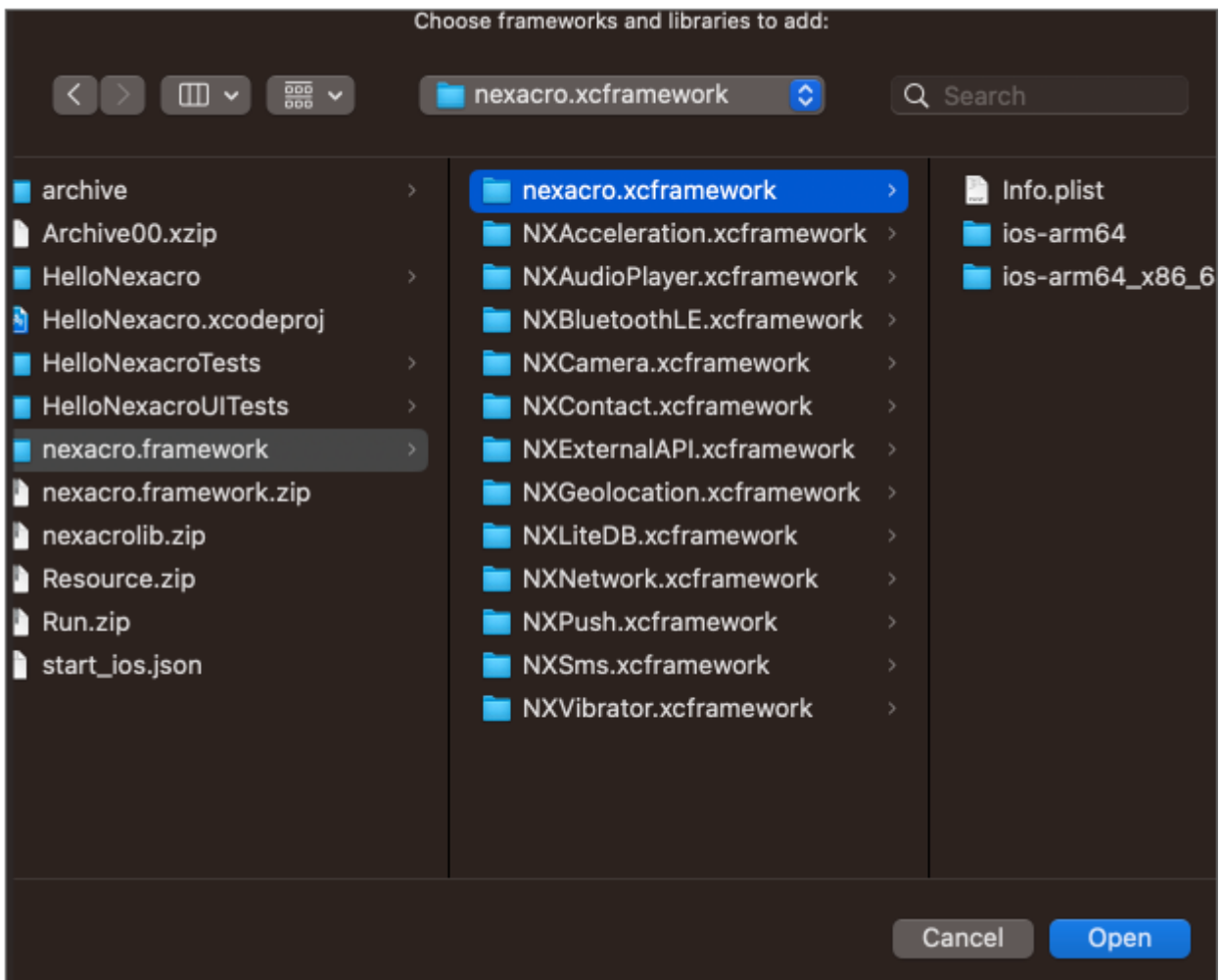


iPhone X 디바이스를 사용하는 경우에 위쪽과 아래쪽에 여백이 생기는 현상을 방지하기 위해서는 프로젝트에 LaunchScreen 스토리 보드를 추가해야 합니다.

7.2.2 넥사크로 라이브러리 설정

iOS 프로젝트에서 넥사크로에 최적화된 환경을 만들기 위해 추가로 제공되는 넥사크로 라이브러리 파일을 설정합니다.

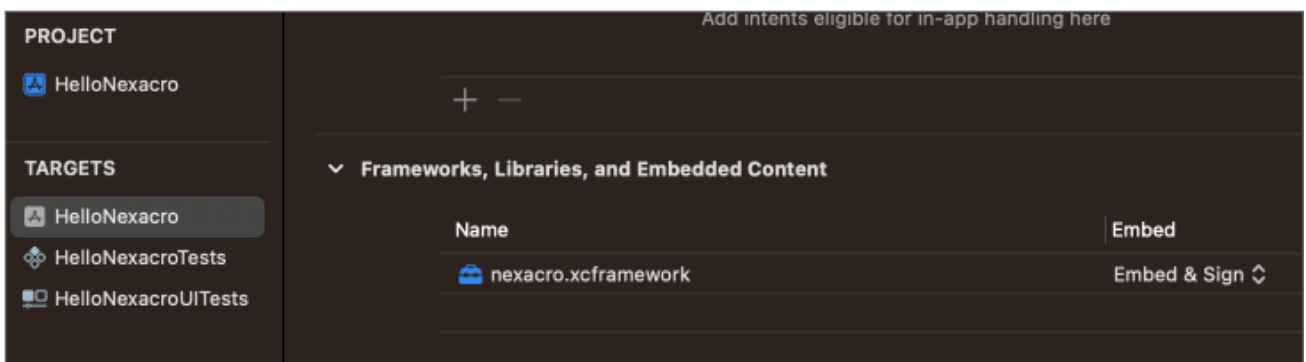
넥사크로 라이브러리는 압축 파일 형태로 제공되며 nexacro.framework.zip이라는 파일명으로 제공됩니다. 먼저 파일 압축을 해제해줍니다. 그리고 'Frameworks, Libraries, and Embedded Content' 항목에서 [+] 버튼을 클릭하고 Add Files 항목을 선택해서 nexacro.framework 폴더 아래에 있는 파일을 선택해 추가합니다.



기본 라이브러리인 nexacro.xcframework는 필수 설정 대상이며 나머지 항목은 앱에서 사용하는 Device API에 따

라 필요한 항목만 추가합니다.

라이브러리	필수 여부
nexacro.xcframework	필수
NXAcceleration.xcframework	앱에서 사용하는 Device API에 따라 선택적으로 추가
NXAudioPlayer.xcframework	
NXBluetoothLE.xcframework	
NXCamera.xcframework	
NXContact.xcframework	
NXExternalAPI.xcframework	
NXGeolocation.xcframework	
NXLiteDB.xcframework	
NXNetwork.xcframework	
NXPush.xcframework	
NXSms.xcframework	
NXVibrator.xcframework	



7.2.3 리소스 설정

앱에서 사용할 로딩 이미지, 아이콘, 메시지, 레이아웃 등을 설정하는 단계입니다. 진행하는 프로젝트에 따라 변경해 적용할 수 있습니다.

이미지 설정

Xcode의 템플릿으로 생성한 프로젝트에는 리소스를 관리하는데 필요한 빈 애셋 카탈로그(Asset Catalog)가 포함됩니다. 사용자는 필요에 따라 앱 아이콘, 런처 이미지 등을 애셋 카탈로그에 등록하여 사용할 수 있습니다.

iOS 앱에서 사용하는 앱 아이콘과 런처 스크린 이미지는 개별적으로 설정할 수 있으며 미리 만들어진 이미지를 연결할 수 있습니다.



이미지 설정시 반드시 해당 iOS 단말기에 맞게 제작된 이미지를 사용해야 합니다. 단말기에 맞지 않는 크기의 이미지 혹은 사양과 맞지 않는 속성을 갖는 이미지를 설정하면 앱 빌드시 에러가 발생하거나 정상적으로 동작하지 않습니다.

아이콘 및 이미지 설정과 관련된 자세한 내용은 애플 개발자 사이트를 참고하세요.

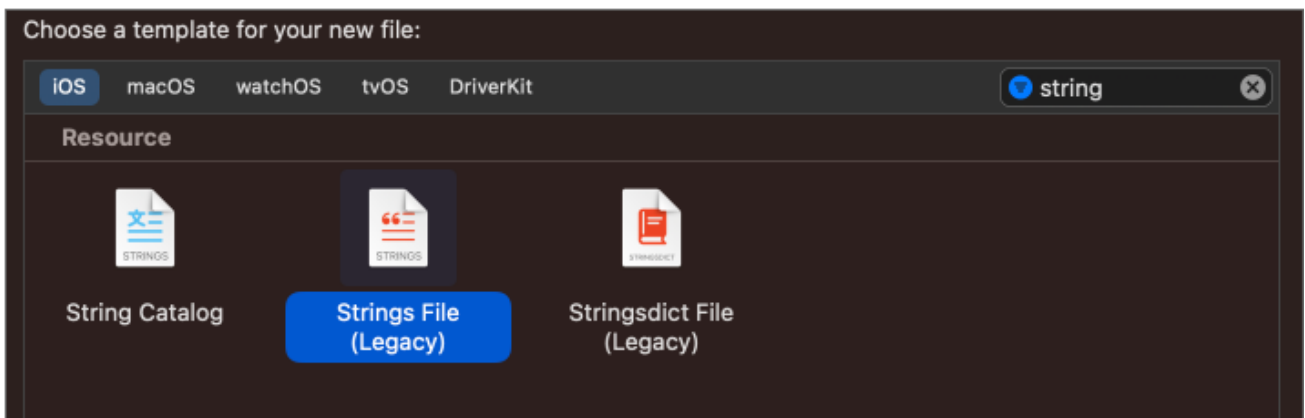
<https://developer.apple.com/documentation/xcode/configuring-your-app-icon>

메시지 설정

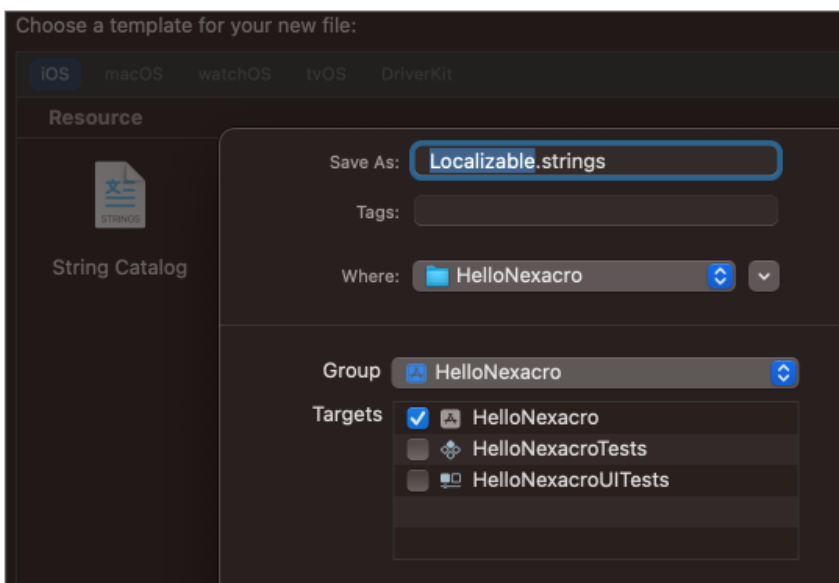
앱에서 사용하는 기본 메시지는 넥사크로 라이브러리에 포함되어 있습니다. 기본 메시지를 재정의하는 경우에 메시지를 별도 설정할 수 있습니다.

새로운 파일을 생성하려면 프로젝트 폴더를 선택 후 메뉴에서 [File > New > File] 항목을 선택합니다.

화면에 보이는 템플릿 중에서 'Strings File'을 선택합니다.



"Save As" 항목에 파일명이 'Localizable.strings'인지 확인합니다.

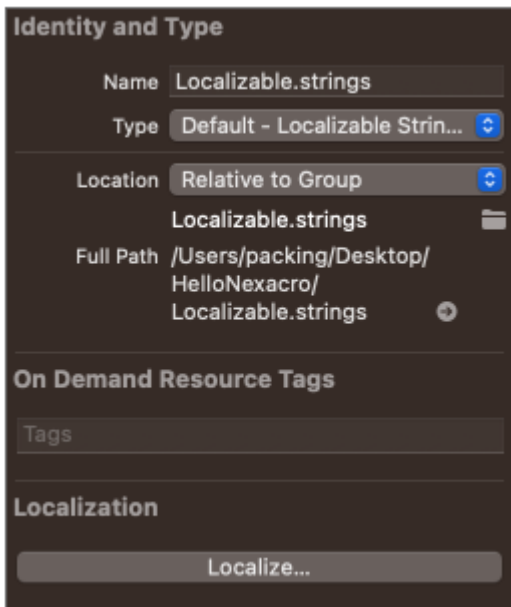


생성된 Localizable.strings 파일은 아래와 같이 수정합니다.

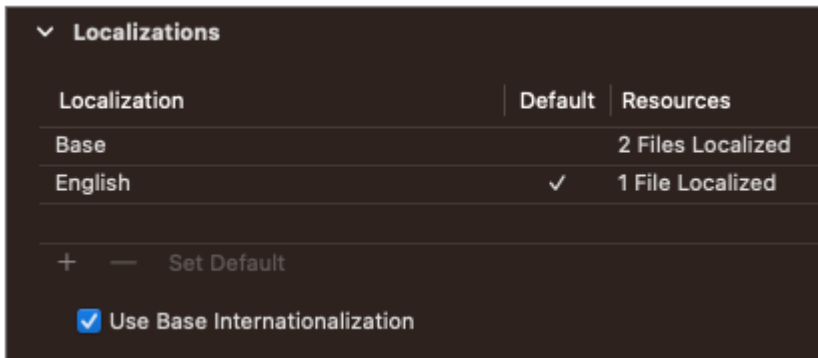
```
/*
Localizable.strings english
nexacro
*/

"needupdate" = "It is need to update. \r\nIt will start after the update is completed.";
"loadingFail" = "First loading is fail. \r\nPlease restart.";
"updateFail" = "Update is fail. \r\nPlease restart.";
"notexist" = "Start file is NOT exist. \r\nPlease restart.";
"BeingUpdated" = "Being updated.";
"ok" = "OK";
"cancel" = "Cancel";
"alert_title" = "";
"checkforupdates" = "Check for updates.";
"installforupdates" = "Install for updates.";
"downloadingforupdates" = "Downloading for updates.";
"new_engine_available" = "Engine update required.";
```

Xcode 화면 오른쪽에 있는 Utilities 창에서 'Localize...' 버튼을 클릭하고 언어를 선택합니다. 사전에 프로젝트 설정에서 언어를 추가하지 않은 경우에는 기본으로 영어가 선택됩니다.



영어 이외의 언어를 지원하려면 [프로젝트 > Info > Localizations]에서 + 버튼을 눌러 다른 언어를 추가할 수 있습니다.



Localization에 관련된 자세한 내용은 애플 개발자 사이트를 참고하세요.
<https://developer.apple.com/documentation/xcode/localization>

7.2.4 Config 설정

넥사크로에서 제공하는 기능을 앱에서 사용할 수 있도록 nexacro_config.xml 파일을 설정합니다. 앱 업데이트, Notification, 에러 정보 처리 등의 기능을 활성화 할 수 있습니다. nexacro_config.xml 파일은 사용자가 직접 생성해야 하며 [{Asset} > data] 폴더 아래에 배치합니다. 이 파일은 Xcode에서 앱 빌드시 프로젝트에 포함되어 있어야 합니다.



nexacro_config.xml 파일이 없으면 기본 설정이 적용되므로 앱 실행에는 아무런 지장이 없습니다.

nexacro_config.xml 파일은 XML 형식이며 다음 예와 같이 작성합니다.

```
<?xml version="1.0" encoding="utf-8"?>
<nexacro-config>
  <application dialog-position="top" use-wkwebview="true" file-logging="true" />
  <notification enable="true" handler="DefaultNotificationManagerHandler"/>
  <updater force="false" cancelable="false" quiet="true" errmsg="true" failpass="true"/>
  <xpush-server request-missing-message="true" />
</nexacro-config>
```

nexacro_config.xml에 설정할 수 있는 기능은 다음과 같습니다.

기능	속성	설정 값	설명
application	dialog-position	"top" "center" "bottom"	앱 업데이트 진행 정보를 표시할 팝업 위치를 설정합니다.
	file-logging	"true" "false"	로딩 에러 정보를 파일로

기능	속성	설정 값	설명
			저장할지 설정합니다. iTunes를 통해서 로그 파일을 확인할 수 있습니다.
updater	force	"true" "false"	start_ios.json에 업데이트 파일 정보 존재시 팝업으로 표시할지 설정합니다. "true"로 설정시 팝업 알림없이 강제로 업데이트를 진행합니다.
	cancelable	"true" "false"	업데이트 파일 존재시 팝업에 업데이트 취소 버튼을 표시할지 설정합니다. 취소 버튼을 클릭하면 업데이트를 진행하지 않고 앱을 종료합니다.
	errormsg	"true" "false"	앱 로딩에 실패했을 때 에러 정보를 팝업으로 표시할지 설정합니다.
	quiet	"true" "false"	업데이트 팝업을 표시할지 설정합니다. "true"로 설정해도 업데이트 파일 존재시 진행 단계는 표시됩니다.
	failpass	"true" "false"	업데이트 오류를 무시할지 설정합니다.
xpush-server	request-missing-message	"true" "false"	xpush 서버 사용시 수신하지 못한 신뢰성 메시지를 자동으로 요청할지 설정합니다.
notification	enable	"true" "false"	알림 기능을 사용할지 설정합니다.
	handler	"[클래스 이름]"	알림 수신시 처리할 핸들러를 구현한 클래스를 설정합니다. 기본 값은 "DefaultNotificationManagerHandler"입니다.

7.2.5 빌드 환경 설정

iOS 프로젝트를 생성하면서 기본으로 만들어진 AppDelegate.h, AppDelegate.m 파일과 main.m 파일을 넥사크로 환경에 맞게 수정합니다.

AppDelegate.h

생성된 앱 델리게이트가 넥사크로 AppDelegate를 상속받도록 수정합니다. 파일을 다음과 같이 수정하여 nexacro.framework가 동작할 수 있도록 합니다.

```
//
// AppDelegate.h
// HelloNexacro
//

#import <UIKit/UIKit.h>
#import <nexacro/NXAppDelegate.h>

@interface AppDelegate : NXAppDelegate

@end
```

AppDelegate.m

AppDelegate.m 파일에서는 넥사크로 스튜디오에서 만든 start_ios.json 파일이 배치된 서버 URL 주소를 지정합니다. 아래 코드에서 굵게 표시된 부분을 사용자의 환경에 맞게 수정합니다.

```
//
// AppDelegate.m
// HelloNexacro
//

#import "AppDelegate.h"
#import "ViewController.h"

@interface AppDelegate ()

@end
```

```

@implementation AppDelegate

- (BOOL)application:(UIApplication *)application
    didFinishLaunchingWithOptions:(NSDictionary *)launchOptions {
    self.viewController = [[ViewController alloc] init];

    // Update Type: Server, Update
    self.viewController.projectUrl = @"http://[URL]/_ios/";
    self.viewController.bootstrapUrl = @"http://[URL]/_ios/start_ios.json";

    // Update Type: Local
    // self.viewController.bootstrapUrl = @"file:///ios_asset/archive/start_ios.json";

    return [super application:application didFinishLaunchingWithOptions:launchOptions];
}

@end

```

main.m

UIApplicationMain 함수는 UIKit 프레임워크에서 제공하는 기본 함수로 앱의 시작점입니다. 앱의 구동에 필요한 사전 준비를 하고 메인 루프를 시작해 사용자의 입력을 받을 준비를 합니다.

main.m 파일을 아래와 같은지 확인하고 다르다면 수정합니다.

```

//
//  main.m
//  HelloNexacro
//

#import <UIKit/UIKit.h>
#import "AppDelegate.h"

int main(int argc, char * argv[]) {
    NSString * appDelegateClassName;
    @autoreleasepool {
        // Setup code that might create autoreleased objects goes here.
        appDelegateClassName = NSStringFromClass([AppDelegate class]);
    }
    return UIApplicationMain(argc, argv, nil, appDelegateClassName);
}

```

}

ViewController.h

```
//
// ViewController.h
// HelloNexacro
//

#import <UIKit/UIKit.h>
#import <nexacro/NXViewController.h>

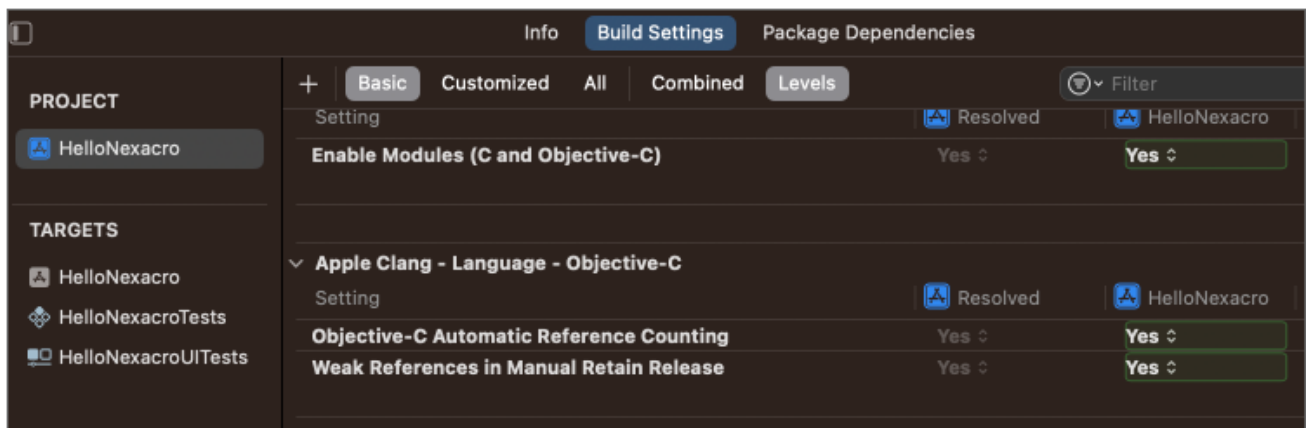
@interface ViewController : NXViewController

@end
```

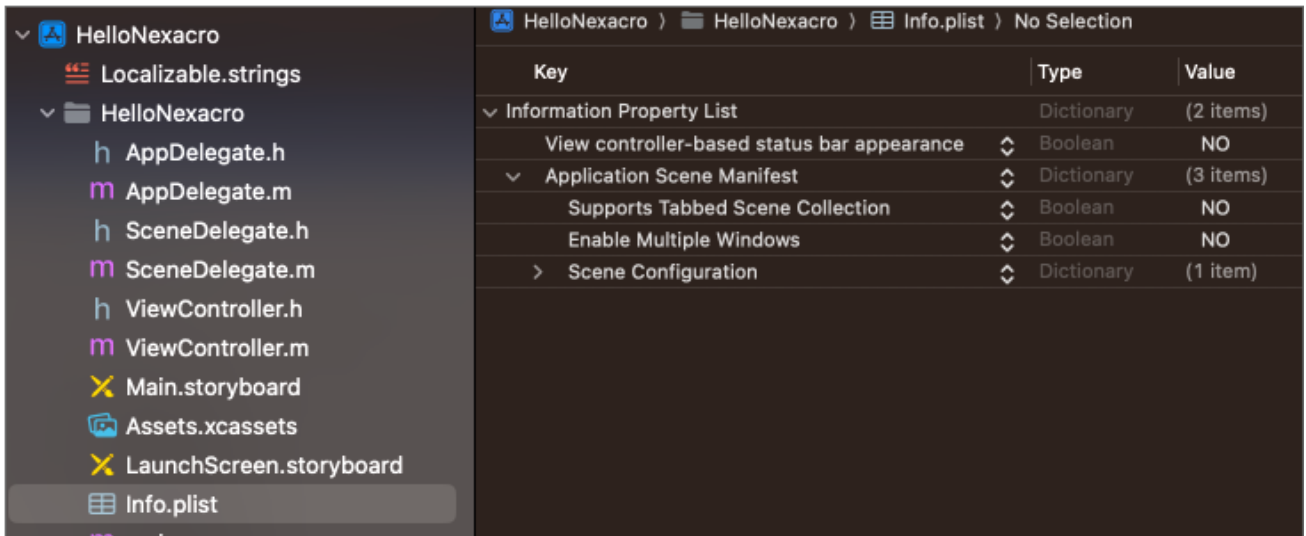
기타 설정

- 'Build Settings' 항목에서 설정값을 확인하고 필요한 경우 수정합니다.

Build Settings	항목	값
Architectures > Build Active Architecture Only	Debug	No
	Release	No
Apple Clang - Language - Objective-C	Objective-C Automatic Reference Counting	Yes



- Info.plist 파일을 선택하고 Information Property List에서 'View controller-based status bar appearance' 항목을 추가하고 값을 'NO'로 설정합니다.



- projectUrl, bootstrapUrl 경로를 HTTP로 설정하는 경우에는 Info.plist 파일을 선택하고 'App Transport Security Settings' 항목을 추가하고 그 아래에 'Allow Arbitrary Loads' 항목을 추가한 후 값을 'YES'로 설정합니다.

Key	Type	Value
Information Property List	Dictionary	(3 items)
App Transport Security Settings	Dictionary	(1 item)
Allow Arbitrary Loads	Boolean	YES
View controller-based status bar appearance...	Boolean	NO
Application Scene Manifest	Dictionary	(2 items)

7.2.6 네트워크 연결 없는 앱 실행 설정

넥사크로 스튜디오에서 앱을 패키징할 때 Update Type을 Local이나 Update(Local+Server)으로 설정하면 네트워크 연결 없이도 앱을 실행할 수 있습니다. 일반적으로 네트워크 연결이 필요 없는 경우에는 Local 타입을 사용하고, 네트워크가 연결된 상태에서 앱 실행 후 로컬 파일로 앱을 동작해야 할 경우에는 Update(Local+Server) 타입을 사용할 수 있습니다. 이처럼 네트워크 연결이 필요 없거나 문제 발생 시 앱 단독으로 실행이 가능하게 하려면 앱 빌드 시 실행에 필요한 파일을 적재하여 배포해야 합니다.

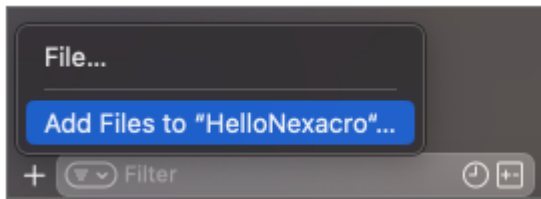
앱 실행에 필요한 파일은 모듈, 소스, 리소스 그리고 Run.zip과 start_ios.json으로 넥사크로 스튜디오에서 Packing(Archive&Update)을 수행하여 얻을 수 있습니다. 이 파일을 Xcode 프로젝트에서 archive 폴더에 적재한 후 앱을 빌드하면 네트워크 연결 없이도 앱을 실행할 수 있습니다.

- 1 Finder에서 archive 폴더를 생성하고 모듈, 소스, 리소스 파일을 복사합니다.

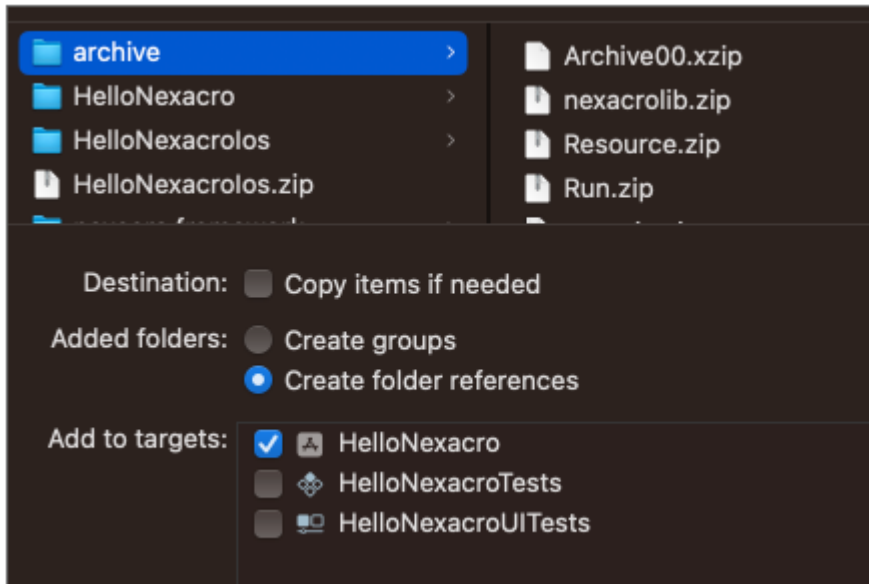
Finder를 실행한 후 Xcode 프로젝트 경로의 하위에 archive 폴더를 생성합니다. 그리고 넥사크로 스튜디오에서 [Packing (Archive&Update)]으로 생성한 파일들을 archive 폴더로 복사합니다.

- 2 Xcode 프로젝트에 archive 폴더를 추가합니다.

Xcode의 프로젝트 탐색기 하단의 + 버튼을 클릭하여 [Add Files to "프로젝트명"]을 선택합니다.

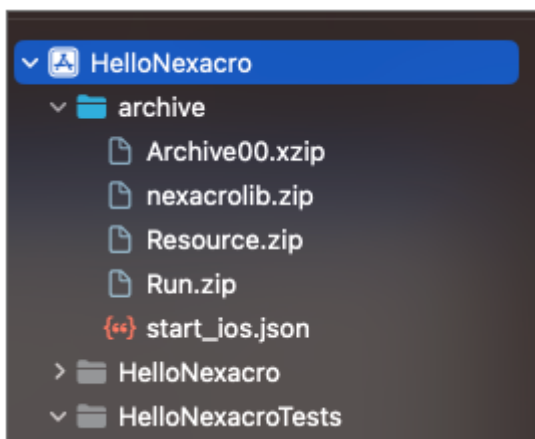


archive 폴더를 선택하고 폴더 추가 옵션에서 "Create folder references"을 선택 후 Add 버튼을 클릭합니다.



- ③ 폴더 및 파일이 프로젝트에 정상적으로 추가되었는지 확인합니다.

Xcode의 프로젝트 탐색기에서 archive 폴더 및 파일이 정상적으로 추가되었는지 확인합니다.



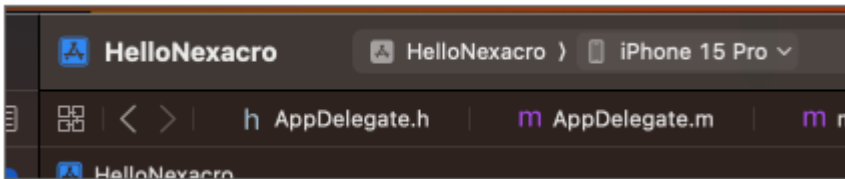
Xcode의 프로젝트 탐색기에 표시되는 폴더는 노란색과 파란색의 두 가지로 표시됩니다. 노란색 폴더는 Xcode에서 프로젝트 구성을 표시하기 위해 존재하는 논리적인 그룹을 표시하며 실제로는 존재하지 않습니다. 파란색 폴더는 실재하는 물리적인 폴더를 표시합니다. 빌드 수행시 노란색 폴

더의 파일은 앱의 루트 디렉터리에 포함되며 파란색 폴더의 파일은 폴더 구조를 유지한 채 포함됩니다.

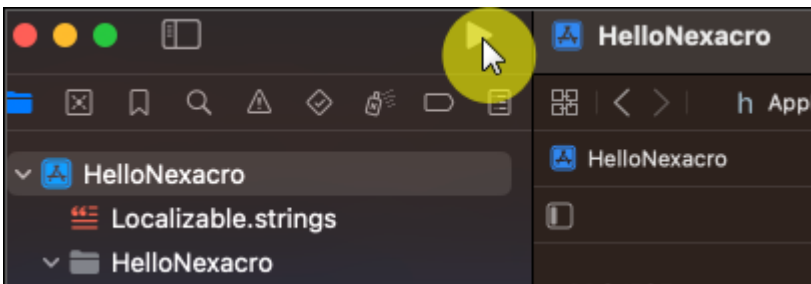
7.3 앱 테스트

iOS 단말기를 PC에 USB로 연결해 바로 테스트하거나 시뮬레이터에 연결해 테스트할 수 있습니다. 단말기를 직접 연결하는 경우에는 단말기에 적절한 프로파일이 설치되어 있어야 합니다.

1. Xcode 상단의 Target Name을 클릭하여 다음과 같이 앱을 구동시킬 iOS 단말기 혹은 시뮬레이터를 선택합니다.



2. Run 기능을 수행합니다. 화면 상단의 아이콘을 직접 클릭하거나 Xcode 메뉴에서 실행할 수 있습니다.



7.4 UIScene 설정 (신규)



24.0.0.1000 이후 버전부터 UIScene 설정을 지원합니다.

아래 문서의 내용은 24.0.0.1000 이후 버전에서만 정상적으로 동작합니다.

UIScene 설정을 위해서는 iOS 13.0 이상, Xcode 14.0 이상 환경이 필요합니다.

프로젝트 생성 및 프레임워크 연결, 빌드 환경 설정은 기존 앱 프로젝트 개발과 같은 방식으로 진행합니다.

7.4.1 SceneDelegate 설정

신규 프로젝트 생성 시 파일이 자동 생성됩니다. 생성된 파일을 다음과 같이 편집합니다.

SceneDelegate.h

```
#import <UIKit/UIKit.h>
#import <nexacro/NXAppDelegate.h>

@interface SceneDelegate : UIResponder <UIWindowSceneDelegate>

@property (strong, nonatomic) UIWindow *window;

@end
```

SceneDelegate.m

```
#import "SceneDelegate.h"

@interface SceneDelegate ()

@end

@implementation SceneDelegate

- (void)scene:(UIScene *)scene willConnectToSession:(UISceneSession *)session options:(
UISceneConnectionOptions *)connectionOptions {
    // Cast to UIWindowScene
    UIWindowScene *windowScene = (UIWindowScene *)scene;

    // Calling configureWindowForScene in the framework
    NXAppDelegate *appDelegate = (NXAppDelegate *)UIApplication.sharedApplication.delegate;
    [appDelegate configureWindowForScene:windowScene];

    // Setting the window reference in SceneDelegate
    self.window = appDelegate.window;
}
```

```

- (void)sceneDidDisconnect:(UIScene *)scene {
}

- (void)sceneDidBecomeActive:(UIScene *)scene {
}

- (void)sceneWillResignActive:(UIScene *)scene {
}

- (void)sceneWillEnterForeground:(UIScene *)scene {
}

- (void)sceneDidEnterBackground:(UIScene *)scene {
}

```

@end

7.5 UIScene 설정 (마이그레이션)



24.0.0.1000 이후 버전부터 UIScene 설정을 지원합니다.

아래 문서의 내용은 24.0.0.1000 이후 버전에서만 정상적으로 동작합니다.

UIScene 설정을 위해서는 iOS 13.0 이상, Xcode 14.0 이상 환경이 필요합니다.

기존에 작성한 앱 프로젝트를 UIScene 방식으로 전환할 때 필요한 내용을 설명합니다.

	기존 방식	UIScene 방식
앱 라이프사이클	UIApplicationDelegate	UIWindowSceneDelegate
Window 관리	AppDelegate	SceneDelegate
멀티 윈도우	미지원	지원 (iPad)
iOS 지원	iOS 13 이전	iOS 13 이상
향후 지원	지원 중단 예정	필수

7.5.1 프레임워크 업데이트

NexacroAppDelegate.h 파일에 `configureWindowForScene` 메서드가 선언되어 있는지를 확인합니다. 해당 메서드가 없는 경우 프레임워크를 업데이트합니다.

```
- (void)configureWindowForScene:(UIWindowScene *)windowScene API_AVAILABLE(ios(13.0));
```

7.5.2 SceneDelegate 설정

프로젝트 폴더에서 `SceneDelegate.h`, `SceneDelegate.m` 파일을 생성하고 파일 편집 후 다음과 같이 프로젝트에 파일을 추가합니다.

1. Xcode > File > Add Files to "YourProject"...
2. `SceneDelegate.h`, `SceneDelegate.m` 파일 선택
3. Add 클릭

SceneDelegate.h

```
#import <UIKit/UIKit.h>
#import <nexacro/NXAppDelegate.h>

@interface SceneDelegate : UIResponder <UIWindowSceneDelegate>

@property (strong, nonatomic) UIWindow *window;

@end
```

SceneDelegate.m

```
#import "SceneDelegate.h"

@interface SceneDelegate ()

@end

@implementation SceneDelegate
```

```

- (void)scene:(UIScene *)scene willConnectToSession:(UISceneSession *)session options:(
UISceneConnectionOptions *)connectionOptions {
    // Cast to UIWindowScene
    UIWindowScene *windowScene = (UIWindowScene *)scene;

    // Calling configureWindowForScene in the framework
    AppDelegate *appDelegate = (AppDelegate *)UIApplication.sharedApplication.delegate;
    [appDelegate configureWindowForScene:windowScene];

    // Setting the window reference in SceneDelegate
    self.window = appDelegate.window;
}

- (void)sceneDidDisconnect:(UIScene *)scene {
}

- (void)sceneDidBecomeActive:(UIScene *)scene {
}

- (void)sceneWillResignActive:(UIScene *)scene {
}

- (void)sceneWillEnterForeground:(UIScene *)scene {
}

- (void)sceneDidEnterBackground:(UIScene *)scene {
}

@end

```

7.5.3 Scene 설정 추가

Info.plist 파일을 편집기로 열어 UIApplicationSceneManifest 키를 추가합니다.

```

...
<key>UIApplicationSceneManifest</key>
<dict>
    <key>UIApplicationSupportsMultipleScenes</key>

```

```

<false/>
<key>UISceneConfigurations</key>
<dict>
  <key>UIWindowSceneSessionRoleApplication</key>
  <array>
    <dict>
      <key>UISceneConfigurationName</key>
      <string>Default Configuration</string>
      <key>UISceneDelegateClassName</key>
      <string>SceneDelegate</string>
    </dict>
  </array>
</dict>
</dict>
...

```

7.5.4 Info.plist 설정 확인

키	조치
UIMainStoryboardFile	삭제 또는 유지 (Scene 설정에 영향을 미치지 않음)
UISceneStoryboardFile	삭제
UILaunchStoryboardName	유지

8.

앱 개발 및 실행 (Android - Android Studio)

8.1 앱 개발 환경 설정

넥사크로 안드로이드 앱은 안드로이드 앱 개발 도구인 안드로이드 스튜디오를 이용하여 개발하거나 넥사크로 제품군의 하나인 앱 빌더(App Builder)를 이용하여 개발할 수 있습니다. 이번 장에서는 안드로이드 스튜디오를 이용하여 앱을 개발하는 방법을 설명합니다. 기본적인 개발 절차는 일반적인 안드로이드 앱을 만드는 방법과 동일합니다.



이미 앱 개발 환경이 설정되어 있다면 앱 개발 환경 설정은 건너뛰어도 됩니다.

안드로이드 스튜디오 설치파일은 기본적으로 SDK 도구를 포함하고 있습니다. 아래 사이트에서 환경에 맞는 설치파일을 내려받을 수 있습니다.

<https://developer.android.com/studio/index.html>



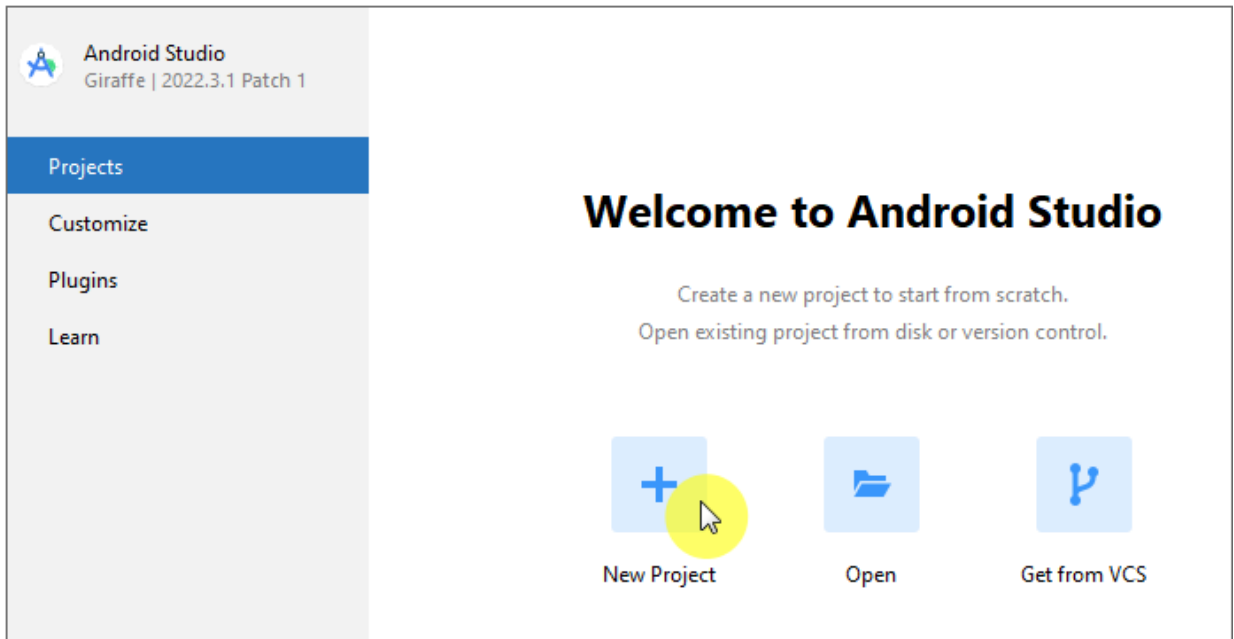
이번 장에서는 안드로이드 스튜디오 2023.3.1 버전 설치를 기준으로 작성했습니다.

8.2 앱 프로젝트 개발

앱 개발 환경이 설정되었다면 안드로이드 스튜디오를 실행해서 안드로이드 운영체제에서 동작하는 앱을 만들 수 있습니다. 앱 프로젝트를 진행하기 전에 넥사크로 스튜디오에서 생성한 모듈, 소스, 리소스 파일은 지정된 경로에 위치해야 합니다.

8.2.1 프로젝트 생성

- 1 기존에 열어놓은 프로젝트가 없다면 빠른 시작(Quick Start) 창에서 새로운 프로젝트를 생성합니다. 아래 그림에서 [Create New Project]를 선택합니다.

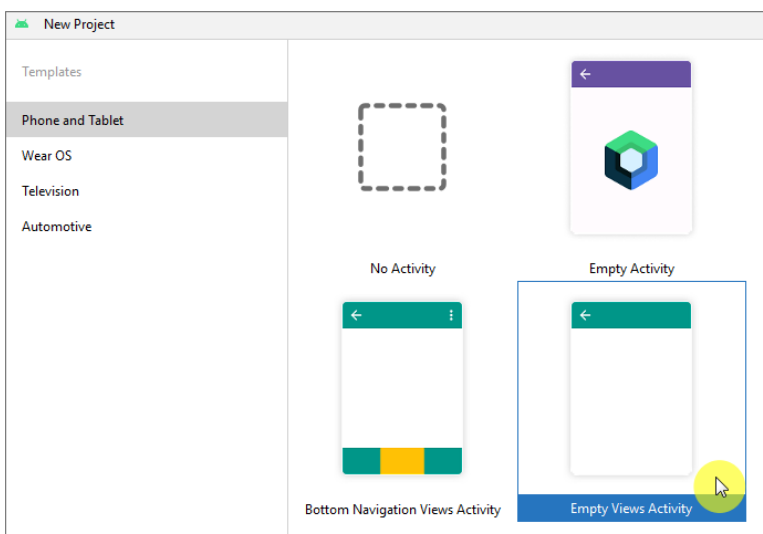


다른 프로젝트 작업 중에 새로운 프로젝트를 생성하고자 한다면 아래 메뉴에서 생성할 수 있습니다.

File > New > New Project

- 2 안드로이드에서 제공하는 프로젝트 템플릿을 선택합니다.

선택하는 템플릿에 따라 구조, 화면 레이아웃, 파일 등의 구성이 달라집니다. 여기서는 넥사크로 앱을 담기 위한 기본적인 앱 형태면 충분하므로 Empty Views Activity를 선택한 다음 [Next] 버튼을 클릭합니다.



- ③ 앱의 이름, 패키지 이름 그리고 프로젝트의 경로를 설정합니다.

Empty Views Activity

Creates a new empty activity

Name **1** Hello Nexacro Android

Package name **2** com.example.hellonexacroandroid

Save location **3** D:\TEST\AndroidStudioProjects\HelloNexacroAndroid

Language **4** Java

Minimum SDK **5** API 24 ("Nougat"; Android 7.0)

i Your app will run on approximately 95.4% of devices.
[Help me choose](#)

Build configuration language **?** Kotlin DSL (build.gradle.kts) [Recommended]

	항목	설명
1	Name	생성할 앱 이름을 입력합니다.
2	Package Name	패키지 이름을 입력합니다.
3	Save location	프로젝트가 저장될 위치를 지정합니다.
4	Language	개발 언어를 선택합니다. 기본 설정값은 kotlin이지만, 이번 장에서는 Java로 설정합니다.
5	Minimum API	최소 사양 SDK를 설정합니다. SDK 항목을 변경하면 하단에서 해당 버전 API 레벨의 디바이스 호환성에 관한 정보를 확인할 수 있습니다.



넥사크로에서 지원하는 안드로이드 최소 지원 사양은 5.0 버전입니다. 따라서 SDK 설정시 API 레벨 21 이상을 지원하는 SDK를 사용해야 합니다.

- ④ [Finish] 버튼을 클릭하면 프로젝트가 생성됩니다.

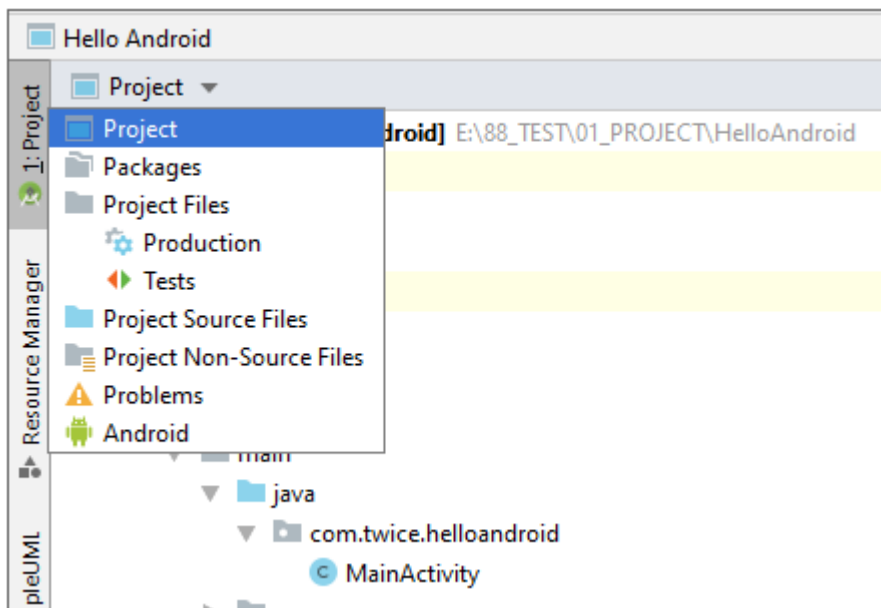
프로젝트 창이 열리면서 추가적으로 필요한 파일은 자동으로 내려받아 설치합니다.

8.2.2 넥사크로 라이브러리 설정

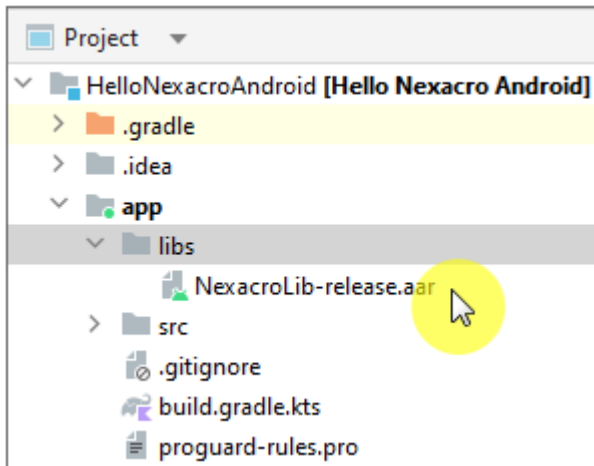
앱 개발시 사용할 수 있도록 제공되는 넥사크로 안드로이드 아카이브(AAR) 파일을 안드로이드 프로젝트에 설정합니다. 별도의 설정은 필요치 않으며 정해진 경로에 복사해주면 됩니다.

라이브러리 파일	경로	필수 여부
NexacroLib-release.aar	[프로젝트 경로]\WappWlibsW	필수
Acceleration-release.aar	[프로젝트 경로]\WappWlibsW	앱에서 사용하는 Device API에 따라 선택적으로 추가
AudioPlayer-release.aar		
AudioRecorder-release.aar		
BluetoothLE-release.aar		
Camera-release.aar		
Contact-release.aar		
ExternalAPI-release.aar		
Geolocation-release.aar		
ImagePicker-release.aar		
LiteDB-release.aar		
Network-release.aar		
Phone-release.aar		
Sms-release.aar		
Vibrate-release.aar		
XPush-release.aar		

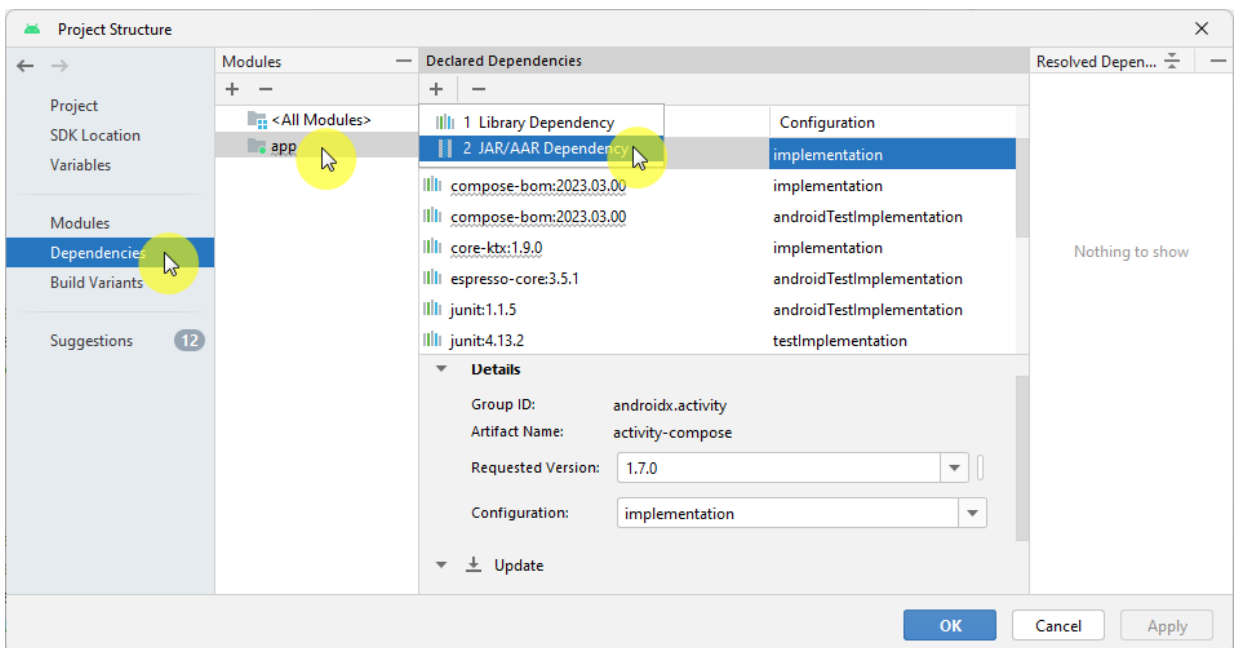
라이브러리 파일을 실제 프로젝트 경로의 정해진 위치에 복사하려면 프로젝트 윈도우 상단에서 뷰 옵션을 [Project]로 변경합니다. 안드로이드 스튜디오를 설치하면 기본적으로 뷰 모드가 [Android]로 되어 있는데 이를 [Project]로 설정해야 모든 폴더 구조를 확인할 수 있습니다.



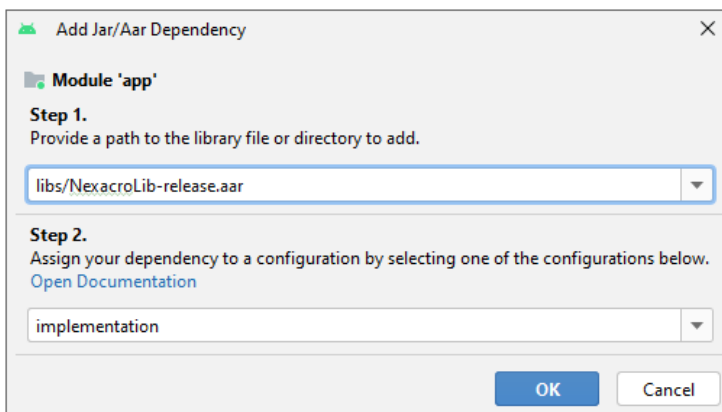
- 1 NexacroLib-release.aar 파일과 필요한 aar 파일은 [프로젝트 경로 > app > libs] 폴더로 복사합니다.



- ② 메뉴 [File > Project Structure]에서 "Dependencies"를 선택하고 Modules 항목에서 "app"을 선택한 후 "+" 버튼을 클릭하고 "JAR/AAR Dependency" 항목을 선택합니다.



- ③ 입력 창에서 aar 파일 경로를 입력하고 [OK] 버튼을 클릭합니다.



- 4 app > build.gradle.kts 파일에 aar 파일이 추가된 것을 확인합니다.

```
dependencies { this: DependencyHandlerScope

    implementation("androidx.appcompat:appcompat:1.6.1")
    implementation("com.google.android.material:material:1.9.0")
    implementation("androidx.constraintlayout:constraintlayout:2.1.4")
    implementation(files(...paths: "libs/NexacroLib-release.aar"))
    testImplementation("junit:junit:4.13.2")
    androidTestImplementation("androidx.test.ext:junit:1.1.5")
    androidTestImplementation("androidx.test.espresso:espresso-core:3.5.1")
}
```



의존성 관리 정보는 [의존성 관리](#) 항목을 참고하세요.

8.2.3 리소스 설정

앱에서 사용할 로딩 이미지, 아이콘, 메시지, 레이아웃 등을 설정하는 단계입니다. 리소스 파일을 단말기의 화면 크기, 밀도에 따라 만든 후 적절한 폴더에 배치하면 안드로이드 시스템은 사용자에게 최적화된 뷰를 제공합니다.

기본 리소스는 넥사크로 안드로이드 아카이브(AAR) 파일에 포함되어 있습니다. 기본 리소스를 사용할 경우에는 별도 설정이 필요 없습니다.



리소스에 관한 더 상세한 내용은 안드로이드 매뉴얼을 참조하시기 바랍니다.



<https://developer.android.com/guide/topics/resources/providing-resources>

이미지 설정

앱에서 사용하는 이미지는 res 폴더 아래에 밀도에 따라 알맞은 폴더에 지정합니다. 예를 들어, 로딩 이미지는 drawable 폴더로, 앱 아이콘은 mipmap 폴더로 지정합니다. 적용할 이미지 파일은 앱에서 지원하는 해상도에 따라 폴더를 달리 설정해야 합니다.

항목	설명
ic_launcher.png	바탕화면에 설치되는 앱 아이콘 이미지입니다. AndroidManifest.xml 파일에서 파일명을 변경할 수 있습니다.
splashimage_phone_landscape.png	안드로이드 폰 가로 방향 스플래시 이미지입니다(모두 소문자).
splashimage_phone_portrait.png	안드로이드 폰 세로 방향 스플래시 이미지입니다(모두 소문자).
splashimage_pad_landscape.png	안드로이드 태블릿 가로 방향 스플래시 이미지입니다(모두 소문자).
splashimage_pad_portrait.png	안드로이드 태블릿 세로 방향 스플래시 이미지입니다(모두 소문자).



런처 아이콘, 로딩 이미지 등을 별도로 지정하지 않는 경우에는 기본 설정이 적용되므로 앱 동작에는 아무런 문제가 없습니다.

문자열 설정

기본 문자열 설정은 넥사크로 안드로이드 아카이브(AAR) 파일에 포함된 strings.xml 파일 설정에 따라 적용됩니다. 기본 문자열 설정을 변경하려면 res 폴더 아래 values, values-ko, values-ja, values-zh 폴더에 strings.xml 파일을 만들고 변경할 항목만 설정할 수 있습니다.

아래 코드는 넥사크로 안드로이드 아카이브 파일에 포함된 strings.xml 파일 내용입니다. 배포 버전에 따라 내용은 달라질 수 있습니다.

values-ko\strings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="needupdate">업데이트가 있습니다. \n업데이트 완료 후 시작합니다.</string>
    <string name="update">업데이트</string>
    <string name="exit">종료</string>
    <string name="updateFail">업데이트가 실패하였습니다. \n재시작 해 주세요.</string>
    <string name="loadingFail">초기 로딩이 실패하였습니다. \n재시작 해 주세요.</string>
    <string name="updatedone">업데이트가 되었습니다.</string>
    <string name="checkforupdates">업데이트 확인 중입니다.</string>
    <string name="installforupdates">업데이트 설치 중입니다.</string>
    <string name="ok">확인</string>
    <string name="downloadingforupdates">업데이트 다운 중입니다.</string>
    <string name="allow_permission">엔진 업데이트의 정상적인 기능을 위해 해당 권한이 반드시 필요하니 권한을 허용하세요.</string>
```

```
<string name="denied_permission">엔진 업데이트 권한거부로 종료합니다.</string>
<string name="ssl_invalid">SSL 인증서가 올바르지 않습니다. 계속 진행하시겠습니까?</string>
<string name="cancel">아니오</string>
</resources>
```

문자열은 사용자의 상황에 맞게 수정할 수 있습니다. 그러나 위에 정의된 문자열 name은 앱에서 고정적으로 사용하므로 수정하지 않습니다.

영문 문자열 외 한국어나 일본어 등의 문자열을 추가하려면 res 폴더 아래 values-ko, values-ja 와 같은 식으로 폴더를 추가하고 strings.xml 파일을 추가합니다. 더 상세한 내용은 안드로이드 매뉴얼을 참조하시기 바랍니다.



<https://developer.android.com/training/basics/supporting-devices/languages.html>

8.2.4 Config 설정

기본 Config 설정은 넥사크로 안드로이드 아카이브(AAR) 파일에 포함된 nexacro_config.xml 파일 설정에 따라 적용됩니다. Config 설정을 변경하려면 xml 폴더에 nexacro_config.xml 파일을 만들어 설정합니다.

아래 코드는 넥사크로 안드로이드 아카이브 파일에 포함된 nexacro_config.xml 파일 내용입니다. 배포 버전에 따라 내용은 달라질 수 있습니다.

xml\nexacro_config.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<nexacro-config>
  <application style="default" dialog-position="bottom" file-logging="true" quiet="false"
    loglevel="debug" tracemode="none" traceduration="-1"/>
  <notification enable="true" handler="com.nexacro.notification.DefaultHandler"/>
  <xpush-server request-missing-message="false" sender-id="1234567890"/>
  <updater cancelable="true" force="true" restart="false" errormsg="true" quiet="false"
    failpass="true"/>
  <log filepath="%SD_CARD%\nreLog.log" filesize="408960" backupfilecount="5"/>
</nexacro-config>
```

nexacro_config.xml에 설정할 수 있는 기능은 다음과 같습니다.

기능	속성	설정 값	설명
application	dialog-position	"top" "center" "bottom"	앱 업데이트 진행 정보를 표시할 팝업 위치를 설정합니다.

기능	속성	설정 값	설명
	file-logging	"true" "false"	로딩 에러 정보를 파일로 저장할지 설정합니다. "true"로 설정시 외부저장소(context.getExternalCacheDir()) > 내부저장소(context.getCacheDir()) 순서로 파일 저장을 시도합니다. 파일은 [외부/내부저장소 경로]/logs/yyyy_MM_dd.txt 형태로 저장됩니다.
	quiet	"true" "false"	앱 실행 팝업을 표시할지 설정합니다. "true"로 설정시 "Loading Application" 실행 메시지가 출력되지 않습니다.
updater	force	"true" "false"	start_android.json에 업데이트 파일 정보 존재시 팝업으로 표시할지 설정합니다. "true"로 설정시 팝업 알림없이 강제로 업데이트를 진행합니다.
	cancelable	"true" "false"	업데이트 파일 존재시 팝업에 업데이트 취소 버튼을 표시할지 설정합니다. 취소 버튼을 클릭하면 업데이트를 진행하지 않고 앱을 실행합니다.
	errmsg	"true" "false"	앱 로딩에 실패했을 때 에러 정보를 팝업으로 표시할지 설정합니다.
	quiet	"true" "false"	업데이트 팝업을 표시할지 설정합니다. "true"로 설정해도 업데이트 파일 존재시 진행 단계는 표시됩니다.
	failpass	"true" "false"	업데이트 오류를 무시할지 설정합니다.
xpush-server	request-missing-message	"true" "false"	xpush 서버로 미수신 메시지를 자동으로 요청할지 설정합니다.
notification	enable	"true" "false"	알림기능을 사용할지 설정

기능	속성	설정 값	설명
			합니다.
	handler	“[함수 이름]”	알림 수신시 메시지 혹은 데이터 처리를 위한 함수를 설정합니다. 기본 값은 “com.nexacro.notification.DefaultHandler” 입니다.



file-logging 속성의 [저장소 경로] 위치에 대한 자세한 정보는 아래 링크를 참고하세요.

[https://developer.android.com/reference/android/content/Context.html#getExternalCacheDir\(\)](https://developer.android.com/reference/android/content/Context.html#getExternalCacheDir())

[https://developer.android.com/reference/android/content/Context.html#getCacheDir\(\)](https://developer.android.com/reference/android/content/Context.html#getCacheDir())

8.2.5 빌드 환경 설정

안드로이드 프로젝트를 생성하면서 기본으로 만들어진 MainActivity.java 파일과 AndroidManifest.xml 파일을 넥사크로 환경에 맞게 수정합니다.

MainActivity.java

프로젝트 생성 시 액티비티 이름을 지정해주는데 별도로 수정하지 않는다면 MainActivity.java 라는 이름으로 생성됩니다. [프로젝트 > app > src > main > java] 폴더에서 지정된 패키지 이름 아래에 MainActivity.java 파일을 선택하고 아래와 같이 수정해줍니다.

MainActivity.java

```
package com.example.hellonexacroandroid;

import android.os.Bundle;
import com.nexacro.NexacroActivity;

public class MainActivity extends NexacroActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        // Update Type: Server, Update
    }
}
```

```

setProjectURL("http://[URL]/");
setBootstrapURL("http://[URL]/start_android.json");

// Update Type: Local
// setBootstrapURL("file:///android_asset/archive/start_android.json");
}
}

```

plugin_config.xml

plugin_config.xml 파일은 [프로젝트 > app > src > main > res > xml]에 위치하며 Device API에 대한 기본적인 정보와 필요한 정보를 담고 있습니다. 기본적으로 필요한 정보는 아래 코드를 참고해 작성합니다.



넥사크로 라이브러리 설정 시 NexacroLib-release.aar 외에 다른 라이브러리를 설정하지 않은 경우에는 plugin_config.xml 파일이 없어도 동작에 영향을 미치지 않습니다.

plugin_config.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<plugin-config>
  <!-- Nexacro Device API -->
  <plugin name="Network" class="com.nexacro.deviceapi.Network" onload="false"/>
  <plugin name="Geolocation" class="com.nexacro.deviceapi.Geolocation" onload="false"/>
  <plugin name="Acceleration" class="com.nexacro.deviceapi.Acceleration" onload="false"/>
  <plugin name="Sound" class="com.nexacro.deviceapi.AudioPlayer" onload="false"/>
  <plugin name="AudioRecorder" class="com.nexacro.deviceapi.AudioRecorder" onload="false"/>
  <plugin name="Camera" class="com.nexacro.deviceapi.Camera" onload="false"/>
  <plugin name="ImagePicker" class="com.nexacro.deviceapi.ImagePicker" onload="false"/>
  <plugin name="ExternalAPI" class="com.nexacro.deviceapi.ExternalAPI" onload="false"/>
  <plugin name="Vibrator" class="com.nexacro.deviceapi.Vibrate" onload="false"/>
  <plugin name="Call" class="com.nexacro.deviceapi.Phone" onload="false"/>
  <plugin name="Sms" class="com.nexacro.deviceapi.SmsManager" onload="false"/>
  <plugin name="ContactSet" class="com.nexacro.deviceapi.Contact" onload="false"/>
  <plugin name="SQLStatement" class="com.nexacro.deviceapi.LiteDBStatement" onload="false"/>
  <plugin name="SQLConnection" class="com.nexacro.deviceapi.LiteDBConnection" onload="false"/>
</plugin-config>

```

AndroidManifest.xml

AndroidManifest.xml 파일은 [프로젝트 > app > src > main]에 위치하며 앱에 대한 기본적인 정보와 특정 기능 실행 시 필요한 정보를 담고 있습니다. 기본적으로 필요한 정보는 아래 코드를 참고해 작성합니다.

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.HelloNexacroAndroid"
        android:usesCleartextTraffic="true"
        tools:targetApi="31">

        <activity
            android:name=".MainActivity"
            android:configChanges="orientation|screenSize|keyboardHidden"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <action android:name="OPEN_NEXACRO_ACTIVITY_EXAMPLE" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

    </application>

</manifest>
```



targetsdk 버전을 28 이상으로 지정한 경우에 HTTP 통신을 유지하기 위해서는 `usesCleartextTraffic` 값을 `true`로 지정해주어야 합니다.

해당 설정이 없을 경우 default 값이 `false`로 적용되어 HTTP 통신이 허용되지 않습니다.

<https://developer.android.com/guide/topics/manifest/application-element#usesCleartextTraffic>

```
<application
    android:allowBackup="true"
    android:usesCleartextTraffic="true"
```

AndroidManifest.xml > 추가 권한 설정

배포할 앱의 기능에 따라 필요한 권한 및 기타 정보를 추가로 지정합니다. 예를 들어, 카메라 기능을 사용한다면 아래와 같은 항목이 추가되어야 합니다.

```
...
<uses-permission android:name="android.permission.CAMERA" />
<activity android:name="com.nexacro.deviceAPI.CameraListener"
    android:screenOrientation="landscape">
    <intent-filter>
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
<uses-feature android:name="android.hardware.camera" android:required="false"/>
<uses-feature android:name="android.hardware.camera.autofocus"
    android:required="false"/>
...
```

다음은 많이 사용되는 기능의 권한 설정 코드입니다. 필요한 항목만 `AndroidManifest.xml` 파일에 추가합니다. 이 외에 안드로이드에서 정의한 모든 권한 목록을 보려면 [Manifest.permission](#)을 참조하시기 바랍니다.



기능에 필요한 권한을 설정하지 않으면 앱 실행시 해당 권한이 없다는 메시지와 함께 기능이 동작하지 않습니다.

```
<!-- MediaPlayer, AudioManager -->
<uses-permission android:name="android.permission.RECORD_AUDIO" />
<uses-permission android:name="android.permission.MODIFY_AUDIO_SETTINGS" />
```

```
<!-- Call -->
<uses-permission android:name="android.permission.CALL_PHONE" />
```

```
<!-- Contact -->
<uses-permission android:name="android.permission.READ_CONTACTS" />
<uses-permission android:name="android.permission.WRITE_CONTACTS" />
```

```
<!-- Camera -->
<uses-permission android:name="android.permission.CAMERA" />
<activity android:name="com.nexacro.deviceAPI.CameraListener" android:screenOrientation="
landscape">
    <intent-filter>
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
<uses-feature android:name="android.hardware.camera" android:required="false"/>
<uses-feature android:name="android.hardware.camera.autofocus" android:required="false"/>
```

```
<!-- Geolocation -->
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_LOCATION_EXTRA_COMMANDS" />
```

```
<!-- Acceleration -->
<uses-permission android:name="android.permission.HIGH_SAMPLING_RATE_SENSORS" tools:ignore="
HighSamplingRate" />
```

```
<!-- Map -->
<uses-permission android:name="com.google.android.providers.gsf.permission.READ_GSERVICES"/>
<uses-library android:name="com.google.android.maps" />
<meta-data android:name="com.google.android.maps.v2.API_KEY" android:value="user value" />
```



구글맵을 사용하려면 API KEY 값을 받아서 입력해야 합니다. 자세한 내용은 다음 링크를 참조하십시오.
https://developers.google.com/maps/documentation/android/start#obtain_a_google_maps_api_key

```

<!-- SMS -->
<uses-permission android:name="android.permission.SEND_SMS" />
<uses-permission android:name="android.permission.RECEIVE_SMS" />
<uses-permission android:name="android.permission.READ_SMS" />
<uses-permission android:name="android.permission.WRITE_SMS" />

<receiver android:name="com.nexacro.deviceAPI.SmsRecv">
    <intent-filter>
        <action android:name="android.provider.Telephony.SMS_RECEIVED" />
    </intent-filter>
    <intent-filter>
        <action android:name="android.provider.Telephony.WAP_PUSH_RECEIVED" />
        <data android:mimeType="application/vnd.wap.mms-message" />
    </intent-filter>
</receiver>

```

```

<!-- Vibrator -->
<uses-permission android:name="android.permission.VIBRATE" />

```

```

<!-- Bluetooth -->
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

<service
    android:name="com.nexacro.deviceAPI.BluetoothLEAdapterService"
    android:enabled="true"
    android:exported="true">
</service>

```

```

<!-- ExternalAPI -->
<uses-permission android:name="android.permission.GET_TASKS" />

```

```

<!-- Etc -->
<uses-permission android:name="android.permission.GET_ACCOUNTS" />
<uses-permission android:name="android.permission.READ_PHONE_STATE" />

```

AndroidManifest.xml > FCM 설정



About FCM messages

<https://firebase.google.com/docs/cloud-messaging/concept-options>

```
<!-- Adding NexacroNotificationService -->
<service android:name="com.nexacro.notification.NexacroNotificationService">
    <intent-filter>
        <action android:name="com.google.firebase.MESSAGING_EVENT" />
    </intent-filter>
</service>

<!-- Adding NexacroActivity intent-filter -->
<activity android:name="com.nexacro.NexacroActivity"
    android:launchMode="singleTask"
    android:noHistory="false"
    android:hardwareAccelerated="true"
    android:windowSoftInputMode="adjustResize"
    android:configChanges="orientation|keyboard|keyboardHidden|screenSize|navigation|uiMode">
    <intent-filter>
        <action android:name="OPEN_NEXACRO_ACTIVITY_EXAMPLE" />
        <category android:name="android.intent.category.DEFAULT" />
    </intent-filter>
</activity>
```

FCM 서비스를 이용하는 경우에는 build.gradle 설정에 아래 항목을 추가해주어야 합니다. 안드로이드 스튜디오에서 자동으로 설정되지 않는다면 확인 후 추가해주고 Firebase 콘솔에서 생성한 "google-services.json" 파일을 모듈 경로에 복사합니다.



Android 프로젝트에 Firebase 추가

<https://firebase.google.com/docs/android/setup>

AndroidManifest.xml > ImagePicker, Camera, FileUpload, FileDialog 설정



targetsdk 버전을 24 이상으로 지정하고 ImagePicker, FileDialog, FileUpload, Camera 컴포넌트를 사용하는 경우(Camera 컴포넌트는 targetsdk 버전 29 이상)에는 res/xml 폴더안에 file_paths.xml 파일을 생성하고 AndroidManifest.xml 파일과 build.gradle 파일에 아래 항목을 추가해주어야 합니다.



WebBrowser 컴포넌트에 연결된 HTML 콘텐츠에서 Input 태그의 capture 속성을 사용할 경우 파일에 접근하기 위해 같은 설정이 필요합니다.

file_paths.xml

```
<?xml version="1.0" encoding="utf-8"?>
<paths xmlns:android="http://schemas.android.com/apk/res/android">
    <external-cache-path name="cache" path="." />
    <external-files-path name="files" path="." />
</paths>
```

AndroidManifest.xml / FileProvider

```
<provider
    android:name="android.support.v4.content.FileProvider"
    android:authorities="${applicationId}.fileprovider"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data
        android:name="android.support.FILE_PROVIDER_PATHS"
        android:resource="@xml/file_paths"/>
</provider>

<!-- AndroidX -->
<provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="${applicationId}.fileprovider"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data
        android:name="android.support.FILE_PROVIDER_PATHS"
        android:resource="@xml/file_paths"/>
</provider>
```

build.gradle / exifinterface

```
dependencies {
    implementation 'com.android.support:exifinterface:28.0.0'
    //AndroidX
    implementation 'androidx.exifinterface:exifinterface:1.3.2'
}
```

의존성 관리

build.gradle 파일에 라이브러리 정보를 작성합니다.

```
implementation 'androidx.appcompat:appcompat:1.5.1'
implementation 'com.google.android.material:material:1.6.1'
implementation 'androidx.constraintlayout:constraintlayout:2.1.4'
implementation "androidx.room:room-runtime:2.4.3"
implementation 'androidx.exifinterface:exifinterface:1.3.4'
implementation 'com.squareup.okhttp3:okhttp:4.10.0'
annotationProcessor "androidx.room:room-compiler:2.4.3"
```

XPush 사용 시에는 아래 라이브러리 정보를 추가합니다.

```
implementation 'org.slf4j:slf4j-api:1.7.36'
```

Firebase 사용 시에는 아래 라이브러리 정보를 추가합니다.

```
implementation 'com.google.firebase:firebase-core:21.1.1'
implementation 'com.google.firebase:firebase-messaging:23.0.8'
```

8.2.6 네트워크 연결 없는 앱 실행 설정

넥사크로 스튜디오에서 앱을 패키징할 때 Update Type을 Local이나 Update(Local+Server)으로 설정하면 네트워크 연결 없이도 앱을 실행할 수 있습니다. 일반적으로 네트워크 연결이 필요 없는 경우에는 Local 타입을 사용하고, 평소에는 서버와 통신이 필요하지만 연결에 문제가 생길 시에도 앱이 동작해야 할 경우에는 Update(Local+Server) 타입을 사용합니다. 이처럼 네트워크 연결이 필요 없거나 문제 발생 시 앱 단독으로 실행이 가능하게 하려면 앱 빌드 시 실행에 필요한 파일을 적재하여 배포해야 합니다.



Update Type 설정값이 Update(Local+Server)인 경우 네트워크가 연결되어 있지 않으면 앱을 실행할 수 없습니다. 앱 로딩 시 부트스트랩 URL을 확인하고 연결이 실패하면 앱을 실행하지 않습니다.

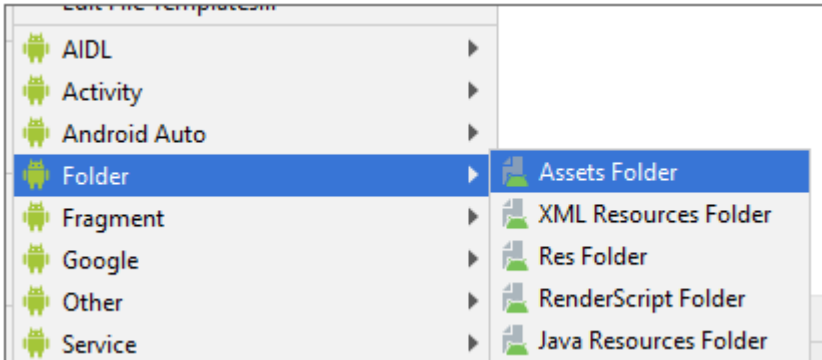
앱 실행에 필요한 파일은 모듈, 소스, 리소스 그리고 start_android.json으로 넥사크로 스튜디오에서 Packing(Archive&Update)을 수행하여 얻을 수 있습니다. 이 파일을 안드로이드 프로젝트에서 애셋(Asset)의 archive 폴더에 적재한 후 앱을 빌드하면 네트워크 연결 없이도 앱을 실행할 수 있습니다.

- ① 안드로이드 스튜디오 프로젝트에서 assets 폴더를 생성합니다.



애셋은 안드로이드 앱에서 사용할 수 있는 논리적인 저장 공간입니다. 안드로이드 프로젝트에서 assets 폴더를 만들고 앱에서 사용할 파일을 복사한 후 APK를 생성하면 앱 실행 시 그 파일을 사용할 수 있습니다. 애셋은 읽기 전용 공간으로 사용되므로 앱 실행 중 변경되지 않을 파일을 이곳에 놓고 사용합니다.

먼저 프로젝트의 app을 선택하고 마우스 오른쪽 버튼을 클릭합니다. 다음과 같이 컨텍스트 메뉴에서 [New > Folder > Assets Folder]를 선택하여 [프로젝트 > app > src > main] 하위에 Assets 폴더를 생성합니다.

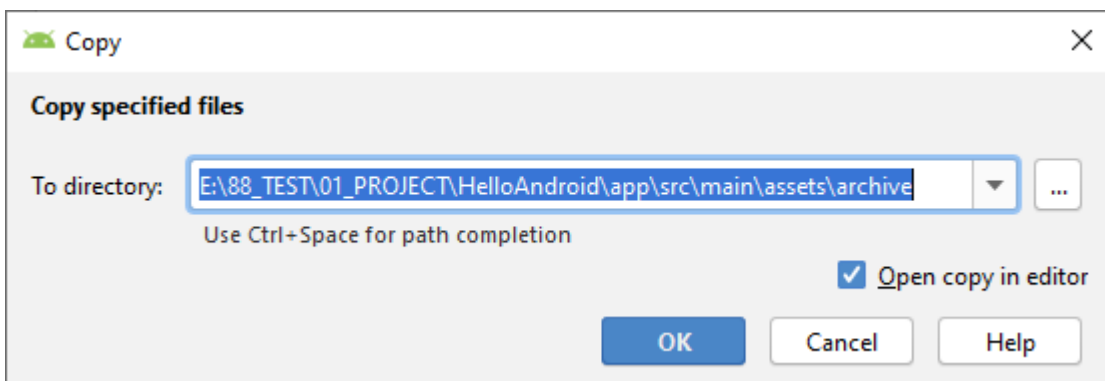


- ② assets 폴더 하위에 archive 폴더를 생성합니다.

[프로젝트 > app > src > main > assets] 폴더를 선택한 후 [New > Directory] 메뉴를 사용해 archive 폴더를 assets 폴더 하위에 생성합니다.

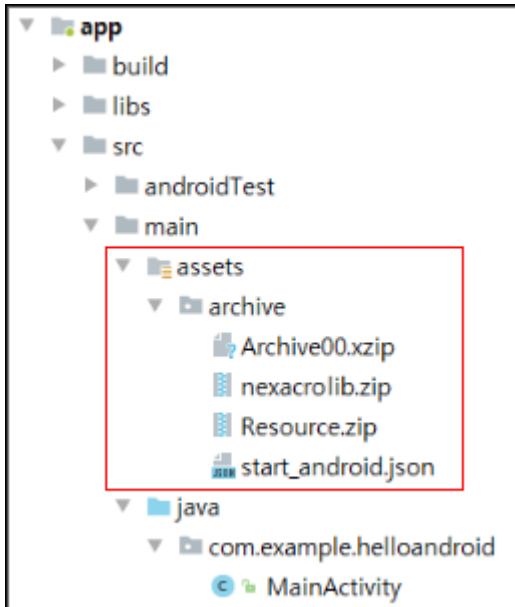
- ③ 모듈, 소스, 리소스 파일을 assets/archive 폴더에 복사합니다.

모듈, 소스, 리소스 파일을 [프로젝트 > app > src > main > assets > archive] 폴더로 복사합니다. 윈도우 탐색기에서 파일을 선택한 후 안드로이드 스튜디오로 복사/붙여넣기 합니다.



- ④ 폴더 및 파일이 프로젝트에 정상적으로 추가되었는지 확인합니다.

안드로이드 스튜디오의 프로젝트 창에서 assets/archive 폴더 및 파일들이 정상적으로 추가되었는지 확인합니다.



8.3 빌드

8.3.1 앱 테스트

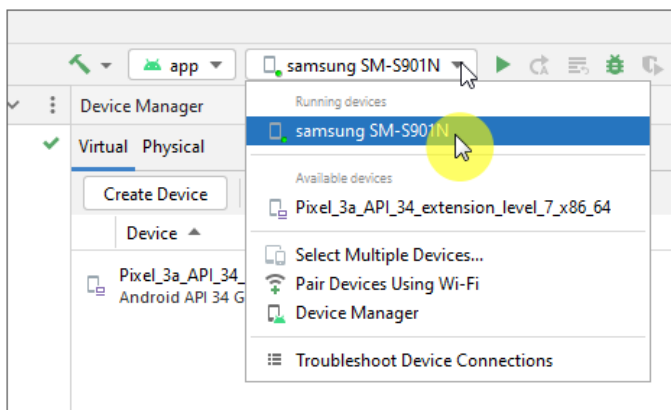
안드로이드 단말기를 PC에 USB로 연결해 바로 테스트할 수 있습니다. 단말기를 직접 연결하는 경우에는 단말기에 개발자 옵션인 [USB 디버깅] 항목을 설정하고 단말기에 맞는 드라이버를 설치해주어야 합니다.



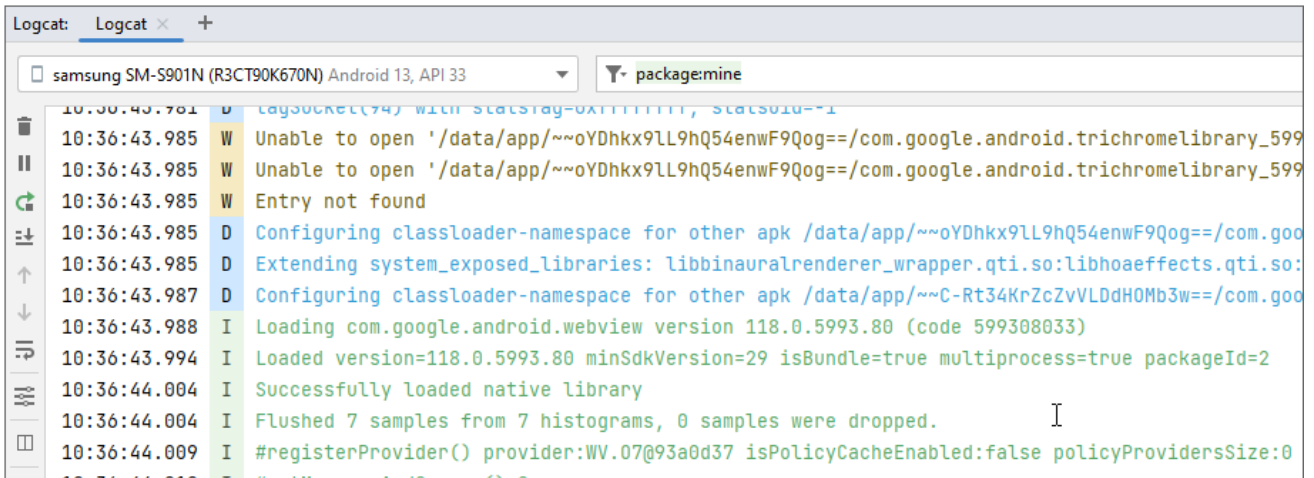
USB 디버깅 옵션 선택 방법은 아래 정보를 참고하세요.

<https://developer.android.com/studio/debug/dev-options>

안드로이드 스튜디오 상단 단말기 목록에서 가상 디바이스 또는 연결된 디바이스를 선택하고 Run 메뉴를 실행합니다.



앱 실행 로그는 안드로이드 스튜디오 하단의 Logcat을 통해 확인할 수 있습니다.



8.3.2 설치 파일 생성

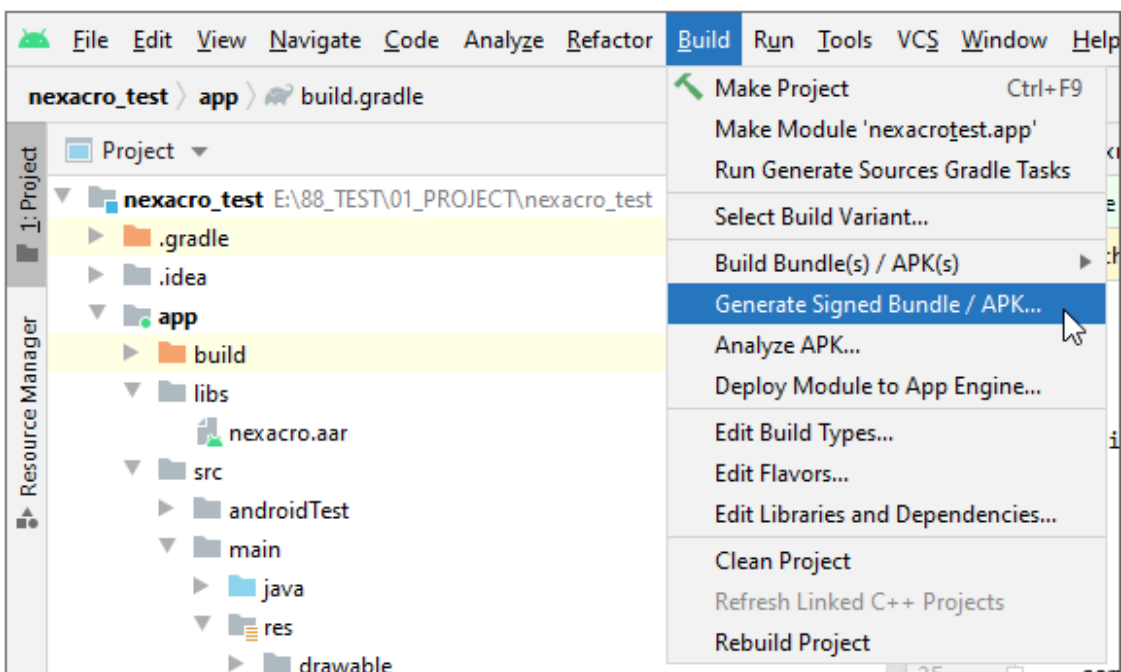
완성된 앱을 배포하기 위해서는 앱을 인식할 수 있는 keystore를 생성하고 서명된 AAB, APK 파일을 생성해야 합니다. keystore 파일은 최초 1회 생성하며 이후 앱이 업데이트되었을때는 최초 생성된 keystore 파일을 사용합니다.

keystore 생성 및 Signed AAB, APK 를 생성하는 방법은 구글의 웹페이지를 참조하십시오.

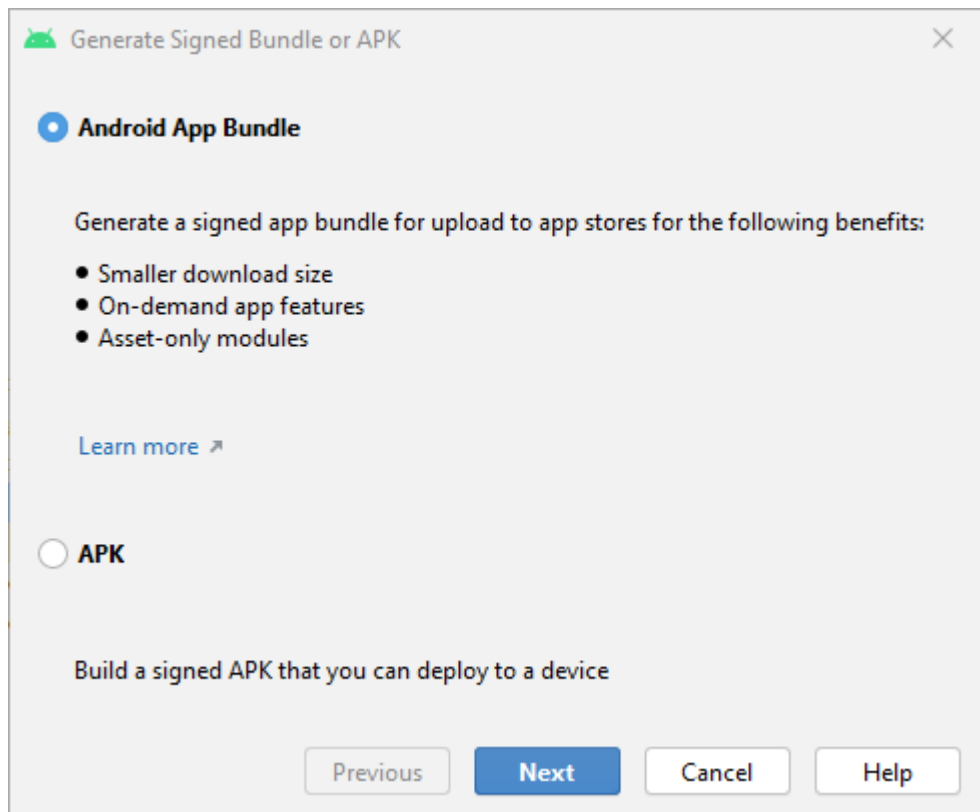


<http://developer.android.com/guide/publishing/app-signing.html>

상단 메뉴에서 [Build > Generate Signed Bundle/APK] 항목을 선택합니다.



기본 설정은 AAB 파일 생성입니다. 필요한 경우 APK 항목을 선택하고 APK 파일을 생성할 수 있습니다.



9.

앱 개발 및 실행 (macOS - Xcode)

넥사크로로 개발한 서비스를 맥 운영체제 사용자에게 하이브리드 운영방식으로 배포할 수 있습니다. 개발자가 작성한 프로젝트를 가지고 관리자는 배포 파일을 생성해야 합니다. 이번 장에서는 배포 파일을 생성하는 과정에 대한 간략한 가이드를 제공합니다.

9.1 준비 사항

9.1.1 개발 환경

맥 운영체제에서 사용할 수 있는 앱을 생성하기 위해서는 Xcode 개발 환경이 필요합니다. 추가적인 코딩 과정은 없고 이미 만들어진 넥사크로 앱을 맥 운영체제에서 사용할 수 있도록 만드는 작업만 필요합니다.

기본적인 개발 환경은 아래와 같습니다.

항목	설명
Xcode	설치가 필요할 수 있습니다. https://developer.apple.com/xcode/
xcodebuild	Xcode 설치 시 같이 설치됩니다.

9.2 앱 프로젝트 개발

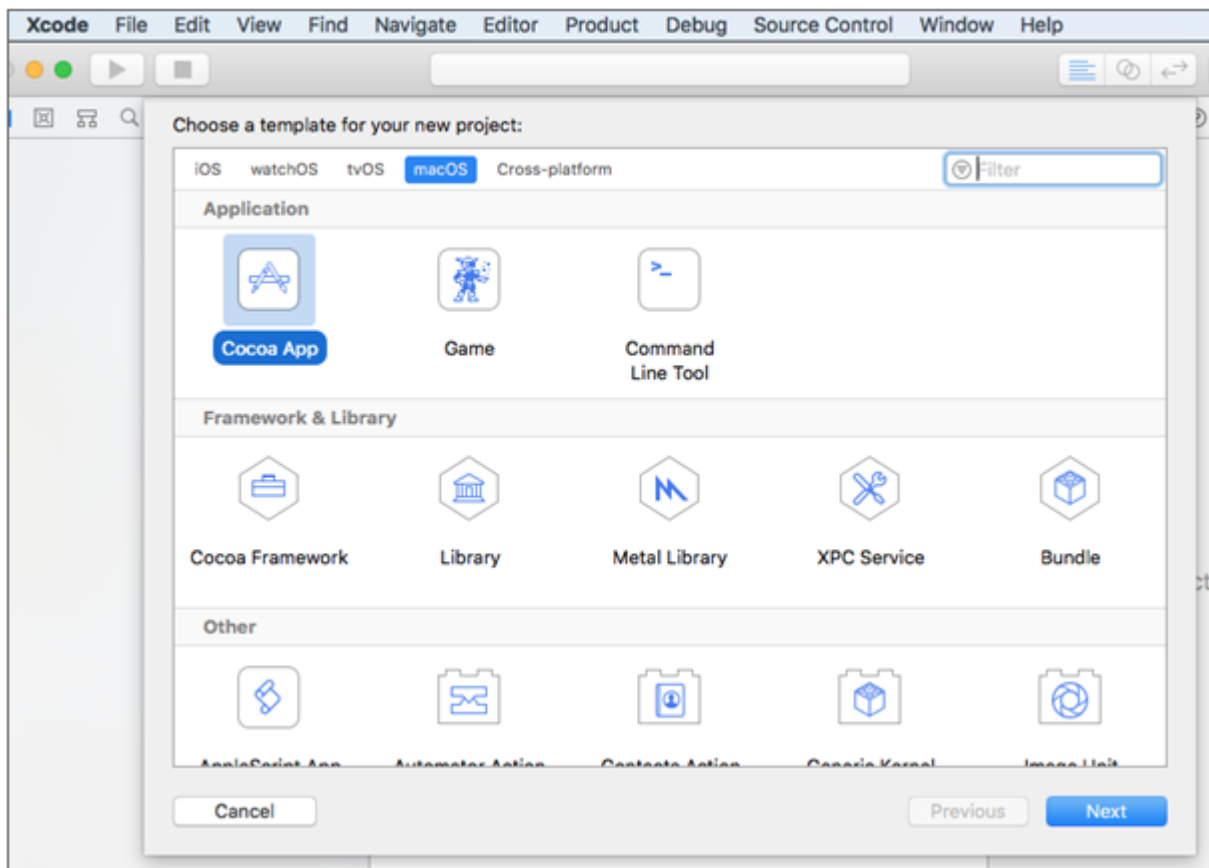
Xcode에서 아래와 같은 절차에 따라 macOS용 앱 프로젝트를 개발합니다. 앱 프로젝트를 진행하기 전에 넥사크로 스튜디오에서 개발된 앱에서 만들어진 아카이브 파일은 지정된 경로에 위치해야 합니다.

9.2.1 프로젝트 생성

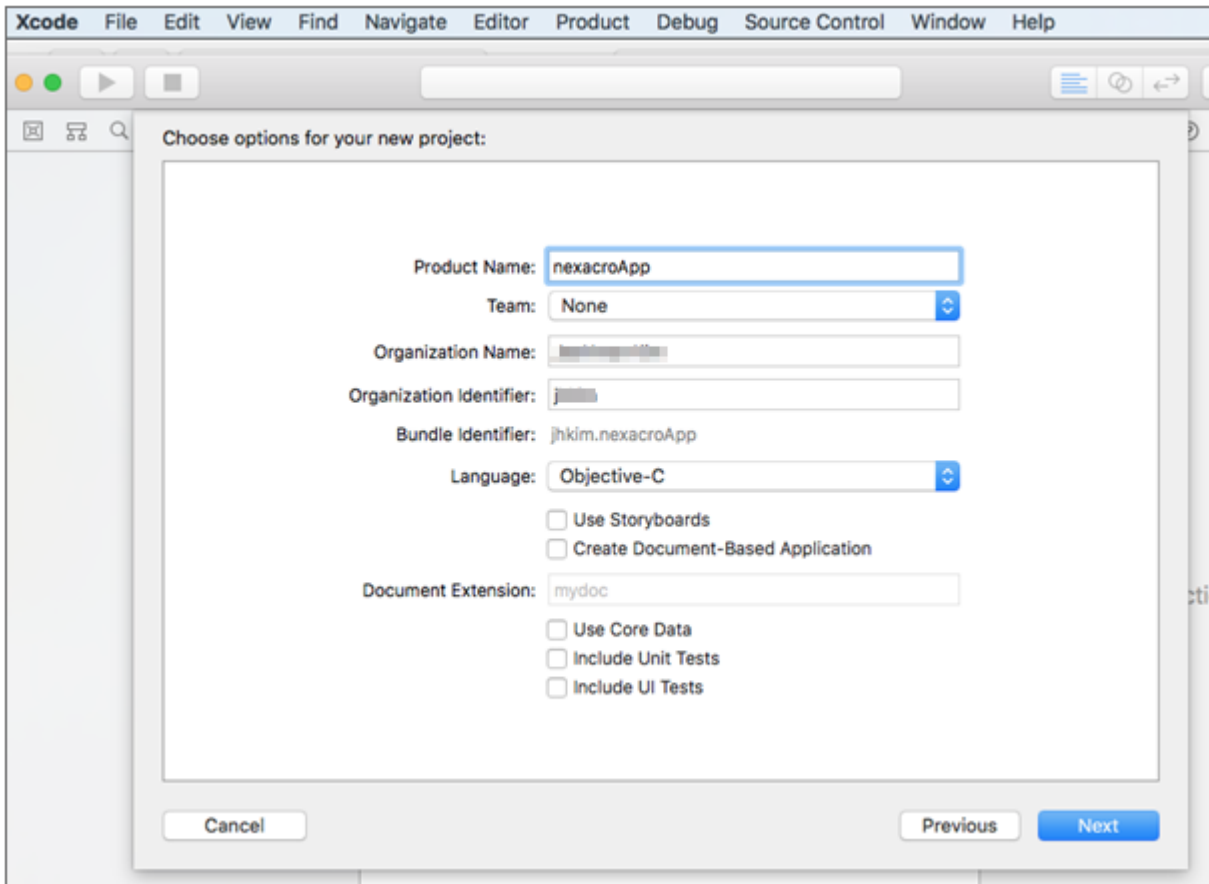
넥사크로로 개발된 앱을 담을 macOS 프로젝트를 생성하고 기본 환경을 설정해야 합니다. 새로운 프로젝트는 아래 메뉴에서 생성할 수 있습니다.

File > New > Project > macOS

프로젝트 생성을 위한 템플릿 화면에서 'Cocoa App' 항목을 선택합니다.



Language는 "Objective-C"로 설정하고 Product Name과 기타 필요한 정보를 입력하고 프로젝트를 생성할 폴더 위치를 지정한 후 [Create] 버튼을 클릭합니다.

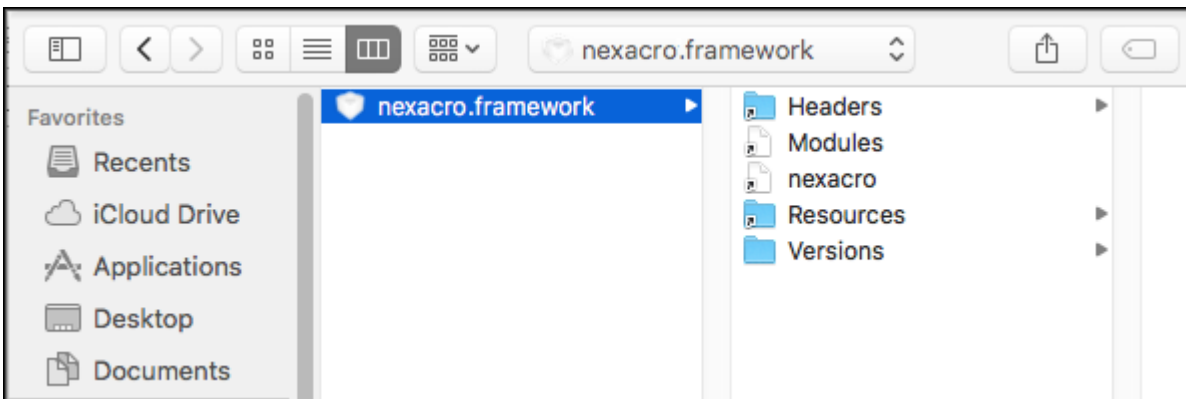


9.2.2 넥사크로 라이브러리 설정

macOS 프로젝트에서 넥사크로에 최적화된 환경을 만들기 위해 추가로 제공되는 넥사크로 라이브러리 파일을 설정합니다.

넥사크로 라이브러리는 압축 파일 형태로 제공되며 nexacro.framework.zip이라는 파일명으로 제공됩니다. 제공되는 파일은 압축을 풀어 생성된 프로젝트의 임의의 폴더에 끌어다 놓거나 [Add Files to] 메뉴를 통해 추가합니다.

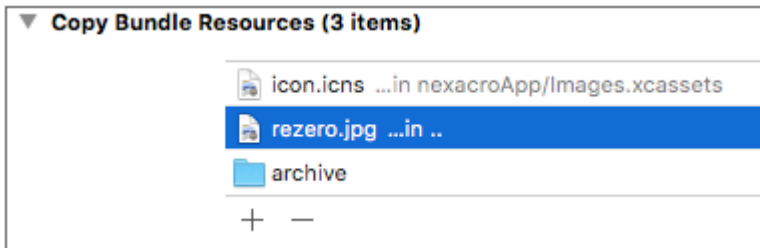
File > Add Files to "..."



9.2.3 리소스 설정

앱에서 사용할 로딩 이미지, 아이콘, 메시지, 레이아웃 등을 설정하는 단계입니다. 진행하는 프로젝트에 따라 변경해 적용할 수 있습니다.

Copy Bundle Resources 에 이미지 파일 추가 후 main.m 파일을 수정합니다.



```
startManager.splashImageName = @"rezero.jpg";
```

9.2.4 빌드 환경 설정



기본으로 만들어진 파일 중에 사용하지 않는 AppDelegate.h, AppDelegate.m, MainMenu.xib 파일은 삭제합니다.

main.m

Supporting Files 폴더 아래에 있는 main.m 파일을 아래와 같이 수정합니다.

```
// main.m

#import <Cocoa/Cocoa.h>
#import "nexacro/nexacro.h"

int main(int argc, const char * argv[]) {
    NexacroStartManager *startManager = [NexacroStartManager sharedManager];

    // 생략시 앱 이름으로 자동 설정
    startManager.applicationKey = @"";

    // 필수 입력값이며 디폴트 부트스트랩 파일명을 사용할 경우 start_macos.json 파일명은
```

생략가능

```

startManager.bootstrapUrl = @"http://172.10.12.180:8080/Mobile_Test/start_macos.json";

// 생략시 bootstrap파일이 있는 경로로 자동 설정
startManager.projectUrl = @"http://172.10.12.180:8080/Mobile_Test/";

// 초기 로딩화면 설정 (리소스 폴더를 참조함. 생략시 넥사크로 디폴트 로딩 화면을 사용함)
// startManager.splashImageName = @"sample.png";

[NSApplication sharedApplication];
[NSBundle loadNibFile:[startManager getNextacronibFile] externalNameTable:nil withZone:nil];
[NSApp run];

return EXIT_SUCCESS;
}

```

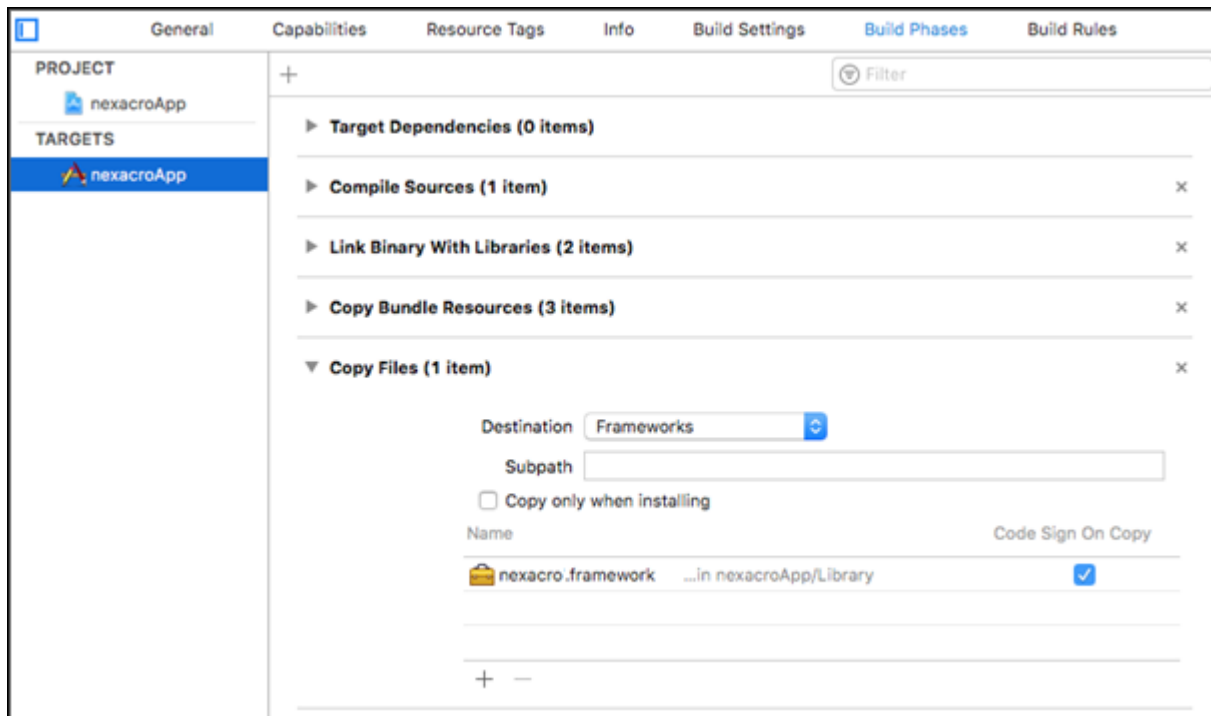
기타 설정

General	Deployment Target: 10.7 Main Interface: MainMenu를 삭제
Build Settings	Architectures: \$(ARCHS_STANDARD) macOS Deployment Target: macOS 10.7 Objective-C Automatic Reference Counting: Yes



2018년 출시될 macOS부터 32비트 앱 지원은 deprecated로 변경되고, 2019년부터는 iOS 11처럼 64비트 앱으로 만들어진 앱은 실행이 되지 않습니다.

- Build Phases - Copy Files' 항목에서 프레임워크 추가
(+) 클릭 후 > New Copy Files Phase > Destination: Frameworks
nexacro.framework 추가



- info 항목에서 아래 설정 추가 또는 변경
App Transport Security Settings > Allow Arbitrary Loads

▼ App Transport Security Settings	Dictionary	(1 item)
Allow Arbitrary Loads	Boolean	YES



Allow Arbitrary Loads 항목값을 "YES"로 설정하지 않으면 WebBrowser 컴포넌트가 화면에 표시되지 않습니다.

부록 A.

경로설정

A.1 Alias 경로

넥사크로는 사용자 PC의 경로설정 시 다음의 Alias 경로를 사용합니다.

Alias 경로 명	실제 경로명
%USERAPP%	%LOCALAPPDATA%LOW₩TOBESOFT₩Nexacro N₩24 (Windows) /Users/[UserName]/Library/TOBESOFT/Nexacro N/24 (macOS) /data/data/[PackageName]/files/NEXACRO/ (Android) /data/data/[PackageName]/files/[Key] (Android key setting) [AppRoot]/Library/Application Support/NEXACRO/ (iOS)
%UPDATE%	%USERAPP% ₩Update
%WEBDEPLOY SETUP%	Server Context root
%WEBDEPLOY PROJECT%	Server Context root
%WEBDEPLOY FRAMEWORK%	%WEBDEPLOY PROJECT% ₩nexacrolib₩framework
%WEBDEPLOY COMPONENT%	%WEBDEPLOY PROJECT% ₩nexacrolib₩component
%WEBDEPLOY RESOURCE%	%WEBDEPLOY PROJECT% ₩nexacrolib₩resources
%WEBDEPLOY THEME%	%WEBDEPLOY PROJECT% ₩_resource_ ₩_theme_
%SD_CARD%	/storage/emulated/0/Android/data/[PackageName]/files/

이 Alias 경로는 배포정보의 PATH나 응용프로그램에서 PATH 정보를 입력할 때, 사용할 수 있습니다.



설치 파일 배포 시 Setup Manager를 사용해 설치 경로를 수정한 경우에는 %NEXACRO% 실제 경로가 달라질 수 있습니다.

A.2 nexacro.xml

nexacro.xml 파일은 사용자별로 정보를 담기 위해 사용하는 XML 파일입니다. 파일 경로는 아래와 같습니다.

%USERAPP%

A.3 상대경로

A.3.1 Project 내 경로의 상대경로 지원여부

아래 표는 Project 내에서 주요경로의 종류 및 상대경로 지원여부, Service이용경로(Service::a.xfdl형태의 경로)의 지원여부를 정리한 것입니다.

File종류	항목	상대경로		Service이용경로 지원여부
		지원여부	기준경로	
Typedefinition	Component UpdateURL ❶	지원	TypeDefintion	지원
	Service 경로	지원	TypeDefintion	지원 불가능
GlobalVariable	Image	지원	GlobalVariable	지원
XADL	TypeDefinition ❷	지원	XADL	지원 불가능
	GlobalVariable	지원	XADL	지원
	EngineUrl ❸	지원	XADL	지원 불가능
	LiceneceUrl ❹	지원	XADL	지원 불가능
	Themeid	지원	XADL	지원

1. Component UpdateURL

Component를 Update하기 위한 서버 경로를 말하며 Tool에서 TypeDefinition Update Url을 의미합니다.

단, 이 경로의 Domain(예:http://a.b.c/path에서 a.b.c)이 XADL의 경로와 일치하지 않으면 Update 하지 않습니다. 만일, XADL이 Local 경로일 경우는 Domain정보에 상관없이 Update합니다.

2. TypeDefinition

Service의 경로를 말하며 Tool에서 TypeDefinition Services url을 의미합니다.

3. Engine 경로

Engine 업데이트를 위한 경로를 말하며 Tool의 ADL engineurl을 의미합니다. 단 XADL경로와 Domain이 일치하지 않으면 Update 하지 않습니다. XADL이 Local 경로일 때는 Domain에 상관없이 Update합니다.

4. 라이선스 경로

Typedefinition을 읽기 전에 라이선스를 체크하기 위한 경로입니다.

Tool의 ADL licenseurl을 의미합니다.

단, XADL경로와 Domain이 일치해야 합니다. XADL이 Local경로일 때는 Domain에 상관없이 라이선스 파일을 체크합니다.

부록 B.

예외상황

운영 과정에서 예외적으로 발생할 수 있는 상황에 대한 대처 방안을 정리한 것입니다. 사용 환경에 따라 적용 방식이 달라질 수 있으니 참고로만 활용하시기 바랍니다.

B.1 웹브라우저 옵션

사용자가 웹브라우저에서 특정한 목적으로 옵션을 변경했을 경우에 앱에서 해당 기능을 사용하고 있다면 정상적인 동작을 하지 못할 수 있습니다. 주로 아래와 같은 상황에서 문제가 생길 수 있습니다.

B.1.1 자바스크립트 활성화

사용자가 웹브라우저에서 자바스크립트를 사용할 수 없도록 설정했을 경우에는 앱이 정상적으로 동작하지 않습니다. 일반적으로 기본 옵션은 활성화 상태이지만 사용자가 다른 옵션을 선택할 수 있으므로 앱이 화면에 보이지 않을 때는 해당 옵션을 먼저 확인합니다.

사용하는 웹브라우저에 따라 자바스크립트 사용 옵션을 활성화해주어야 합니다. 사용하는 버전에 따라 해당 옵션은 달라질 수 있으며 상세한 내용은 아래 링크 또는 각 웹브라우저 도움말을 참고하세요.

<http://www.enable-javascript.com>

B.1.2 파일 다운로드 활성화

사용자가 웹브라우저에서 파일 다운로드를 사용할 수 없게 제한한 경우에는 Filddownload 컴포넌트를 사용해 파일을 내려받을 수 없습니다. 일반적으로 기본 옵션은 활성화 상태이지만 사용자가 다른 옵션을 선택할 수 있으므로 파일 다운로드가 동작하지 않을 때는 해당 옵션을 먼저 확인합니다.

사용하는 웹브라우저에 따라 파일 다운로드 옵션을 활성화해주어야 합니다. 사용하는 버전에 따라 해당 옵션은 달라

질 수 있으며 상세한 내용은 각 웹브라우저 도움말을 참고하세요.

부록 C.

NRE 로그 확인하기

사용자 환경에서 앱이 예상치 못하게 종료되거나 실행중 발생하는 오류나 현상을 분석하기 위해 로그 파일을 생성하고 문제를 확인할 수 있습니다.



로그 기능은 윈도우, macOS 운영체제 NRE만 지원합니다.



앱 실행 시 로그 관리자가 실행되기 전에 발생하는 에러는 로그로 남길 수 없습니다. 앱 실행 시 alert 창 형태로 사용자에게 표시됩니다.

C.1 로그 파일 생성 설정하기

로그 파일을 생성하는 기능은 기본 설정이 아니며 추가 설정이 필요합니다. 로그 파일 생성 설정은 넥사크로 스튜디오에서 설정할 수도 있고 config 파일을 설정하는 방법이 있습니다.

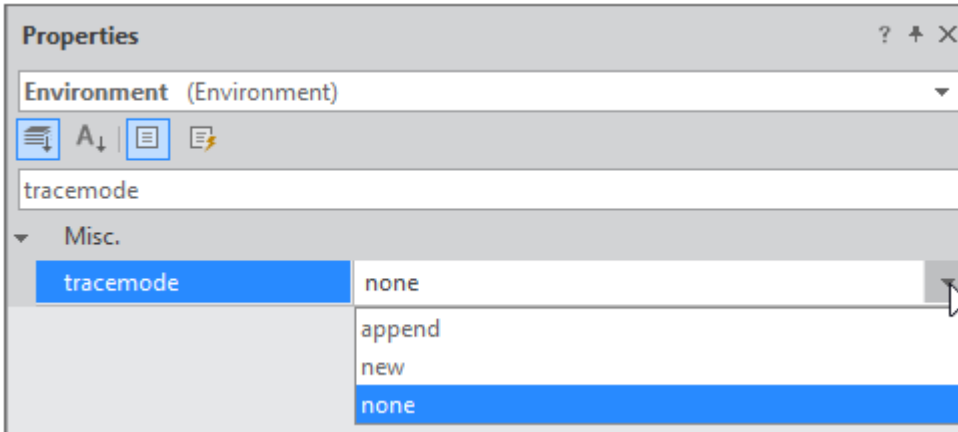
앱 실행 시에는 config 파일에 설정한 내용을 우선적으로 적용합니다. config 파일이 있는 경우에는 Environment 항목에서 설정한 값은 무시합니다.

C.1.1 Environment 항목에서 로그 파일 생성 설정하기

넥사크로 스튜디오에서 속성값을 설정하고 로그 기능을 설정할 수 있습니다. 관련 속성값은 넥사크로 스튜디오에서만 설정할 수 있으며 앱 실행중에는 변경할 수 없습니다.

tracemode 속성에서 로그 파일 생성 설정하기

넥사크로 스튜디오에서 Environment 항목을 선택하고 tracemode를 변경합니다. 기본값 "none"인 경우에는 로그 기능을 처리하지 않으며 속성값을 "new" 또는 "append"로 설정하면 로그 파일을 생성합니다.



로그 작성 수준과 주기 설정하기

넥사크로 스튜디오에서 Environment 항목을 선택하고 loglevel, traceduration 속성을 설정할 수 있습니다. loglevel 속성은 로그의 어떤 상황에서 로그를 기록할 것인지를 설정하고 traceduration 속성은 로그 파일 생성 주기를 설정합니다.

C.1.2 config 파일 설정하기

config 파일에서 로그 기능을 설정할 수 있습니다. Environment 속성은 수정하지 않고 config 파일을 사용자 실행 환경에 복사해 넣거나 빌드 시 포함해서 설정할 수 있습니다.

-NC 옵션 설정하기

윈도우 운영체제에서 nexacro.exe 파일을 실행할 때 "-NC" 옵션을 사용할 수 있습니다. 실행할 nexacro.exe 파일과 같은 위치에 있는 config 파일을 설정할 수 있습니다.

예를 들어 nexacro.exe 파일의 위치가 아래와 같은 경로 -NC 옵션값으로 "log_config.xml" 파일을 지정했다면 같은 위치에 "log_config.xml" 파일이 있어야 합니다.

```
C:\Program Files\TOBESOFT\Nexacro N\21\TEST_LOG\nexacro.exe
C:\Program Files\TOBESOFT\Nexacro N\21\TEST_LOG\log_config.xml
```

-NC 옵션을 사용하지 않는 경우에는 기본 경로에 있는 nexacro_config.xml 파일을 찾고 해당 파일이 있는 경우 참조합니다.

nexacro_config.xml 파일 설정하기

-NC 옵션 설정하기는 파일명을 지정할 수 있지만 별도 옵션을 지정해주어야 합니다. nexacro_config.xml 파일은 지정한 위치에 정해진 이름을 가진 파일을 가져다놓으면 자동으로 앱 실행 시 이를 인식하고 처리합니다.

윈도우 운영체제

윈도우 운영체제에서는 nexacro_config.xml 파일을 포함해 앱을 배포하는 기능을 포함하지 않습니다. 아래 경로에 nexacro_config.xml 파일을 생성하고 앱 실행 시 참조할 수 있습니다.

```
%USERAPP%\Config\[Key]\nexacro_config.xml
```



[Key]는 nexacro.exe 실행 옵션에 지정하는 -K 옵션 또는 Build App 단계에서 지정하는 App Title입니다.

macOS 운영체제

아래 내용을 참고하세요.

Config 설정

C.2 nexacro_config.xml 파일 속성

설정 파일(nexacro_config.xml) 파일 또는 -NC 옵션으로 지정하는 파일은 아래와 같은 형식으로 작성합니다.

```
<?xml version="1.0" encoding="UTF-8"?>
<nexacro-config os="windows">
  <application
    file-logging="true"
    loglevel="debug"
    tracemode="new"
    traceduration="-1"/>
  <log
    filepath="%USERAPP%\Log\nre_log_test.log"
    filesize="408960"
    backupfilecount="5"/>
</nexacro-config>
```

nexacro_config.xml 파일에 지정하는 속성 중 로그 파일 생성과 관련된 속성만 아래에 설명합니다.

태그/속성	설명
nexacro-config	
os	운영체제를 설정하는 속성입니다. - windows: 윈도우 운영체제 - Android: 안드로이드 운영체제 - osx: macOS 운영체제
application	
file-logging	초기 로딩 에러 정보 저장 여부를 설정하는 속성입니다. %USERAPP% 경로에 nexacro_startError.log 파일에 기록되는 정보와 같음
loglevel	로그 출력 레벨을 설정하는 속성입니다. fatal error warn info debug fatal 부터 info 까지는 프레임워크 내부에서 처리하는 로그 정보이며 trace 메소드를 사용하거나 스크립트 에러가 발생하는 경우에는 debug로 처리됩니다.
tracemode	로그 파일 처리 방식을 설정하는 속성입니다. new: 이전 기록을 삭제하고 새롭게 로그 기록 append: 이전 기록 뒤에 로그 기록
traceduration	로그 파일을 일별로 생성하는 옵션을 설정하는 속성입니다. 기본값은 -1입니다.
log	
filepath	로그 파일 경로와 파일명을 설정하는 속성입니다. 경로를 지정하지 않으면 아래 경로에 파일을 생성합니다. - Windows: %USERAPP% - Android: %SD_CARD%/Android/data/[package name]/files/nexacro/21/Log/ - macOS: %USERAPP%/Log 파일명을 설정하지 않으면 아래와 같은 형식으로 파일을 생성합니다. - nexacro_[프로토콜+url+port][부트스트랩파일명].log ex) nexacro_httplocalhost8080startjson.log
filesize	로그 파일 최대 크기를 설정하는 속성입니다. 기본값은 204,800 바이트(byte)입니다. 기본값보다 작은 값을 설정한 경우에는 기본값이 적용됩니다.
backupfilecount	로그 파일 최대 개수를 설정하는 속성입니다. 로그 파일 크기가 filesize 보다 커지는 경우 백업 파일로 남길 수 있는 파일 개수 기본값은 1개입니다.



os 속성값이 "windows"인 경우 filepath 속성값은 보안상 문제가 될 수 있어 아래와 같은 제약을 적용합니다.

- 허용된 경로 또는 허용된 경로의 하위 경로가 아닌 경우 기본 경로로 강제 변경합니다.
허용된 경로는 system.convertRealPath 메소드에서 Alias를 사용해 지정할 수 있는 경로입니다.
- 시작 프로그램 경로와 같은 위치를 지정하면 기본 경로로 강제 변경합니다.
- 파일 확장자가 아래와 같은 경우 "log"로 확장자를 강제 변경합니다.

bat, bin, cmd, com, cpl, exe, gadget, inf1, ins, inx, isu, job, jse, lnk, msc, msi, msp, mst, paf, pif, ps1, reg, rgs, scr, sct, shb, shs, u3p, vb, vbe, vbs, vbscript, js, ws, wsf, wsh, dll, hta

C.3 속성별 파일 생성 예시

C.3.1 backupfilecount

생성된 로그파일 크기가 filesize 속성값보다 커지면 백업파일을 추가로 생성합니다.

파일명 뒤에 번호를 붙여서 파일을 생성합니다. backupfilecount 속성으로 지정한 값에 도달하게 되면 처음 생성한 파일을 지우고 다시 로그를 기록합니다.

1시간마다 앱이 실행되고 앱 실행 시 300KB 로그를 기록한다면 로그 파일은 아래와 같이 생성됩니다.

```
<log filepath="%USERAPP%nrelog.log" filesize="200" backupfilecount="3"/>
```

로그파일명	파일크기	파일생성/변경 시간	작업순서
2020/05/26 13:00			
nrelog.log	100KB	2020/05/26 13:00	마지막 로그
nrelog.log.1	200KB	2020/05/26 13:00	시작 로그
2020/05/26 14:00			
nrelog.log	10KB	2020/05/26 14:00	마지막 로그
nrelog.log.1	200KB	2020/05/26 14:00	
nrelog.log.2	200KB	2020/05/26 14:00	
nrelog.log.3	200KB	2020/05/26 13:00	시작 로그
2020/05/26 15:00			
nrelog.log	180KB	2020/05/26 15:00	마지막 로그
nrelog.log.1	200KB	2020/05/26 14:00	
nrelog.log.2	200KB	2020/05/26 14:00	
nrelog.log.3	200KB	2020/05/26 15:00	시작로그 / roll over

C.3.2 traceduration

로그 파일이 생성되는 날짜 기준으로 traceduration 속성값보다 이전에 생성된 파일은 삭제합니다.

파일명 뒤에 날짜("_yyyymmdd")를 붙여서 파일을 생성합니다. 날짜 비교는 운영체제 시스템 시간을 기준으로 처리합니다.

하루에 한 번 앱이 실행되고 주말에는 실행하지 않는다면 로그 파일은 아래와 같이 생성되고 삭제됩니다. 주말(토요일, 일요일)에는 앱이 실행되지 않으므로 기존에 생성된 로그 파일이 삭제되지 않으며 월요일에 앱이 실행되면서 traceduration 속성값 이전 로그 파일을 삭제합니다.

```
<nexacro-config os="windows">
  <application file-logging="true" quiet="false" loglevel="debug" tracemode="new"
    traceduration="2"/>
  <log filepath="%USERAPP%nrelog.log" filesize="408960" backupfilecount="3"/>
</nexacro-config>
```

로그파일명	파일생성일	작업
2020/05/21 (목요일)		
nrelog_20200521.log	2020/05/21	파일 생성
2020/05/22 (금요일)		
nrelog_20200521.log	2020/05/21	유지
nrelog_20200522.log	2020/05/22	파일 생성
2020/05/23 (토요일)		
nrelog_20200521.log	2020/05/21	유지
nrelog_20200522.log	2020/05/22	유지
2020/05/24 (일요일)		
nrelog_20200521.log	2020/05/21	유지
nrelog_20200522.log	2020/05/22	유지
2020/05/25 (월요일)		
nrelog_20200521.log	2020/05/21	파일 삭제
nrelog_20200522.log	2020/05/22	파일 삭제
nrelog_20200525.log	2020/05/25	파일 생성
2020/05/26 (화요일)		
nrelog_20200525.log	2020/05/25	유지
nrelog_20200526.log	2020/05/26	파일 생성
2020/05/27 (수요일)		
nrelog_20200525.log	2020/05/25	파일 삭제
nrelog_20200526.log	2020/05/26	유지

로그파일명	파일생성일	작업
2020/05/21 (목요일)		
nrelog_20200527.log	2020/05/27	파일 생성

C.4 로그 파일 읽기

해당 운영체제 파일 탐색 프로그램을 사용해 로그 파일을 확인할 수 있습니다.

앱 내에서 로그 파일을 확인하는 기능을 구현하려면 로그 파일이 저장된 경로를 가져와서 VirtualFile 오브젝트를 사용해 파일 내용을 읽을 수 있습니다.

C.4.1 로그 파일 읽기 스크립트 예제

Button 클릭 시 해당 경로에 저장된 첫 번째 파일을 가져와 TextArea 컴포넌트에 표시하는 예제입니다.

```

this.Button01_onclick = function(obj:Button, e:nexacro.ClickEventInfo)
{
    var strLogpath = nexacro.getLogFilePath();
    this.VirtualFile00.getFileList( strLogpath, ".log", VirtualFile.findFileOnly );
}

this.VirtualFile00_onsuccess = function(obj:VirtualFile, e:nexacro.VirtualFileEventInfo)
{
    switch (e.reason)
    {
        case 1:    // open
            this.VirtualFile00.read();
            break;
        case 3:    // read
            this.TextArea00.value = e.textdata;
            break;
        case 8:    // getFileList
            var logfilename = logpath + e.fileattributelist[0].filename;
            this.VirtualFile00.open(logfilename, VirtualFile.openText);
            break;
    }
}

```

```
}  
}
```

C.4.2 윈도우 운영체제 실행 시 제약

윈도우 운영체제에서 로그 파일에 접근하려면 Environment filesecurelevel 속성값을 "all"로 지정해주어야 하며 최초 실행 시 샌드박스 창이 표시됩니다.

부록 D.

NRE 원격 디버깅

D.1 NRE 원격 디버깅

NRE 실행 시 크롬, Edge 브라우저의 Devices inspect 기능을 사용해 실행 중인 앱 디버깅을 할 수 있습니다. 윈도우, macOS 운영체제를 지원합니다.



개발한 스크립트와 자바스크립트 라이브러리까지 디버깅을 할 수 있습니다. DeviceAdaptors로 설치한 모듈이나 엔진 내부 코드 디버깅은 지원하지 않습니다.

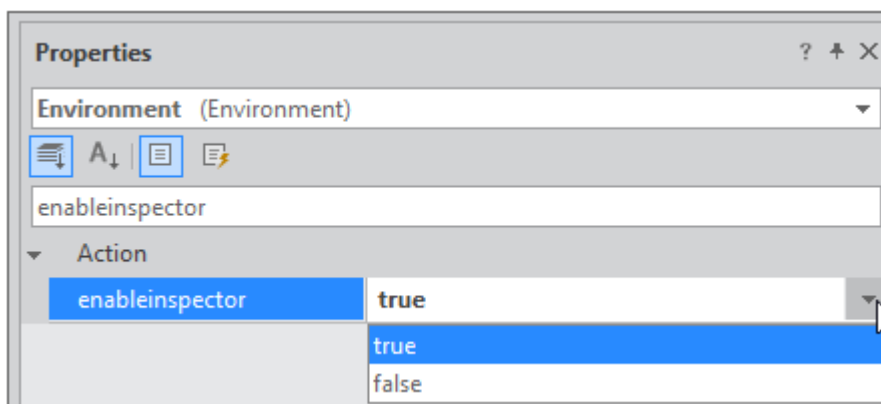


안드로이드 운영체제 NRE는 WebView 디버깅 기능을 사용합니다. 아래 문서를 참고해주세요.

[https://developer.android.com/reference/android/webkit/WebView#setWebContentsDebuggingEnabled\(boolean\)](https://developer.android.com/reference/android/webkit/WebView#setWebContentsDebuggingEnabled(boolean))

D.1.1 프로젝트 설정

Environment의 enableinspector 속성값을 true로 변경합니다.



D.1.2 실행 설정

윈도우

nexacro.exe 실행 옵션으로 -RPP 옵션을 아래와 같이 추가하고 실행합니다.

```
nexacro.exe -K "INSPECTOR_TEST" -S "http://localhost:8080/start.json" -RPP 127.0.0.1:9223
nexacro.exe -K "INSPECTOR_TEST" -S "http://localhost:8080/start.json" -RPP 9223
```



RPP 옵션 설정 시 PORT만 설정한 경우에는 localhost 로 동작합니다.

macOS

터미널에서 실행 시 -RPP 옵션을 아래와 같이 추가하고 실행합니다.

```
open nexaApp.app --args -K INSPECTOR_TEST -S http://localhost:8080/start_macos.json -RPP
"127.0.0.1:9223"
open nexaApp.app --args -K INSPECTOR_TEST -S http://localhost:8080/start_macos.json -RPP
"9223"
```

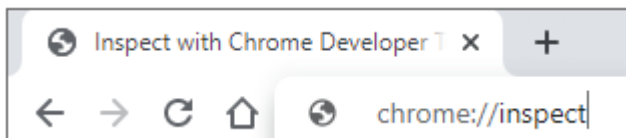


RPP 옵션 설정 시 PORT만 설정한 경우에는 localhost 로 동작합니다.

D.1.3 Devices inspect 기능 실행

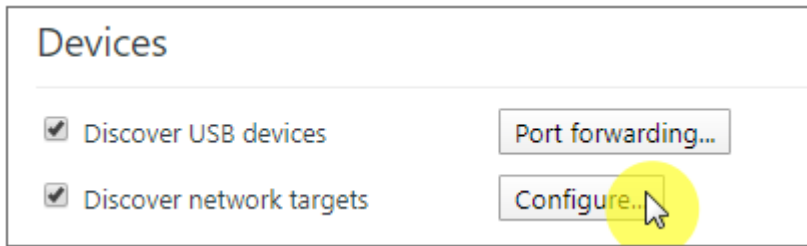
- 1 크롬 브라우저를 실행하고 주소창에 "chrome://inspect" 을 입력합니다.

chrome://inspect

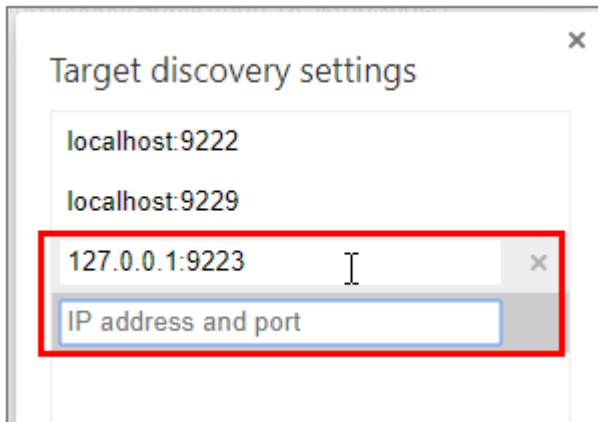


- 2 Configure 버튼을 클릭하고 Target discovery settings 창을 띄웁니다.

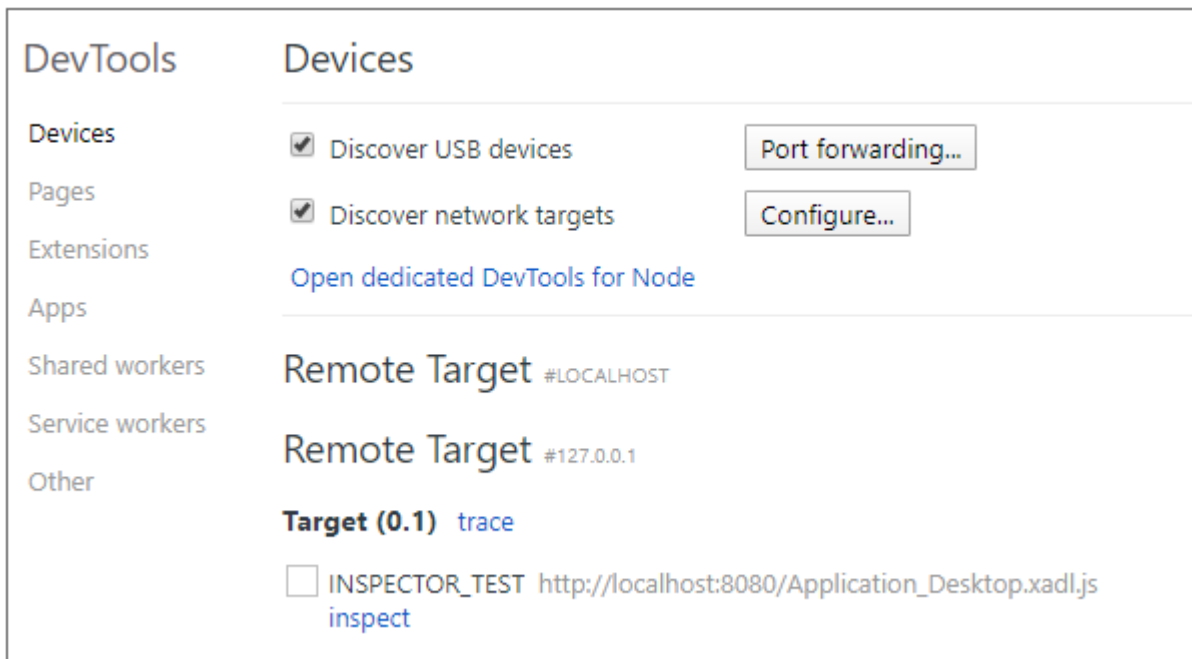
Discover network targets 항목이 체크되어 있지 않다면 해당 항목을 체크해줍니다.



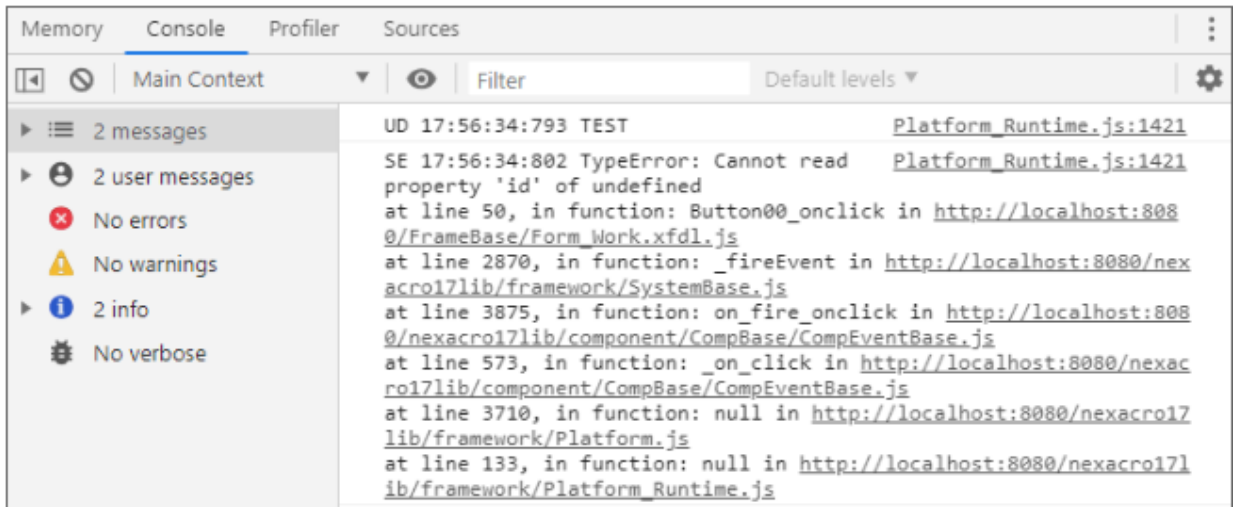
- 3 Target discovery settings 창에 앞에서 설정한 RPP 주소값을 추가합니다.



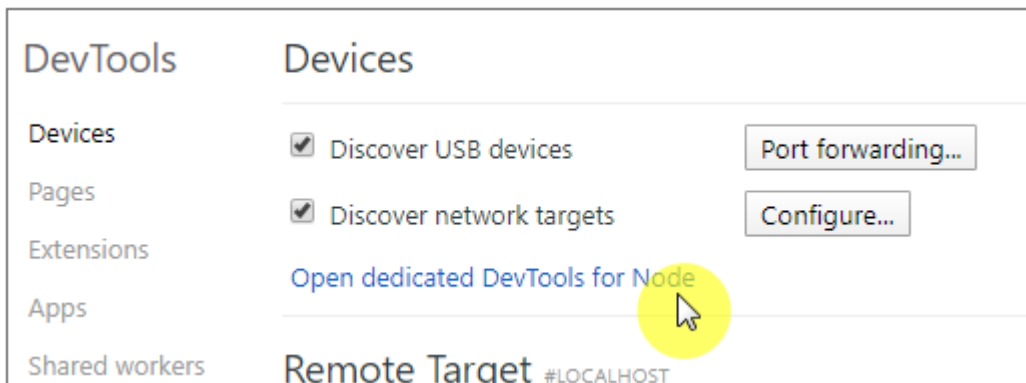
- 4 앱을 실행하면 Remote Target 항목에 실행된 앱 key 값이 표시됩니다.



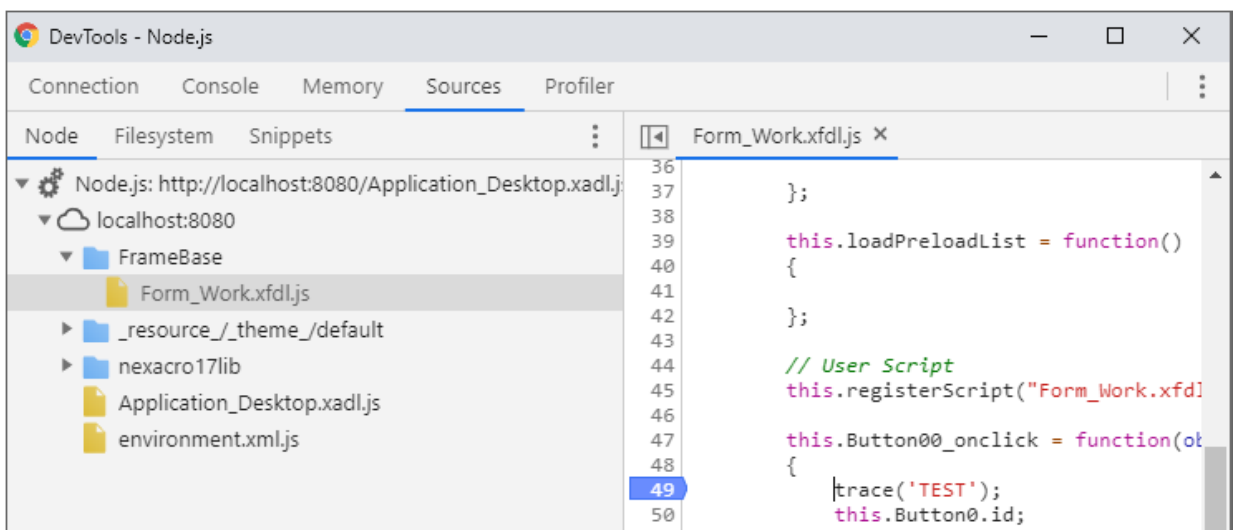
- 5 Target 항목 아래에 있는 inspect 링크를 클릭하면 콘솔창으로 이동하며 trace 메소드 실행 결과나 오류 발생 시 메시지를 확인할 수 있습니다.



- ⑥ open dedicated devtools for node 링크를 클릭하면 서버 리소스를 탐색할 수 있는 창을 띄울 수 있습니다.



[Sources > Node] 탭에서 원하는 파일을 선택하고 중단점을 추가하거나 추가적인 디버깅 작업을 수행할 수 있습니다.



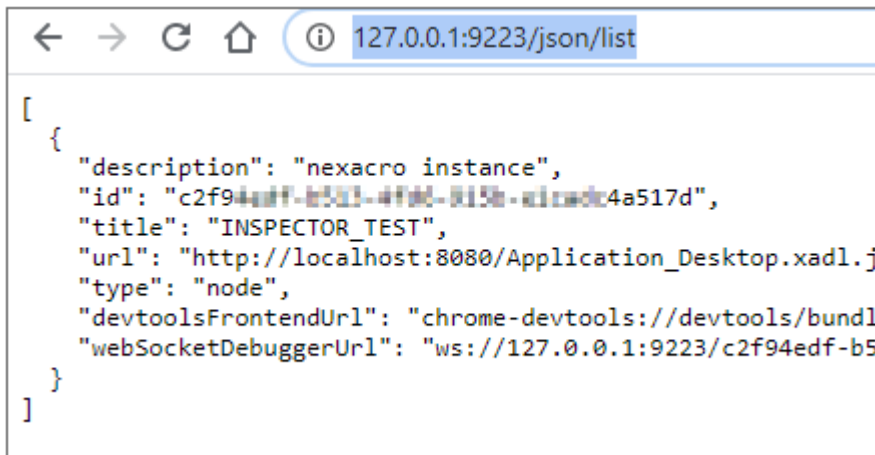
Remote Target 항목에 앱이 보이지 않을때

앱 실행 후 20초 이상 앱이 표시되지 않는다면 아래와 같이 조치합니다.

- ① 브라우저 종료 후 다시 접속합니다.
- ② 앱에서 inspect 기능이 정상 실행되고 있는지 확인합니다.

앱 실행 후 브라우저에서 아래 URL로 접속하면 정보가 표시되어야 합니다. 응답이 없다면 프로젝트 설정에 문제가 있거나 다른 문제가 있을 수 있습니다.

```
[Target discovery settings RPP Address]/json/list
http://127.0.0.1:9223/json/list
```



D.1.4 로딩 중 멈춤 기능 옵션 설정

NRE 원격 디버깅은 실행 중인 앱을 접속해서 디버깅을 실행하기 때문에 로딩 중 발생하는 오류나 현상을 디버깅할 수 없습니다. 아래와 같이 추가적인 옵션을 통해 로딩 시점에 강제로 멈추는 설정을 추가할 수 있습니다.



로딩 중 멈춤 기능은 테마 파일(theme.map.js) 수신 완료 시점에서 멈춘 상태로 Devices inspect 연결을 기다립니다. 프로젝트 로딩 방식이나 네트워크 상태에 따라 해당 시점에 일부 파일을 내려받지 않은 상태일 수 있으며 내려받지 않은 파일은 디버깅을 처리할 수 없습니다.

윈도우, macOS

nexacro.exe 실행 옵션으로 -brk "true" 옵션을 아래와 같이 추가하고 실행합니다.

```
nexacro.exe -K "INSPECTOR_TEST" -S "http://localhost:8080/start.json" -RPP 127.0.0.1:9223 -brk  
"true"  
nexacro.exe -K "INSPECTOR_TEST" -S "http://localhost:8080/start.json" -RPP 9223 -brk "true"
```

macOS

터미널에서 실행 시 -brk "true" 옵션을 아래와 같이 추가하고 실행합니다.

```
open nexaApp.app --args -K INSPECTOR_TEST -S http://localhost:8080/start_macos.json -RPP  
"127.0.0.1:9223" -brk "true"  
open nexaApp.app --args -K INSPECTOR_TEST -S http://localhost:8080/start_macos.json -RPP  
"9223" -brk "true"
```

부록 E.

Cordova 플러그인 적용

Cordova 플러그인을 적용한 앱을 빌드할 수 있습니다.

E.1 넥사크로 스튜디오

- 1 넥사크로 스튜디오에서 Cordova 플러그인을 설치합니다.
- 2 Cordova 플러그인을 설치하고 Packing 작업을 진행합니다.
- 3 Project Explorer에서 TypeDefinition > CordovaPlugins 항목을 선택하고 컨텍스트 메뉴에서 Export를 실행합니다.

E.2 Android - Android Studio

- 1 넥사크로 스튜디오에서 Export 한 파일 폴더를 앱 프로젝트 경로에 맞추어 복사합니다.
- 2 플러그인 plugin.xml 파일과 가이드 문서를 참조해 plugin_config.xml 파일에 등록합니다.

예를 들어 cordova-plugin-statusbar의 경우 plugin.xml 파일은 아래와 같습니다.

```
<platform name="android">
  <source-file src="src/android/StatusBar.java" target-dir="src/org/apache/cordova/statusbar" />

  <config-file target="res/xml/config.xml" parent="/*">
    <feature name="StatusBar">
      <param name="android-package" value="org.apache.cordova.statusbar.StatusBar" />
      <param name="onload" value="true" />
    </feature>
    <preference name="StatusBarOverlaysWebView" value="true" />
  </config-file>
</platform>
```

name, class 항목은 plugin.xml 파일의 feature name 항목과 android-package 항목을 참고합니다.

```
...
<plugin name="StatusBar" class="org.apache.cordova.statusbar.StatusBar" onload="true"/>
<preference name="StatusBarBackgroundColor" value="#FFFF00" />
...
```

preference 항목은 각 플러그인에서 제공하는 문서를 참고합니다. cordova-plugin-statusbar에서 제공하는 문서는 아래와 같습니다.

| <https://github.com/apache/cordova-plugin-statusbar/tree/master#preferences>

E.3 iOS/iPadOS - Xcode

- ① 넥사크로 스튜디오에서 Export 한 파일 폴더를 압축해서 Xcode가 설치된 장비로 복사한 후 압축을 풀어줍니다.
- ② 플러그인 plugin.xml 파일과 가이드 문서를 참조해 plugin_config.xml 파일에 등록합니다.
예를 들어 cordova-plugin-statusbar의 경우 plugin.xml 파일은 아래와 같습니다.

```
<platform name="ios">
  <config-file target="config.xml" parent="/*">
    <feature name="StatusBar">
      <param name="ios-package" value="CDVStatusBar" />
      <param name="onload" value="true" />
    </feature>
    <preference name="StatusBarOverlaysWebView" value="true" />
    <preference name="StatusBarStyle" value="lightcontent" />
  </config-file>

  <header-file src="src/ios/CDVStatusBar.h" />
  <source-file src="src/ios/CDVStatusBar.m" />
</platform>
```

name, class 항목은 plugin.xml 파일의 feature name 항목과 ios-package 항목을 참고합니다.

```
...
<plugin name="StatusBar" class="CDVStatusBar" onload="true"/>
<preference name="StatusBarOverlaysWebView" value="false" />
<preference name="StatusBarBackgroundColor" value="#FFFF00" />
...
```

preference 항목은 각 플러그인에서 제공하는 문서를 참고합니다. cordova-plugin-statusbar에서 제공하는 문서는 아래와 같습니다.

| <https://github.com/apache/cordova-plugin-statusbar/tree/master#preferences>

- ③ Xcode에서 프로젝트를 선택하고 컨텍스트 메뉴에서 [Add Files to] 항목을 선택하고 넥사크로 스튜디오에서 Export 한 파일 폴더 안에 있는 파일을 추가합니다.

예를 들어 cordova-plugin-statusbar의 경우는 아래와 같습니다.

| CDVStatusBar.h
| CDVStatusBar.m

- ④ 복사한 파일 내 import 구문 경로 앞에 "nexacro/"를 추가해 수정합니다.

```
#import <Cordova/CDVPlugin.h>
>
#import <nexacro/Cordova/CDVPlugin.h>
```